Fully Adaptive Threshold Blind Signature Without AGM

Shaolong Tang[1], Peng Jiang[1], Fuchun Guo[2], Willy Susilo[2], and Liehuang Zhu[1]

[1] Beijing Institute of Technology, Beijing, China
{shaolongtang12, pennyjiang0301}@gmail.com, liehuangz@bit.edu.cn
Corresponding author is Peng Jiang.
[2] University of Wollongong, Wollongong, NSW, Australia
{fuchun,wsusilo}@uow.edu.au

Abstract. Threshold blind signatures (TBS) allow any set of issuers whose size exceeds a predefined threshold to jointly generate a signature without learning the message. Adaptively secure TBS schemes allow the adversary to corrupt issuers at any point during protocol execution, capturing realistic threat models. Adaptive security methods rely on the algebraic group model (AGM) in security proofs to extract the discrete logarithm of the blinded protocol message. However, as a strong idealized assumption, AGM requires the adversary, upon outputting any group element, to also provide an explicit linear representation in terms of previously seen group elements. Constructing an adaptively secure TBS scheme without AGM remains challenging.

In this paper, we propose Rainblind, the first TBS scheme achieving adaptive security without AGM. The core idea is to apply OR compilation between a discrete logarithm equality (DLEQ) statement and a decisional Diffie-Hellman (DDH) tuple statement such that the security proof does not need the AGM to extract the discrete logarithm of the blinded protocol message and instead only needs to extract the group element. Concretely, the DLEQ statement concerns the secret key, while the DDH-tuple statement is derived from a hash function. In real execution, it fails to satisfy the DDH-tuple statement by the uniform hash randomness, forcing the DLEQ branch. In security proof, by programming OR compilation into the DDH-tuple branch, we can leverage random elements to replace the DLEQ elements during the signing queries. To extract the group element, we design a non-interactive proof system $\mathsf{NIPS}_{\mathsf{ped}}$. Rainblind is built from the tagged linear function (TLF), so that the proof can answer adaptive corruption queries via the inversion oracle in the t-algebraic translation resistance assumption of TLF, which implies DDH assumption. We also provide a plain signature $\mathsf{Sig}_{\mathsf{m}}$ and a blind signature BS_{m} for transitioning to Rainblind.

Keywords: Threshold Blind Signature · Adaptive Security

1 Introduction

Blind signatures (BS), first proposed by Chaum [1], enable the user to get a valid signature on a message while maintaining message confidentiality from the

signer's view. Blind signatures are the building blocks of many applications, such as e-voting [2], anonymous credentials [3,4,5], Apple's iCloud Private Relay [6], and Google One's VPN Service [7]. To address the critical vulnerability of single signer's compromise, Crites et al. [8] proposed threshold blind signatures (TBS) by distributing the signer's operations to multiple issuers. Threshold blind signatures have been used in threshold issuance scenarios [9,10,11]. In a (t, n) TBS scheme, the secret sharing mechanism is used to distribute the secret key among n issuers. At least $t + 1$ issuers must collaborate to generate a valid constant-size signature, independent of t and n. The scheme remains secure when up to t issuers fall under adversarial control.

Recent progress. Crites et al. [8] defined OMUF-SB, the one-more unforgeability model, where the adversary can act as both user and issuers. Before protocol execution, the adversary can corrupt c ($c \leq t$) issuers to obtain their secret keys. As the user, the adversary can query the signing oracle $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_k}$ for the k-th round. If the adversary returns $l + 1$ valid message-signature pairs against the common public key PK, where l counts legitimately obtained signatures, it wins the one-more unforgeability game. The counter l increases when any honest issuer generates a partial signature for the same session identifier sid and the issuer set $\mathcal{SS}$. Moreover, Crites et al. proposed a TBS scheme Snowblind and proved its OMUF-SB security in the random oracle model (ROM) and algebraic group model (AGM). Lehmann et al. [12] proposed a detailed hierarchy, defining which interactions yield the trivial signatures (i.e., how the counter l increases). They established security models OMUF-x for $\mathsf{x} \in \{0, 1, 2, 3, \mathsf{FC}\}$, each with different policies for incrementing trivial signatures, allowing the issuers to gain more control over the combination of their partial signatures. They also proposed an interactive scheme Snowblind+ and a non-interactive (one-round) scheme tBlindBLS-3, both proved to achieve OMUF-3 security in the ROM. In the post-quantum setting, Faller et al. [13] proposed a lattice-based threshold blind signature scheme TBS_B and proved its OMUF-3 security in the ROM.

Static security vs. adaptive security. The above security models OMUF-x for $\mathsf{x} \in \{0, 1, 2, 3, \mathsf{FC}, \mathsf{SB}\}$ apply only to static corruption [14], i.e., the adversary must specify corrupted issuers before making any signing queries, which restricts the adversary's flexibility in real-world attacks. For adaptive corruption, the adversary can dynamically select targets for subsequent corruption based on obtained signatures and secret keys of the issuers. We emphasise that the distinction among the different security models OMUF-x for $\mathsf{x} \in \{0, 1, 2, 3, \mathsf{FC}, \mathsf{SB}\}$ is in the counting of legitimate signatures, and the gap between static and adaptive corruption is independent of how l is counted. The threshold blind signature schemes Snowblind, Snowblind+, TBS_B, and tBlindBLS-3 achieve only static security. Recently, Jarecki et al. [15] proposed a non-interactive (one-round) scheme, 2B-tBlindBLS, achieving adaptive security in the pairing setting. They proved its TBS-OMUF-1 security in the ROM and AGM. Fuchsbauer et al. [16] proposed the FRW26 and proved its TBS-OMUF-1 security in the AGM. TBS-OMUF-1 is an adaptive corruption version of OMUF-1. In this paper, we use the prefix TBS to denote models that allow adaptive corruption; otherwise, corruption is static.

Algebraic group model vs. plain model. The algebraic algorithms are widely applied in cryptography, particularly for proving impossibility results. The algebraic group model [17] differs from the plain model in that the adversary can only conduct algebraic operations. Any group element queried or output by the adversary must be expressible as a linear combination of group elements already received by the adversary. The AGM can be viewed as an idealized model analogous to the generic group model (GGM) [18]. However, research by Zhang et al. [19] reveals that "hardness in AGM may not imply hardness in GGM", thereby challenging the security of AGM. In fact, the real-world adversary can also execute non-algebraic attacks observed in practice. In particular, within a TBS scenario, the adversary, acting as a user, can perform blinding operation on the result of message hashing. The actual adversary can substitute the normal blinded value with a random value, the combination of multiple blinded values, or the result of recursive hash computation [15]. However, AGM requires the adversary to provide a linear representation of the group element, thereby rendering all the above attack methods trivial. Thus, it is more practical to construct a TBS scheme with provable security in the plain model. In this paper, the plain model refers to the random oracle model without AGM, instead of the standard model. To date, TBS scheme still relies on AGM for adaptive security.

An adaptive TBS scheme without AGM remains unknown.

1.1 Our Contribution

OR compilation for DLEQ statement and DDH-tuple statement. We design an NP relation R_{bb} for discrete logarithm equality (DLEQ) statement, combined with the NP relation $\mathsf{R}_{\mathsf{ddh}}$ for decisional Diffie-Hellman (DDH) tuple statement, and further design a plain signature scheme $\mathsf{Sig}_{\mathsf{m}}$ using OR compilation. We prove the security of $\mathsf{Sig}_{\mathsf{m}}$ under the computational Diffie-Hellman (CDH) assumption in the ROM. Embedding the DLEQ branch is crucial for achieving adaptive security in our threshold blind signature scheme. OR compilation enables scheme construction and security proof to target different relations so that the simulator can answer the signing queries without secret keys in the security proof for blind signature and threshold blind signature schemes.

Non-interactive proof system for mixed extraction of group and scalar elements. We design a mixed Σ_{ped} protocol for a well-designed NP relation $\mathsf{R}_{\mathsf{ped}}$, and accordingly construct a non-interactive proof system $\mathsf{NIPS}_{\mathsf{ped}}$ for extracting group and scalar elements from the blinded protocol messages. Through mixed extraction, hash queries for messages can be categorized into simulatable part and reducible part, which allows the identification of the special signature forgery generated by the adversary itself among the submitted forgeries. We further propose a blind signature scheme BS_{m} built upon $\mathsf{Sig}_{\mathsf{m}}$, utilizing $\mathsf{NIPS}_{\mathsf{ped}}$ and a refined blinding technique. We prove the one-more unforgeable security of BS_{m} under the CDH assumption in the ROM.

Adaptively secure TBS scheme without AGM. We propose Rainblind, the first TBS scheme achieving adaptive security without AGM. The user interacts with the issuers over three signing rounds and outputs a signature consisting of

eight group elements and five scalars. Leveraging Lagrange interpolation with fine-grained coefficient control, we lift the blind signature scheme BS_m to the threshold setting to obtain Rainblind. We construct Rainblind by using an instantiation of tagged linear function (TLF) and embedding an OR compilation proof in the signature. The OR compilation is between the DLEQ statement for secret key and DDH-tuple statement derived from a hash function. It fails to satisfy the DDH-tuple statement due to the uniform randomness of the hash function, forcing the execution of the DLEQ statement. We formalize the one-more unforgeable security model with adaptive corruption queries, denoted as TBS-OMUF-SB, under which the security of Rainblind is reduced to the t-algebraic translation resistance assumption of TLF (implying DDH assumption) in the ROM. The inversion oracle embedded in this assumption can be used to answer the adversary's adaptive secret key corruption queries. By programming OR compilation into the DDH-tuple statement, we can leverage random elements to replace the DLEQ elements when answering the signing queries, avoiding the use of AGM. Crucially, we employ $\mathsf{NIPS}_\mathsf{ped}$ to distinguish the adversary's self-generated forgery.

Table 1 compares the security of state-of-the-art TBS schemes. No prior scheme achieves adaptive security without AGM. The lattice-based construction TBS_B [13], which provides only static security, is omitted from the table. From Table 1, the complexity of Rainblind is similar to that of related schemes, with a small increase. A practical efficiency analysis is provided in Appendix F.

Table 1. Comparison of security and performance between classic TBS schemes. PRF: pseudorandom function, BOMDH: bilinear one-more Diffie-Hellman, OMDL: one more discrete logarithm. Assuming "one round" includes the user sending a protocol message to the issuers, and the issuers returning the related protocol message.

Scheme	Adaptive	Model	Idealization	Assumption	Pairing	Round	Signature Size
Snowblind [8]	×	OMUF-SB	ROM+AGM	DL	×	3	$1\mathbb{G}+2\mathbb{Z}_p$
Snowblind+ [12]	×	OMUF-3	ROM	PRF	×	4	$1\mathbb{G}+2\mathbb{Z}_p$
tBlindBLS-3 [12]	×	OMUF-3	ROM	BOMDH+ PRF	✓	1	$1\mathbb{G}_2$
2B-tBlindBLS [15]	✓	TBS-OMUF-1	ROM+AGM	OMDL+DDH	✓	1	$1\mathbb{G}_1$
FRW26 [16]	✓	TBS-OMUF-1	AGM	(2,1)-DL	✓	1	$2\mathbb{G}_1$
Rainblind (ours)	✓	TBS-OMUF-SB	ROM	DDH	×	3	$8\mathbb{G}+5\mathbb{Z}_p$

1.2 Technical Overview

High-level syntax of TBS. In a TBS scheme, the user can acquire a signature for a message while keeping the message concealed, on the condition that at least $t+1$ issuers take part in the signing process. Concretely, the user blinds the message to conceal its contents, after which each issuer holding a secret key share generates a partial signature on the blinded message. After collecting a sufficient number of partial signatures, they can be combined and unblinded to yield a complete signature on the original message. The resulting signature is publicly verifiable by anyone in possession of public key.

Why 2B-tBlindBLS requires AGM for adaptive security? [3] In the one-more unforgeability security model of threshold blind signatures, the adversary $\mathcal{A}$ can make both signing queries and adaptive corruption queries. Jarecki et al. [15] work in the Type-3 pairing group: $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e) \leftarrow\$\ \mathsf{GPGen}(1^\lambda)$, where $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$, and $g, u, v \leftarrow\$\ \mathbb{G}_1$, $g_2 \leftarrow\$\ \mathbb{G}_2$, $g_T \leftarrow\$\ \mathbb{G}_T$. The core idea of their 2B-tBlindBLS is as follows. In the construction, they employ a multi-key-coupled public key and apply the double-hash zero-exponent modifications to group elements. During the security proof, the simulator provides the adversary $\mathcal{A}$ with the offset public key, while using an equivalent secret key pair against the offset public key and the programmed random oracles to respond to the adversary $\mathcal{A}$'s signing queries and adaptive corruption queries. The programming of the random oracles requires the AGM.

Concretely, they define three t-degree polynomials $x(\cdot), w(\cdot), s(\cdot) \leftarrow\$\ \mathbb{Z}_p[\cdot]_{(t)}$ such that $w(0) = s(0) = 0$ and $x(0) = \mathsf{sk}$ represents the common secret key. The common public key is $\mathsf{pk} = g_2^{x(0)}$. Each issuer i is assigned a partial secret key $\mathsf{sk}_i = (x(i), w(i), s(i))$, and the corresponding partial public key $\mathsf{pk}_i = g^{x(i)} u^{w(i)} v^{s(i)}$, where (u, v) are public parameters. In the signing protocol, let $\mathsf{H}, \mathsf{H}_0, \mathsf{H}_1 : \{0,1\}^* \to \mathbb{G}_1$, and $\ell_{i,\mathcal{SS}} = \ell_{i,\mathcal{SS}}(0)$ be the Lagrange coefficient. The user first computes the blinded value h as $h = \bar{h}^\beta$, where $\bar{h} = \mathsf{H}(m)$ and $\beta \in \mathbb{Z}_p$. Then, the user sends h to each issuer in the issuer set $\mathcal{SS}$. Each issuer i computes $Y_i := \left(\mathsf{H}_0(\tau)^{w(i)} \cdot \mathsf{H}_1(\tau)^{s(i)} \cdot h^{x(i)}\right)^{\ell_{i,\mathcal{SS}}}$ and sends Y_i to the user, where τ is a common message. After aggregating each issuer's Y_i, the user can get $Y := \prod_{i \in \mathcal{SS}} Y_i = \mathsf{H}_0(\tau)^{w(0)} \mathsf{H}_1(\tau)^{s(0)} h^{x(0)} = h^{\mathsf{sk}}$, and after de-blinding, the user can get $\sigma = \bar{Y} := Y^{1/\beta} = \bar{h}^{\mathsf{sk}}$. A verifier owning the public key pk can verify the signature by checking: $e(\sigma, g_2) = e(\mathsf{H}(m), \mathsf{pk})$.

We outline the proof strategy of 2B-tBlindBLS. The simulator first performs an honest key generation, thereby learning all $(x(i), w(i), s(i))$. It then sets $u := g^{-s \cdot \varsigma + a}$ and $v := g^\varsigma$ for random $\varsigma, s \in \mathbb{Z}_p^*$. The simulator provides the adversary $\mathcal{A}$ with an offset public key $\mathsf{pk} = g_2^{x(0)+a}$ and offset partial public keys $\mathsf{pk}_i = g^{x(i)} u^{w(i)+1} v^{s(i)+s} = g^{x(i)+a} u^{w(i)} v^{s(i)}$ for $i \in [n]$, where g_2^a and g^a come from the hard problem instance and the simulator does not know a. The simulator answers the adversary $\mathcal{A}$'s secret key corruption queries with $(x(i), w(i)+1, s(i)+s)$. Due to the threshold property, the adversary $\mathcal{A}$ remains unaware that $(w(0), s(0))$ has been shifted from $(0, 0)$ to $(1, s)$. For signing queries, the simulator programs the random oracles as $\mathsf{H}_0(\tau) = h^{-s \cdot \vartheta + a}$ and $\mathsf{H}_1(\tau) = h^\vartheta$, where ϑ is an independent fresh random value sampled for each τ. The simulator computes the partial signature using $(x(i), w(i) + 1, s(i) + s)$. This ensures that $Y = h^{x(0)} \cdot \mathsf{H}_0(\tau) \cdot \mathsf{H}_1(\tau)^s = h^{x(0)+a}$, which is consistent with the offset public key. Finally, the simulator uses the adversary $\mathcal{A}$'s forgery against the offset public key $\mathsf{pk} = g_2^{x(0)+a}$ to extract the solution a for the underlying hard problem.

Since the simulator has no knowledge of a, to properly program $\mathsf{H}_0(\tau) = h^{-s \cdot \vartheta + a} = h^{-s \cdot \vartheta} (g^a)^{\log_g h}$, the simulator is required to possess knowledge of the discrete logarithm of h. However, h is a blinded value computed as $h = \bar{h}^\beta$ by the

[3] Since our proof is in the ROM, we begin from 2B-tBlindBLS rather than FRW26.

adversary $\mathcal{A}$, where $\bar{h} = \mathsf{H}(m)$, thus the simulator does not know the blinding factor β. Moreover, a malicious adversary can substitute h with a random value, a combination of multiple blinded values, or the result of recursive hash queries, thereby preventing the simulator from extracting $\log_g h$. To circumvent this, Jarecki et al. [15] program $\mathsf{H}(m)$ with $\bar{h} = \mathsf{H}(m) = g^{\varpi}$, and then rely on the AGM to force the adversary to disclose the blinding factor. This ensures that the simulator can extract the logarithm of $h = \bar{h}^{\beta} = g^{\varpi \cdot \beta}$, where the AGM essentially acts as a forced extractor.

How Rainblind achieves adaptive security without AGM? To avoid reliance on the AGM, we propose a new pairing-free approach: In the scheme construction, we use the instantiation of tagged linear function (TLF), while embedding an OR compilation in the signature that chooses between the DLEQ statement for secret key and DDH-tuple statement derived from a hash function. It fails to satisfy the DDH-tuple statement due to the uniform randomness of the hash function, forcing the execution of the DLEQ statement. In the security proof, we leverage the oracle in the t-algebraic translation resistance assumption of TLF to answer the adversary $\mathcal{A}$'s adaptive secret key corruption queries. By programming the random oracle to validate the DDH-tuple statement, we can leverage random elements to replace the DLEQ elements when answering the signing queries. We design a non-interactive proof system $\mathsf{NIPS}_{\mathsf{ped}}$ for mixed extraction, which can extract both group and scalar elements from blinded protocol messages. Although our $\mathsf{NIPS}_{\mathsf{ped}}$ neither forces the adversary to query the hash oracle H on a message m nor extracts the discrete logarithm of h, the randomness substitution makes knowledge of the discrete logarithm of h unnecessary, thereby eliminating the need for AGM. Since the adversary's corruption queries target only secret keys, the use of randomness remains undetectable.

TLF admits two distinct instantiations, based on the algebraic one-more CDH (AOMCDH) and DDH assumptions respectively. For clarity in the technical overview, we use the AOMCDH instantiation (where the scalar set $\mathcal{S} = \mathbb{Z}_p$, tag set $\mathcal{T} = \mathbb{G}$, domain $\mathcal{D} = \mathbb{Z}_p$, range $\mathcal{R} = \mathbb{G}$, and the function is defined as $\mathsf{T}(u \in \mathcal{T}, x \in \mathcal{D}) = u^x$ in the multiplication group). For the concrete scheme construction, however, we adopt the DDH instantiation (with $\mathcal{S} = \mathbb{Z}_p$, $\mathcal{T} = \mathbb{G}^{2\times 2}$, $\mathcal{D} = \mathbb{Z}_p^2$, $\mathcal{R} = \mathbb{G}^2$, and $\mathsf{T}([\mathbf{G}] \in \mathcal{T}, \mathbf{x} \in \mathcal{D}) = [\mathbf{G}] \circ \mathbf{x}$ in the addition group) to achieve a reduction to the DDH assumption. The choice of instantiation affects only the underlying hardness assumption for the final security reduction and does not impact our core ideas. We follow the "Sig–BS–TBS" paradigm.

OR compilation and plain signature scheme $\mathsf{Sig}_{\mathsf{m}}$. We design two NP relations $(\mathsf{R}_{\mathsf{bb}}, \mathsf{R}_{\mathsf{ddh}})$ and two corresponding Σ protocols (Σ_0, Σ_1), and construct a plain signature scheme $\mathsf{Sig}_{\mathsf{m}}$ essentially for the disjunctive relation $\mathsf{R}_{\mathsf{bb}} \vee \mathsf{R}_{\mathsf{ddh}}$ by OR compilation. We use the Σ_0 protocol to ensure that the signer has signed $\bar{h}$ using sk (DLEQ), which implies NP relation R_{bb}. We use the Σ_1 protocol to ensure that $\mathbf{D} = (D_1, D_2, D_3)$ is a DDH tuple where $D_1 \in \mathbb{G}, (D_2, D_3) := \mathsf{H}_{\mathsf{ddh}}(\tau), \mathsf{H}_{\mathsf{ddh}} : \{0,1\}^* \to \mathbb{G}^2, \tau \in \mathcal{CM}$ is a common message. For $g, W, \bar{h} \in \mathcal{T} = \mathbb{G}, s, \mathsf{sk} \in \mathcal{D} = \mathbb{Z}_p$ and $\mathbf{S} \in \mathcal{R}^3$, we define R_{bb}:

$$\mathsf{R}_{\mathsf{bb}} := \{\mathbb{x}_0 = (g, W, \bar{h}, \mathbf{S}), \mathbb{w}_0 = (s, \mathsf{sk}) : \mathbf{S} = \phi_0(\bar{h}, (s, \mathsf{sk}))\},$$

where $\phi_0(\bar{h},(s,\mathsf{sk})) := (W^s \cdot \bar{h}^{\mathsf{sk}}, g^s, g^{\mathsf{sk}}) = (S_1,S_2,S_3)$. For $g, D_1 \in \mathbb{G}$, $d_2 \in \mathbb{Z}_p$, $\mathbf{D} \in \mathbb{G}^3$ and $\phi_1(d_2) := (g^{d_2}, D_1^{d_2}) = (D_2, D_3)$, we define $\mathsf{R}_{\mathsf{ddh}}$:

$$\mathsf{R}_{\mathsf{ddh}} := \{\mathbb{x}_1 = (g, \mathbf{D} = (D_1,D_2,D_3)), \mathbb{w}_1 = d_2 : \phi_1(d_2)\}.$$

We design two Σ-protocols, $\Sigma_0 = (\mathsf{Init}_0, \mathsf{Resp}_0, \mathsf{Verify}_0)$ for R_{bb} and $\Sigma_1 = (\mathsf{Init}_1, \mathsf{Resp}_1, \mathsf{Verify}_1)$ for $\mathsf{R}_{\mathsf{ddh}}$. For $b \in \{0,1\}$, we have that

$$\mathsf{Init}_b(\mathbb{x}_b, \mathbb{w}_b) \Rightarrow (\mathbf{A}_b, \mathsf{st}_b), \quad \mathsf{Resp}_b(\mathsf{st}_b, c_b) \Rightarrow \mathbf{z}_b,$$

$$\mathsf{Verify}_b(\mathbb{x}_b, \mathbf{A}_b, c_b, \mathbf{z}_b) \Rightarrow 0/1, \quad \mathsf{Sim}_b(\mathbb{x}_b, c_b) \Rightarrow (\mathbf{A}_b, \mathbf{z}_b).$$

The protocol Σ_0 ensures that the signer signs $\bar{h}$ with sk, while Σ_1 ensures that $\mathbf{D} = (D_1, D_2, D_3)$ is a DDH tuple.

In $\mathsf{Sig}_{\mathsf{m}}$, Σ_0 (for relation R_{bb}) is computed honestly and Σ_1 (for relation $\mathsf{R}_{\mathsf{ddh}}$) is computed using the simulator Sim_1. Finally, the signer produces a proof π for $\mathsf{R}_{\mathsf{bb}} \vee \mathsf{R}_{\mathsf{ddh}}$. Note that Sig_m uses the hash function $\mathsf{H}_{\mathsf{c}} : \{0,1\}^* \to \mathcal{S}$ only once, i.e., $c = \mathsf{H}_{\mathsf{c}}(\mathbf{A}_0, \mathbb{x}_0, \mathbf{A}_1, \mathbb{x}_1, m)$, and (c_0, c_1) satisfy $c = c_0 + c_1$, implying that π is a proof for $\mathsf{R}_{\mathsf{bb}} \vee \mathsf{R}_{\mathsf{ddh}}$. By the property that $\mathsf{H}_{\mathsf{ddh}}$ is uniformly distributed, $(D_1, D_2, D_3) \notin \mathsf{R}_{\mathsf{ddh}}$ with overwhelming probability, hence π is a proof for R_{bb} (involving sk). The verifier or user cannot distinguish which relation π proves. In security proof, the $\mathsf{H}_{\mathsf{ddh}}$ random oracle is programmed such that the simulator knows the corresponding witness $\mathbb{w}_1$ and thus computes Σ_1 ($\mathsf{R}_{\mathsf{ddh}}$) honestly. For Σ_0 (R_{bb}), it is computed using the simulator Sim_0, at this point π is a proof for $\mathsf{R}_{\mathsf{ddh}}$ (involving programmable random oracle). The adversary $\mathcal{A}$ cannot distinguish which relation π proves. Thus, the simulator can randomly sample two elements from $\mathcal{R}$, acting as $S_1 = W^s \cdot \bar{h}^{\mathsf{sk}}, S_2 = g^s$. Before applying the blinding factors, we give a flow of construction and security proof in Fig. 1.

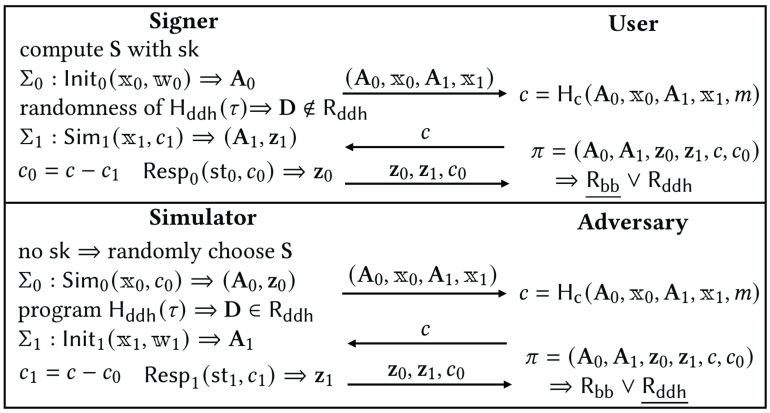

Fig. 1. Scheme construction and security proof.

After implementing the transformation of the disjunctive relation $\mathsf{R}_{\mathsf{bb}} \vee \mathsf{R}_{\mathsf{ddh}}$ between the scheme construction and security proof, the simulator can respond

to the adversary $\mathcal{A}$'s signing queries without the secret key. Note that even if the adversary $\mathcal{A}$ obtains the signer's secret key sk, it cannot validate the (S_1, S_2) in a signature, since the adversary $\mathcal{A}$ neither has access to the signer's ephemeral randomness s nor can solve the discrete logarithm of W. The transition from the plain signature scheme $\mathsf{Sig_m}$ to the blind signature scheme $\mathsf{BS_m}$ introduces a requirement in the security proof: the simulator need leverage the adversary's $(l+1)$ forgery pairs $(m_j, \sigma_j)_{j\in[l+1]}$ to solve a hard problem, which requires identifying the specific signature generated by the adversary itself. To address this, we adopt the following method: assuming that the adversary makes $l+1$ hash queries H_m for different messages (without loss of generality and without impact on probability), we pre-sample a random index q_m^* before the game starts. The hard problem instance is embedded in the response to the query for $\bar{h}^* = \mathsf{H}_m(m_{q_m^*})$. Moreover, we force the adversary to generate the signature for $m_{q_m^*}$ on its own. Specifically, if the simulator extracts $\bar{h}^*$ from the protocol message, it will abort the game. For all other cases not involving $\bar{h}^*$, the simulator proceeds with normal signature generation. Thus, the key problem we now face is: how to extract $\bar{h}$ from the first-round protocol message h submitted by the adversary? Our goal is to extract $\bar{h}$ from h, whereas Jarecki et al. need to extract the discrete logarithm corresponding to h.

Mixed extraction and blind signature scheme $\mathsf{BS_m}$. In order to enable the simulator to extract $\bar{h}$ from the blinded protocol message h sent by the adversary $\mathcal{A}$, we design an NP relation $\mathsf{R_{ped}}$ and its corresponding mixed Σ protocol Σ_{ped}. Drawing on the straight-line extraction [20,21], we design a non-interactive proof system $\mathsf{NIPS_{ped}}$ with extractor $\mathsf{Ext_{ped}}$ able to extract mixed elements. The non-interactive proof system designed by Klooß et al. [20], which only supports the extraction of scalar elements, is insufficient for our BS scheme. For $\mathbb{x}_{\mathsf{ped}} = (g, h, \hat{h}, E)$, $\mathbb{w}_{\mathsf{ped}} = (\bar{h}, \beta, \varphi)$, we define

$$\mathsf{R_{ped}} := \{\mathbb{x}_{\mathsf{ped}}, \mathbb{w}_{\mathsf{ped}} : E := g^{\varphi} \wedge \hat{h} := \bar{h}^{\beta} \wedge h = \hat{h} \cdot E^{\beta}\}.$$

Firstly, we conduct two mixed executions of Σ protocol to construct a mixed Σ_{ped} protocol for the above relation $\mathsf{R_{ped}}$. We run Σ protocol for (E, g, φ): $r_1 \leftarrow\$\ \mathcal{S}, A_1 := g^{r_1}, z_1 := r_1 + c_1 * \varphi$ such that $\pi_1 = (z_1, c_1)$ ensures that the prover knows φ as the witness for (E, g). Then we run Σ protocol for $(E, h, \hat{h}, \beta)$: $r_2 \leftarrow\$\ \mathcal{S}, A_2 := E^{r_2}, z_2 := r_2 + c_2 * \beta$ such that $\pi_2 = (z_2, c_2)$ ensures that the prover knows β as the witness for $(h, \hat{h}, E)$. For a tuple $(\beta, \hat{h})$, $\bar{h}$ is uniquely determined, which completes the Σ_{ped} protocol. Secondly, we construct a mixed extraction $\mathsf{NIPS_{ped}}$ for the relation $\mathsf{R_{ped}}$ from Σ_{ped}. In $\mathsf{NIPS_{ped}}$, the prover generates a proof π_{ped} for $(\mathbb{x}_{\mathsf{ped}}, \mathbb{w}_{\mathsf{ped}})$. There exists an extractor $\mathsf{Ext_{ped}}$ capable of extracting $(\bar{h}, \varphi, \beta)$ from $(g, h, \hat{h}, E, \pi_{\mathsf{ped}})$. By combining with $\mathsf{NIPS_{ped}}$, we construct a blind signature scheme $\mathsf{BS_m}$ from $\mathsf{Sig_m}$. Utilising the non-interactive proof system $\mathsf{NIPS_{ped}}$, the simulator can extract $\bar{h}$ from $(g, h, \hat{h}, E, \pi_{\mathsf{ped}})$ submitted by the adversary $\mathcal{A}$. Note that the adversary $\mathcal{A}$ could not perform the $\mathsf{H_m}$ query against m, or the adversary can perform hash queries $\bar{h} := \mathsf{H_m}(m)$ but set $\bar{h} := \bar{h}^{\alpha}$ with random $\alpha \in \mathcal{S}$. At this point, the simulator still extracts $\bar{h}$, although it cannot find the matching record in the hash list. But this situation

does not affect the computation of (S_1, S_2), because we have already made it possible in the security proof not to program $\mathsf{H_m}$ to compute (S_1, S_2), instead of directly sampling two random elements to act as (S_1, S_2).

Tagged linear function and TBS Scheme Rainblind**.** Now, the simulator can respond to the adversary $\mathcal{A}$'s signing queries without the secret key and leverage the final forgery to solve the hard problem, the final step is to handle the adversary $\mathcal{A}$'s adaptive secret keys corruption queries in the TBS scheme. To this end, we employ the oracle embedded in the t-algebraic translation resistance assumption of tagged linear function. In game $\mathsf{Game}_{\mathsf{TLF}}^{t\text{-A-TRAN-RES}}$ of this assumption, the game samples $g, h_{\#} \leftarrow\$\ \mathcal{T} = \mathbb{G}$, and $x_0, x_1, ..., x_t \leftarrow\$\ \mathcal{D} = \mathbb{Z}_p$, then computes $X_i := g^{x_i}$ for all $i \in [t] \cup \{0\}$. The adversary $\mathcal{F}$ is provided with the tuple $(g, h_{\#}, \{X_i\}_{i=0}^t)$ and is granted access to an oracle $\mathcal{O}_{\mathsf{TLF}}^{\mathsf{Inv}}(\alpha_0, \alpha_1, ..., \alpha_t)$. When the adverary $\mathcal{F}$ queries this oracle with $(\alpha_0, \alpha_1, ..., \alpha_t)$, the game returns the corresponding $x := \sum_{i=0}^t \alpha_i \cdot x_i$. The adversary $\mathcal{F}$ is permitted to query $\mathcal{O}_{\mathsf{TLF}}^{\mathsf{Inv}}$ at most t times, each time with a distinct input. Finally, the adversary $\mathcal{F}$ outputs $(X'_i)_{i=0}^t$ that satisfies: for $\forall i \in \{0\} \cup [t],\ \exists\ \theta_i \in \mathcal{D}$ s.t. $g^{\theta_i} = X_i \wedge h_{\#}^{\theta_i} = X'_i$.

Specifically, the simulator leverages the t-algebraic translation resistance assumption $(g, h_{\#}, \{X_i\}_{i=0}^t)$ to simulate the game $\mathsf{Game}_{\mathsf{TBS}}^{\mathsf{OMUF\text{-}SB}}$ for adversary $\mathcal{A}$. The simulator samples $w \leftarrow\$\ \mathbb{Z}_p$ and sets $W := g^w$ instead of samples $W \leftarrow\$\ \mathbb{G}$. For each $i \in [t]$, the simulator sets $\mathsf{pk}_i = X_i$. For every $i \in [n] \backslash \mathcal{SS}_0$, the simulator sets $\mathsf{pk}_i := \prod_{j \in \mathcal{SS}_0} \mathsf{pk}_j^{\ell_{j,\mathcal{SS}_0}(i)}$, where $\mathcal{SS}_0 := \{0\} \cup [t]$ and $\ell_{j,\mathcal{SS}_0}(i)$ is the Lagrange coefficient. As guaranteed by the OR compilation, the simulator can answer the adversary $\mathcal{A}$'s signing queries without the secret keys. When the adversary $\mathcal{A}$ makes an adaptive secret key corruption query for issuer i, the simulator queries the oracle $x_i = \mathcal{O}_{\mathsf{TLF}}^{\mathsf{Inv}}(\ell_{0,\mathcal{SS}_0}(i), ..., \ell_{t,\mathcal{SS}_0}(i))$ to obtain $\mathsf{sk}_i = x_i$. We assume, without loss of generality, that adversary $\mathcal{A}$ makes exactly t distinct corruption queries. Crucially, the simulator embeds $h_{\#}$ in the q_m^*-th $\mathsf{H_m}$ random oracle query. Finally, after adversary $\mathcal{A}$ submits $l+1$ message-signature pairs, simulator identifies the special one $(m^\star, \sigma^\star = (S_1^\star, S_2^\star, \pi^\star))$ and computes $X'_0 = S_1^\star / (S_2^\star)^w$. Let $\mathcal{SS}^* := \mathsf{Corrupted} \cup \{0\}$ with $|\mathcal{SS}^*| = t+1$. For $i \in \mathsf{Corrupted}$, the simulator sets $X'_i := h_{\#}^{x_i}$. For every $i \in [n] \backslash \mathsf{Corrupted}$, the simulator sets $X'_i := \prod_{j \in \mathcal{SS}^*} (X'_j)^{\ell_{j,\mathcal{SS}^*}(i)}$. The simulator submits $X'_0, ..., X'_t$ to the game $\mathsf{Game}_{\mathsf{TLF}}^{t\text{-A-TRAN-RES}}$. Thus, we reduce the security of our scheme Rainblind to the t-algebraic translation resistance assumption of TLF. Furthermore, when the TLF is instantiated based on the DDH, there exists a reduction [22] from the t-algebraic translation resistance assumption to the DDH assumption.

1.3 More Related Work

Several works have realized adaptive security in threshold signatures, such as Schnorr-like schemes [23,24,25,26,27], BLS-like schemes without pairings [22,28,29], BLS-like schemes with pairings [30,31,32,33], and RSA-based schemes [34,35,36,37]. Achieving adaptive security of threshold signature generally follows two paradigms. One line of work [26,32,28] constructs the public key from a multi-key structure and programs secret keys in the security proof to answer the adaptive corrup-

tion queries. Another approach [22,25,28] enables the simulator to respond to the signing queries without the secret keys, and it relies on the inversion oracle embedded in the hardness assumption to handle adaptive corruption queries. When adapted to the threshold blind setting, both approaches require the simulator to extract the discrete logarithm of the adversary's blinded protocol messages, necessitating the AGM, contradicting our goal. Other threshold signature schemes also face significant challenges in such a conversion.

Research [38,39,8,40,20,41] on blind signatures is extensive. Recent work [39,8] focuses on concurrent OMUF security under ROS attacks [42], a property our scheme also achieves. Existing AGM-free blind signatures [40,41,20] employ tricks that enable the simulator to respond to signing queries without extracting the discrete logarithm of blinded protocol messages and without secret key. These tricks rely on specific number-theoretic structures or random oracle programming. However, thresholdizing these schemes and further achieve adaptive security poses severe challenges to their core mechanisms. Specifically, the threshold setting distributes signing power among multiple issuers, while an adaptive adversary can dynamically corrupt issuers during protocol execution. This fundamentally conflicts structurally with the "extraction-free" and "key-free" simulation strategies used in these schemes.

Blind multi-signatures [5,43,44] can be considered as a special case of threshold blind signatures with a full threshold $t = n$, requiring the participation of all issuers. Blind multi-signatures also face adaptive security challenges, with no existing AGM-free construction. Our threshold blind signature scheme $\mathsf{Rainblind}$ naturally covers $t = n$, yielding (to our knowledge) the first adaptively secure and AGM-free blind multi-signature. A central goal in cryptography is to explore whether and how to eliminate pairings [45,39,46,47,37]. Pairing-free groups offer a double benefit–they reduce overhead while also benefiting from a richer cryptographic library. Moreover, our work builds upon the tagged linear function families employed in several works [48,49,50,47,37].

2 Preliminaries

2.1 Assumptions

We use an addition group generator $(\mathbb{G}, p, G) \leftarrow\$\ \mathsf{GGen}(1^\lambda)$, where for any $a, b \in \mathbb{Z}_p$, the generator G satisfies that $b\cdot(a\cdot G) = (a\cdot b)\cdot G, a\cdot G + b\cdot G = (a+b)\cdot G$.

Definition 1. (**CDH Assumption** [20]). *For $a, b \leftarrow\$\ \mathbb{Z}_p$, any PPT adversary $\mathcal{A}$, the computational Diffie-Hellman (CDH) assumption in $(\mathbb{G}, p, G)$ holds if*

$$\Pr[a, b \leftarrow\$\ \mathbb{Z}_p : \mathcal{A}(G, a\cdot G, b\cdot G) = (ab)\cdot G] \leq \mathsf{negl}(\lambda).$$

Definition 2. (**DDH Assumption** [20]). *For $a, b, c \leftarrow\$\ \mathbb{Z}_p$, any PPT adversary $\mathcal{A}$, the decisional Diffie-Hellman (DDH) assumption in $(\mathbb{G}, p, g)$ holds if*

$$\left|\Pr[\mathcal{A}(aG, bG, abG) = 1] - \Pr[\mathcal{A}(aG, bG, cG) = 1]\right| \leq \mathsf{negl}(\lambda).$$

2.2 Tagged Linear Function and DDH Instantiation

Definition 3. (**Tagged Linear Function** [22]). *A tagged linear function* (TLF) *is a tuple of PPT algorithms* $\mathsf{TLF} = (\mathsf{Gen}, \mathsf{T} = \circ)$ *as follows. In particular, we instantiate* TLF *in the DDH setting.*

- $\mathsf{par} \leftarrow \mathsf{Gen}(1^\lambda)$ *returns parameters* par *after accepting the security parameter* 1^λ. *The following sets are implicitly defined by* par*: scalar set* $\mathcal{S} := \mathbb{Z}_p$, *tag set* $\mathcal{T} := \mathbb{G}^{2\times 2}$, *domain set* $\mathcal{D} := \mathbb{Z}_p^2$ *and range set* $\mathcal{R} := \mathbb{G}^2$. *We use* $+, -$ *and* $*$ *to represent operations on these sets that can be efficiently executed:*

$$\textit{for } +, - : \mathcal{D}\pm\mathcal{D}\Rightarrow\mathcal{D}, \mathcal{S}\pm\mathcal{S}\Rightarrow\mathcal{S}, \mathcal{T}\pm\mathcal{T}\Rightarrow\mathcal{T}, \mathcal{R}\pm\mathcal{R}\Rightarrow\mathcal{R}.$$
$$\textit{for } * : \mathcal{S}*\mathcal{D}\Rightarrow\mathcal{D}, \mathcal{S}*\mathcal{S}\Rightarrow\mathcal{S}, \mathcal{S}*\mathcal{T}\Rightarrow\mathcal{T}, \mathcal{S}*\mathcal{R}\Rightarrow\mathcal{R}.$$

- $\mathbf{R} \leftarrow \mathsf{T}(\mathsf{par}, [\mathbf{G}], \mathbf{s})$ *returns a range element* $\mathbf{R} \in \mathcal{R}$ *after accepting parameters* par*, a tag* $[\mathbf{G}] \in \mathcal{T}$*, a domain element* $\mathbf{s} \in \mathcal{D}$. *We omit* par *if it is obvious from the context. For simplicity, we denote* $\mathsf{T}([\mathbf{G}], \mathbf{s})$ *as* $[\mathbf{G}] \circ \mathbf{s}$*:*

$$\textit{for } \circ : \mathcal{T}\circ\mathcal{D}\Rightarrow\mathcal{R}, [\mathbf{G}]\in\mathcal{T}=\mathbb{G}^{2\times 2}, \mathbf{s}\in\mathcal{D}=\mathbb{Z}_p^2,$$
$$[\mathbf{G}]\circ\mathbf{s}=\begin{bmatrix}G_{1,1}&G_{1,2}\\G_{2,1}&G_{2,2}\end{bmatrix}\circ\begin{bmatrix}s_1\\s_2\end{bmatrix}=\begin{bmatrix}s_1\cdot G_{1,1}+s_2\cdot G_{1,2}\\s_1\cdot G_{2,1}+s_2\cdot G_{2,2}\end{bmatrix}=\begin{bmatrix}R_1\\R_2\end{bmatrix}=\mathbf{R}.$$

  The homomorphism is realized by $\circ$ *for each tag* $[\mathbf{G}] \in \mathcal{T}$*,*

$$\textit{for } \mathbf{s}_1, \mathbf{s}_2 \in \mathcal{D}, c \in \mathcal{S}, [\mathbf{G}]\circ(c*\mathbf{s}_1+\mathbf{s}_2) = c*([\mathbf{G}]\circ\mathbf{s}_1)+[\mathbf{G}]\circ\mathbf{s}_2.$$

The tagged linear function satisfies the following properties.

Regularity. TLF satisfies $\varepsilon_{\mathbf{r}}$-regularity if there is a set Reg s.t.:

- for every $(\mathsf{par}, [\mathbf{H}]) \in \mathsf{Reg}$, the distributions defined as follows are identical:

$$\{(\mathsf{par}, [\mathbf{H}], \mathbf{R}) \,|\, \mathbf{R} \leftarrow\$\,\mathcal{R}\}, \{(\mathsf{par}, [\mathbf{H}], \mathbf{R}) \,|\, \mathbf{s} \leftarrow\$\,\mathcal{D}, \mathbf{R} := [\mathbf{H}]\circ\mathbf{s}\}.$$

- $\Pr[(\mathsf{par}, [\mathbf{H}]) \notin \mathsf{Reg} \mid \mathsf{par} \leftarrow \mathsf{Gen}(1^\lambda), [\mathbf{H}] \leftarrow\$\ \mathcal{T}] \le \varepsilon_{\mathbf{r}}$.

Translatability. TLF is $\varepsilon_{\mathbf{t}}$-translatability if there exist algorithms $\mathsf{Shift}, \mathsf{Tran}, \mathsf{InvTran}$:

- Well-Distributed Tags. $\mathcal{D}_0$ and $\mathcal{D}_1$ are statistically separated by at most $\varepsilon_{\mathbf{t}}$ for $\{(\mathsf{par}, [\mathbf{G}], [\mathbf{H}])\}$:

$$\mathcal{D}_0 := \{\mathsf{par} \leftarrow \mathsf{Gen}(1^\lambda), [\mathbf{G}], [\mathbf{H}] \leftarrow\$\ \mathcal{T}\},$$
$$\mathcal{D}_1 := \{\mathsf{par}\leftarrow\mathsf{Gen}(1^\lambda), [\mathbf{G}]\leftarrow\$\mathcal{T}, ([\mathbf{H}], \mathsf{td})\leftarrow\mathsf{Shift}(\mathsf{par}, [\mathbf{G}])\},$$
$$\text{where } \mathsf{Shift}(\mathsf{par}, [\mathbf{G}])\Rightarrow(\mathsf{td}=\mathbf{r}\leftarrow\$\mathbb{Z}_p^{2\times 2}, [\mathbf{H}]=\mathbf{r}*[\mathbf{G}]\in\mathbb{G}^{2\times 2}).$$

- Translation Completeness. For any $\mathsf{par} \in \mathsf{Gen}(1^\lambda)$, $[\mathbf{G}] \in \mathcal{T}$, $\mathbf{s} \in \mathcal{D}$, and $([\mathbf{H}], \mathsf{td}) \leftarrow \mathsf{Shift}(\mathsf{par}, [\mathbf{G}])$, we have

$$\mathsf{Tran}(\mathsf{td}=\mathbf{r}, [\mathbf{G}]\circ\mathbf{s})\Rightarrow\mathbf{r}*([\mathbf{G}]\circ\mathbf{s})=(\mathbf{r}*[\mathbf{G}])\circ\mathbf{s}=[\mathbf{H}]\circ\mathbf{s},$$
$$\mathsf{InvTran}(\mathsf{td}=\mathbf{r}, [\mathbf{H}]\circ\mathbf{s})\Rightarrow\mathbf{r}^{-1}*([\mathbf{H}]\circ\mathbf{s})=(\mathbf{r}^{-1}*[\mathbf{H}])\circ\mathbf{s}=[\mathbf{G}]\circ\mathbf{s}.$$

Algebraic translation resistance. Let $t \in \mathbb{N}$ be an integer. Considering the game in Fig. 2, and TLF is t-algebraic translation resistant, if the following inequality holds

$$\mathsf{Adv}^{t\text{-A-TRAN-RES}}_{\mathcal{A},\mathsf{TLF}}(\lambda) := \Pr[\mathsf{Game}^{t\text{-A-TRAN-RES}}_{\mathsf{TLF},\mathcal{A}}(\lambda)\Rightarrow 1] \le \mathsf{negl}(\lambda).$$

Game $\mathsf{Game}^{t\text{-A-TRAN-RES}}_{\mathsf{TLF},\mathcal{A}}(\lambda)$

1: $\mathsf{par} \leftarrow \mathsf{Gen}(1^\lambda), [\mathbf{G}], [\mathbf{H}_\#] \leftarrow\$\ \mathcal{T}$
2: $\mathbf{x}_0, \ldots \mathbf{x}_t \leftarrow\$\ \mathcal{D}$
3: **for** $i \in \{0\} \cup [t] : \mathbf{X}_i := [\mathbf{G}] \circ \mathbf{x}_i$
4: $(\mathbf{X}'_i)^t_{i=0} \leftarrow \mathcal{A}^{O^{\mathsf{Inv}}_{\mathsf{TLF}}}(\mathsf{par}, [\mathbf{G}], [\mathbf{H}_\#], (\mathbf{X}_i)^t_{i=0})$
5: **if** $\forall i \in \{0\} \cup [t],\ \exists\ \theta_i \in \mathcal{D}$
6: s.t. $[\mathbf{G}] \circ \theta_i = \mathbf{X}_i \wedge [\mathbf{H}_\#] \circ \theta_i = \mathbf{X}'_i$:
7: **return** 1
8: **return** 0

Oracle $O^{\mathsf{Inv}}_{\mathsf{TLF}}(\alpha_0, \alpha_1, \ldots, \alpha_t)$

1: **if** $q \geq t$: **return** $\perp$
2: $q := q + 1, \quad \mathbf{x} := \sum_{i=0}^t \alpha_i \mathbf{x}_i$
3: **return** $\mathbf{x}$

Fig. 2. The $\mathsf{Game}^{t\text{-A-TRAN-RES}}_{\mathsf{TLF},\mathcal{A}}(\lambda)$ of tagged linear function.

2.3 Blind Signature

We provide a generic definition of blind signature, which specializes to the plain blind signature when the common message τ is fixed, i.e., $|\mathcal{CM}| = 1$. The blind signatures protocol can be initiated either by the user [40,20] or by the signer [8,39]. The choice does not affect adaptive security, and we follow the former. In the one-more unforgeable security model for BS, the counting of trivial signature can be carried out either at the initiation of the session or at the completion. The latter means stronger security, so we adopt the latter setup.

Definition 4. (**Blind Signature** [40]). *A BS scheme* $\mathsf{BS} = (\mathsf{BS.SysGen}, \mathsf{BS.KeyGen}, \mathsf{BS.Sign} = (\{\mathsf{BS.ISign}_k\}^R_{k=1}, \{\mathsf{BS.USign}_k\}^R_{k=0}), \mathsf{BS.Verify})$ *consists of three algorithms and a protocol:*
$\mathsf{par} \leftarrow \mathsf{BS.SysGen}(1^\lambda)$ *returns system parameters* par *given security parameter* 1^λ.
$(\mathsf{PK},\mathsf{SK}) \leftarrow \mathsf{BS.KeyGen}(\mathsf{par})$ *returns* $(\mathsf{PK}, \mathsf{SK})$ *given system parameters* par.
$\sigma \leftarrow \mathsf{BS.Sign}(\mathsf{par}, \mathsf{PK}, \mathsf{SK}, m, \tau)$*: For message* $m \in \{0,1\}^*$*, common message* $\tau \in \mathcal{CM}$*, public key* PK*, the user interacts with the signer according to* $\mathsf{BS.Sign} = (\{\mathsf{BS.ISign}_k\}^R_{k=1}, \{\mathsf{BS.USign}_k\}^R_{k=0})$ *as follows:*

$$
\begin{array}{ll}
 & (\mathsf{st}^U, \mathsf{pm}^U_0) \leftarrow \mathsf{BS.USign}_0(\mathsf{PK}, m, \tau, \mathsf{par}) \\
(\mathsf{st}^I, \mathsf{pm}^I_1) \leftarrow \mathsf{BS.ISign}_1(\mathsf{SK}, \tau, \mathsf{pm}^U_0), & (\mathsf{st}^U, \mathsf{pm}^U_1) \leftarrow \mathsf{BS.USign}_1(\mathsf{st}^U, \mathsf{pm}^I_1) \\
(\mathsf{st}^I, \mathsf{pm}^I_j) \leftarrow \mathsf{BS.ISign}_j(\mathsf{st}^I, \mathsf{pm}^U_{j-1}), & (\mathsf{st}^U, \mathsf{pm}^U_j) \leftarrow \mathsf{BS.USign}_j(\mathsf{st}^U, \mathsf{pm}^I_j) \\
\mathsf{pm}^I_R \leftarrow \mathsf{BS.ISign}_R(\mathsf{st}^I, \mathsf{pm}^U_{R-1}), & \sigma \leftarrow \mathsf{BS.USign}_R(\mathsf{st}^U, \mathsf{pm}^I_R)
\end{array}
$$

where st *is internal state of the user or signer and* pm *is protocol message.*
$b \leftarrow \mathsf{BS.Verify}(\mathsf{par}, \sigma, \mathsf{PK}, m, \tau)$ *outputs a bit* $b \in \{0,1\}$ *given system parameters* par*, signature* σ*, public key* PK*, message* m *and common message* τ.

Definition 5. (BS **Correctness**). BS *is correct with error* γ_{err} *if for any* $m \in \{0,1\}^*$, $\tau \in \mathcal{CM}$ *and* $(\mathsf{PK}, \mathsf{SK}) \leftarrow \mathsf{BS.KeyGen}(\mathsf{par})$ *s.t.:*

$$
\Pr\left[\begin{array}{l}\sigma \leftarrow \mathsf{BS.Sign}(\mathsf{par}, \mathsf{PK}, \mathsf{SK}, m, \tau) \\ \mathsf{BS.Verify}(\mathsf{par}, \sigma, \mathsf{PK}, m, \tau) = 1\end{array}\right] \geq 1 - \gamma_{\mathsf{err}}(\lambda).
$$

Definition 6. (BS-OMUF). *Consider the game in Fig. 3,* BS *is one-more unforgeable if for any PPT adversary* $\mathcal{A}$*,*

$$
\mathsf{Adv}^{\mathsf{BS\text{-}OMUF}}_{\mathcal{A},\mathsf{BS}}(\lambda) := \Pr[\mathsf{Game}^{\mathsf{OMUF}}_{\mathsf{BS},\mathcal{A}}(\lambda) \Rightarrow 1] \leq \mathsf{negl}(\lambda).
$$

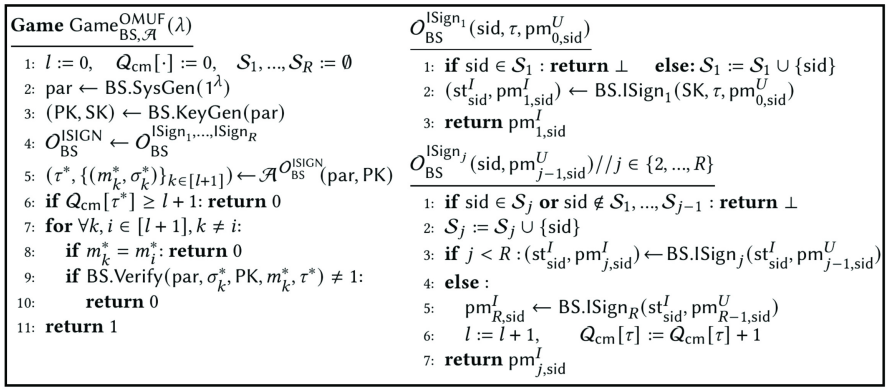

Fig. 3. Game $\mathsf{Game}^{\mathsf{OMUF}}_{\mathsf{BS},\mathcal{A}}(\lambda)$ of blind signature.

Definition 7. (BS **Computational Blindness**). BS *is computationally blind if for any PPT adversary* $\mathcal{A}$, $\mathsf{Adv}^{\mathsf{BS\text{-}Blind}}_{\mathcal{A},\mathsf{BS}}(\lambda) = |\Pr[b = b'] - 1/2| \leq \mathsf{negl}(\lambda)$ *for the game* $\mathsf{Game}^{\mathsf{Blind}}_{\mathsf{BS}}$ *as follows:*

1. *Run* $(\mathsf{PK}, m_0, m_1, \tau, \mathsf{st}) \leftarrow \mathcal{A}(1^\lambda)$.
2. *Set* $\mathcal{O}_0$ *for* $\mathsf{BS.USign}(\mathsf{par}, \mathsf{PK}, m_b, \tau)$ *and* $\mathcal{O}_1$ *for* $\mathsf{BS.USign}(\mathsf{par}, \mathsf{PK}, m_{1-b}, \tau)$, *where we assume that* BS.USign *includes all algorithms executed by the user.*
3. *Run* $\mathsf{st}' \leftarrow \mathcal{A}^{\mathcal{O}_0,\mathcal{O}_1}(\mathsf{st})$, *where* $\mathcal{A}$ *can access* $\mathcal{O}_0$ *and* $\mathcal{O}_1$ *in any interleaved order, but only once for each oracle. Denote the local results of* $\mathcal{O}_0$, $\mathcal{O}_1$ *as* σ_b, σ_{1-b}.
4. *If* $\sigma_0 = \perp$ *or* $\sigma_1 = \perp$, *run* $b' \leftarrow \mathcal{A}(\mathsf{st}', \perp, \perp)$. *Otherwise, run* $b' \leftarrow \mathcal{A}(\mathsf{st}', \sigma_0, \sigma_1)$.
5. *Output* b'.

2.4 Threshold Blind Signature

We provide a generic definition of threshold blind signature, which specializes to the plain threshold blind signature in the case where the common message τ is fixed, i.e., $|\mathcal{CM}| = 1$. Moreover, we define one-more unforgeable security model with fully adaptive corruption queries (TBS-OMUF-SB). We emphasise that the differences in x among the different security models OMUF-x for $\mathsf{x} \in \{0, 1, 2, 3, \mathsf{FC}, \mathsf{SB}\}$ are irrelevant to adaptive security. Our security model is essentially consistent with that of 2B-tBlindBLS [15]. For corrupted issuers, only their secret keys are returned to the adversary, while no other information is revealed. For details, see Fig. 4.

Definition 8. (**Threshold Blind Signature**). *A* (t, n) *threshold blind signature scheme* $\mathsf{TBS} = (\mathsf{TBS.SysGen}, \mathsf{TBS.KeyGen}, \mathsf{TBS.Sign} = (\{\mathsf{TBS.ISign}_k\}^R_{k=1}, \{\mathsf{TBS.USign}_k\}^R_{k=0}), \mathsf{TBS.Verify})$ *consists of three algorithms and a protocol:*
$\mathsf{par} \leftarrow \mathsf{TBS.SysGen}(1^\lambda)$ *returns system parameters* par *for security parameter* 1^λ.
$(\{\mathsf{SK}_i\}_{i\in[n]}, \mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]}) \leftarrow \mathsf{TBS.KeyGen}(\mathsf{par}, n, t)$ *returns a set* $\{\mathsf{SK}_i\}_{i\in[n]}$ *of secret keys for each issuer, a public key* PK *for all issuers, a set* $\{\mathsf{PK}_i\}_{i\in[n]}$

of public keys for each issuer, given the system parameters par*, the number* n *of total issuers, and a threshold* $t < n$*.*

$\sigma \leftarrow \mathsf{TBS.Sign}(\mathsf{par}, \{\mathsf{SK}_i\}_{i\in[n]}, \mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]}, m, \tau, \mathcal{SS})$*: For message* $m \in \{0,1\}^*$*, common message* $\tau \in \mathcal{CM}$*, and public key* PK*, the user interacts with issuers in* $\mathcal{SS} \subseteq [n] (t+1 \le |\mathcal{SS}| \le n)$ *according* $\mathsf{TBS.Sign} = (\{\mathsf{TBS.ISign}_k\}_{k=1}^{R}, \{\mathsf{TBS.USign}_k\}_{k=0}^{R})$*:*

$$(\mathsf{st}^U, \mathsf{pm}_0^U) \leftarrow \mathsf{TBS.USign}_0(\mathsf{PK}, \{\mathsf{PK}_j\}_1^n, m, \tau, \mathsf{par}, \mathcal{SS})$$

$$(\mathsf{st}_i^I, \mathsf{pm}_{1,i}^I) \leftarrow \mathsf{TBS.ISign}_1(i, \mathsf{SK}_i, \{\mathsf{PK}_j\}_1^n, \tau, \mathsf{pm}_0^U), (\mathsf{st}^U, \mathsf{pm}_1^U) \leftarrow \mathsf{TBS.USign}_1(\mathsf{st}^U, \{\mathsf{pm}_{1,i}^I\}_{i\in\mathcal{SS}})$$

$$(\mathsf{st}_i^I, \mathsf{pm}_{j,i}^I) \leftarrow \mathsf{TBS.ISign}_j(i, \mathsf{st}_i^I, \mathsf{pm}_{j-1}^U), (\mathsf{st}^U, \mathsf{pm}_j^U) \leftarrow \mathsf{TBS.USign}_j(\mathsf{st}^U, \{\mathsf{pm}_{j,i}^I\}_{i\in\mathcal{SS}})$$

$$(\mathsf{pm}_{R,i}^I, \mathcal{SS}) \leftarrow \mathsf{TBS.ISign}_R(i, \mathsf{st}_i^I, \mathsf{pm}_{R-1}^U), \sigma \leftarrow \mathsf{TBS.USign}_R(\mathsf{st}^U, \{\mathsf{pm}_{R,i}^I\}_{i\in\mathcal{SS}})$$

where st *is internal state of the user or issuers and* pm *is protocol message.*

$b \leftarrow \mathsf{TBS.Verify}(\mathsf{par}, \sigma, \mathsf{PK}, m, \tau)$ *outputs a bit* $b \in \{0,1\}$ *given system parameters* par*, signature* σ*, the public key* PK*, message* m *and a common message* τ*.*

Definition 9. (TBS **Correctness**).TBS *is correct with correctness error* γ_{err} *if*

$$\Pr\left[\begin{array}{l} \mathsf{par} \leftarrow \mathsf{TBS.SysGen}(1^\lambda) \\ (\{\mathsf{SK}_i\}_{i\in[n]}, \mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]}) \leftarrow \mathsf{TBS.KeyGen}(\mathsf{par}, n, t) \\ \sigma \leftarrow \mathsf{TBS.Sign}(\mathsf{par}, \{\mathsf{SK}_i\}_{i\in[n]}, \mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]}, m, \tau, \mathcal{SS}) \\ b \leftarrow \mathsf{TBS.Verify}(\mathsf{par}, \sigma, \mathsf{PK}, m, \tau) \end{array} \middle| \, b = 1 \right] \ge 1 - \gamma_{\mathsf{err}}(\lambda).$$

Definition 10. (TBS-OMUF-SB). *Consider the game in Fig. 4,* TBS *is one-more unforgeable with fully adaptive corruption if for any PPT adversary* $\mathcal{A}$*, the below condition holds*

$$\mathsf{Adv}^{\mathsf{TBS\text{-}OMUF\text{-}SB}}_{\mathcal{A},\mathsf{TBS}}(\lambda) := \Pr[\mathsf{Game}^{\mathsf{OMUF\text{-}SB}}_{\mathsf{TBS},\mathcal{A}}(\lambda) \Rightarrow 1] \le \mathsf{negl}(\lambda).$$

Game $\mathsf{Game}^{\mathsf{OMUF\text{-}SB}}_{\mathsf{TBS},\mathcal{A}}(\lambda)$

1: $\mathsf{ES}_{\mathsf{SB}}[\cdot] := \emptyset$
2: $\mathcal{SI}_1, ..., \mathcal{SI}_R := \emptyset$
3: $\mathsf{par} \leftarrow \mathsf{TBS.SysGen}(1^\lambda)$
4: $(n, t, \mathsf{Corrupted}) \leftarrow \mathcal{A}(\mathsf{par})$
5: **if** $n < t \vee |\mathsf{Corrupted}| > t$: **return** 0
6: $\mathsf{kin} := (\mathsf{par}, n, t)$
7: $(\{\mathsf{SK}_i\}_1^n, \mathsf{PK}, \{\mathsf{PK}_i\}_1^n) \leftarrow \mathsf{TBS.KeyGen}(\mathsf{kin})$
8: $O_{\mathsf{TBS}}^{\mathsf{ISIGN}} \leftarrow O_{\mathsf{TBS}}^{\mathsf{ISign}_1, ..., \mathsf{ISign}_R}$
9: $\mathsf{In} \leftarrow (\mathsf{par}, \mathsf{PK}, \{\mathsf{PK}_i\}_1^n, \{\mathsf{SK}_i\}_{i\in\mathsf{Corrupted}})$
10: $(\tau^*, \{(m_k^*, \sigma_k^*)\}_{k\in[l+1]}) \leftarrow \mathcal{A}^{O_{\mathsf{TBS}}^{\mathsf{ISIGN}}, O_{\mathsf{TBS}}^{\mathsf{Corr}}}(\mathsf{In})$
11: **if** $|\mathsf{Corrupted}| > t$: **return** 0
12: **if** $|\mathsf{ES}_{\mathsf{SB}}[\tau^*]| \ge l + 1$: **return** 0
13: **for** $\forall k, i \in [l+1], k \ne i$:
14: **if** $m_k^* = m_i^*$: **return** 0
15: **if** $\mathsf{TBS.Verify}(\mathsf{par}, \sigma_k^*, \mathsf{PK}, m_k^*, \tau^*) \ne 1$:
16: **return** 0
17: **return** 1

$O_{\mathsf{TBS}}^{\mathsf{Corr}}(i)$

1: **if** $|\mathsf{Corrupted}| \ge t$: **return** $\perp$
2: $\mathsf{Corrupted} \leftarrow \mathsf{Corrupted} \cup \{i\}$
3: **return** SK_i

$O_{\mathsf{TBS}}^{\mathsf{ISign}_j}(\mathsf{sid}, i, \mathsf{pm}_{j-1,\mathsf{sid}}^U) // j \in \{2, ..., R\}$

1: **if** $i \in \mathsf{Corrupted}$: **return** $\perp$
2: **if** $(i, \mathsf{sid}) \in \mathcal{SI}_j$: **return** $\perp$
3: **if** $j > 1$: $(i, \mathsf{sid}) \notin \mathcal{SI}_1, ..., \mathcal{SI}_{j-1}$: **return** $\perp$
4: $\mathcal{SI}_j := \mathcal{SI}_j \cup \{(i, \mathsf{sid})\}$
5: **if** $j == 1$:
6: $(\mathsf{st}_{i,\mathsf{sid}}^I, \mathsf{pm}_{1,i,\mathsf{sid}}^I) \leftarrow \mathsf{TBS.ISign}_1(i, \mathsf{SK}_i, \mathsf{pm}_{0,\mathsf{sid}}^U)$
7: **else if** $j < R$:
8: $(\mathsf{st}_{i,\mathsf{sid}}^I, \mathsf{pm}_{j,i,\mathsf{sid}}^I) \leftarrow \mathsf{TBS.ISign}_j(i, \mathsf{st}_{i,\mathsf{sid}}^I, \mathsf{pm}_{j-1,\mathsf{sid}}^U)$
9: **else if** $j == R$:
10: $(\mathsf{pm}_{R,i,\mathsf{sid}}^I, \mathcal{SS}) \leftarrow \mathsf{TBS.ISign}_R(i, \mathsf{st}_{i,\mathsf{sid}}^I, \mathsf{pm}_{R-1,\mathsf{sid}}^U)$
11: $\mathsf{ES}_{\mathsf{SB}}[\tau] \leftarrow \mathsf{ES}_{\mathsf{SB}}[\tau] \cup \{(\mathsf{sid}, \mathcal{SS})\}$
12: **return** $(\mathsf{pm}_{j,i,\mathsf{sid}}^I, \mathcal{SS})$
13: **return** $\mathsf{pm}_{j,i,\mathsf{sid}}^I$

Fig. 4. Game $\mathsf{Game}^{\mathsf{OMUF\text{-}SB}}_{\mathsf{TBS},\mathcal{A}}(\lambda)$ of threshold blind signature.

Definition 11. (TBS **Computational Blindness**). TBS *satisfies computational blindness if for any PPT adversary* $\mathcal{A}$, $\mathsf{Adv}^{\mathsf{TBS\text{-}Blind}}_{\mathcal{A},\mathsf{TBS}}(\lambda) = |\Pr[b = b'] - 1/2| \leq \mathsf{negl}(\lambda)$, *where* $\mathsf{Game}^{\mathsf{Blind}}_{\mathsf{TBS}}$ *is defined as follows.*

1. *Run* $(\mathsf{PK}, \{\mathsf{PK}_i\}_1^n, m_0, m_1, \tau, \mathsf{st}) \leftarrow \mathcal{A}(1^\lambda)$.
2. *Set* $\mathcal{O}_0$ *for* $\mathsf{TBS.USign}(\mathsf{par}, \mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]}, \mathcal{SS}_b, m_b, \tau), \mathcal{O}_1$ *for* $\mathsf{TBS.USign}(\mathsf{par}, \mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]}, \mathcal{SS}_{1-b}, m_{1-b}, \tau)$, *and we assume that* TBS.USign *includes all algorithms executed by the user.*
3. *Run* $\mathsf{st}' \leftarrow \mathcal{A}^{\mathcal{O}_0,\mathcal{O}_1}(\mathsf{st})$, *where* $\mathcal{A}$ *can access* $\mathcal{O}_0$ *and* $\mathcal{O}_1$ *in any interleaved order, but only once for each oracle. Denote the local results of* $\mathcal{O}_0$, $\mathcal{O}_1$ *as* σ_b, σ_{1-b}.
4. *If* $\sigma_0 = \perp$ *or* $\sigma_1 = \perp$, *run* $b' \leftarrow \mathcal{A}(\mathsf{st}', \perp, \perp)$. *Otherwise, run* $b' \leftarrow \mathcal{A}(\mathsf{st}', \sigma_0, \sigma_1)$.
5. *Output* b'.

3 Sig_m: Signature for Disjunctive Relation

We first construct two Σ protocols against two particular NP relations, one related to discrete logarithm equality and the other to DDH tuple. By computing honestly for Σ_0 and simulating for Σ_1, we construct a plain signature scheme Sig_m, which essentially produces a proof for the disjunctive relation $\mathsf{R}_{\mathsf{bb}} \vee \mathsf{R}_{\mathsf{ddh}}$. Embedding discrete logarithm equality in R_{bb} that corresponds to Σ_0 is our contribution. We provide a lemma for the security of Sig_m, to prove the security of the subsequent blind signature scheme BS_m.

3.1 NP Relation R_{bb} and Σ_0 Protocol

For $[\mathbf{G}], [\mathbf{W}], [\bar{\mathbf{H}}] \in \mathcal{T}, \mathbf{s}, \mathbf{sk} \in \mathcal{D}$ and $\mathbf{S} = (\mathbf{S}_1, \mathbf{S}_2, \mathbf{S}_3) \in \mathcal{R}^3$, we define a function $\phi^{\mathsf{BB}}_{[\mathbf{G}],[\mathbf{W}]} : \mathcal{T} \times \mathcal{D}^2 \to \mathcal{R}^3$:

$$\phi^{\mathsf{BB}}_{[\mathbf{G}],[\mathbf{W}]}([\bar{\mathbf{H}}], (\mathbf{s}, \mathbf{sk})) = ([\mathbf{W}] \circ \mathbf{s} + [\bar{\mathbf{H}}] \circ \mathbf{sk}, [\mathbf{G}] \circ \mathbf{s}, [\mathbf{G}] \circ \mathbf{sk}) = \mathbf{S}.$$

If $([\mathbf{G}], [\mathbf{W}])$ is obvious from the context, let $\phi_0 = \phi^{\mathsf{BB}}_{[\mathbf{G}],[\mathbf{W}]}$ for brevity. The definition of NP relation is in Appendix A.1. We define NP relation R_{bb} with included language $\mathscr{L}_{\mathsf{bb}}$ as follows: $\mathsf{R}_{\mathsf{bb}} := \{\mathbb{x}_0 = ([\mathbf{G}], [\mathbf{W}], [\bar{\mathbf{H}}], \mathbf{S}), \mathbb{w}_0 = (\mathbf{s}, \mathbf{sk}) : \mathbf{S} = \phi_0([\bar{\mathbf{H}}], (\mathbf{s}, \mathbf{sk}))\}$. For ϕ_0 and R_{bb}, we construct Σ_0 protocol as in Fig. 5. The security properties of Σ_0 protocol are shown in Appendix B.

$\mathsf{Init}_0(\mathbb{x}_0, \mathbb{w}_0)$:
1: Sample $\mathbf{r}_0 \leftarrow\$ \mathcal{D}^2$ and set $\mathbf{A}_0 := \phi_0([\bar{\mathbf{H}}], \mathbf{r}_0)$
2: Output commitment $\mathbf{A}_0$ and state $\mathsf{st}_0 := (\mathbb{w}_0, \mathbf{r}_0)$
$\mathsf{Resp}_0(\mathsf{st}_0, c_0, \ell)$: Output $\mathbf{z}_0 := \mathbf{r}_0 + c_0 * \ell * \mathbb{w}_0$
$\mathsf{Verify}_0(\mathbb{x}_0, \mathbf{A}_0, c_0, \mathbf{z}_0, \ell)$: Output 1 if $\mathbf{A}_0 = \phi_0([\bar{\mathbf{H}}], \mathbf{z}_0) - c_0 * \ell * \mathbf{S}$ and 0 otherwise.
$\mathsf{Sim}_0(\mathbb{x}_0, c_0, \ell)$ Sample $\mathbf{z}_0 \in \mathcal{D}^2$, set $\mathbf{A}_0 = \phi_0([\bar{\mathbf{H}}], \mathbf{z}_0) - c_0 * \ell * \mathbf{S}$. Output $(\mathbf{A}_0, \mathbf{z}_0)$.
(During the construction and security proof of our TBS scheme Rainblind, the Lagrange coefficient with pink highlight will be added.)

Fig. 5. $\Sigma_0 = (\mathsf{Init}_0, \mathsf{Resp}_0, \mathsf{Verify}_0)$ for ϕ_0 and R_{bb} by TLF.

3.2 NP Relation $\mathsf{R}_{\mathsf{ddh}}$ and Σ_1 Protocol

For $G, D_1 \in \mathbb{G}, d_2 \in \mathbb{Z}_p$ and $\mathbf{D} \in \mathbb{G}^3$, we define a function $\phi^{\mathsf{DDH}}_{G,D_1} : \mathbb{Z}_p \to \mathbb{G}^2$: $\phi^{\mathsf{DDH}}_{G,D_1}(d_2) = (d_2 \cdot G, d_2 \cdot D_1)^\top$. If (G, D_1) are obvious from the context, let $\phi_1 = \phi^{\mathsf{DDH}}_{G,D_1}$ for brevity. We define $\mathsf{R}_{\mathsf{ddh}}$ with language $\mathscr{L}_{\mathsf{ddh}}$ as follows: $\mathsf{R}_{\mathsf{ddh}} := \{\mathbb{x}_1 = (G, \mathbf{D} = (D_1, D_2, D_3)), \mathbb{w}_1 = d_2 : (D_2, D_3) = \phi_1(d_2)\}$. For function ϕ_1 and NP relation $\mathsf{R}_{\mathsf{ddh}}$, we construct Σ_1 protocol as in Fig. 6. The security properties of Σ_1 protocol are shown in Appendix B.

$\mathsf{Init}_1(\mathbb{x}_1, \mathbb{w}_1)$:
1: Sample $r_1 \leftarrow\$ \mathbb{Z}_p$ and set $\mathbf{A}_1 := \phi_1(r_1)$
2: Output commitment $\mathbf{A}_1$ and state $\mathsf{st}_1 := (\mathbb{w}_1, r_1)$

$\mathsf{Resp}_1(\mathsf{st}_1, c_1)$: Output $z_1 := r_1 + c_1 * \mathbb{w}_1$
$\mathsf{Verify}_1(\mathbb{x}_1, \mathbf{A}_1, c_1, z_1)$: Output 1 if $\mathbf{A}_1 = \phi_1(z_1) - c_1 * (D_2, D_3)^\top$ and 0 otherwise.
$\mathsf{Sim}_1(\mathbb{x}_1, c_1)$: Sample $z_1 \in \mathbb{Z}_p$, set $\mathbf{A}_1 = \phi_1(z_1) - c_1 * (D_2, D_3)^\top$.

Fig. 6. $\Sigma_1 = (\mathsf{Init}_1, \mathsf{Resp}_1, \mathsf{Verify}_1)$ for ϕ_1 and $\mathsf{R}_{\mathsf{ddh}}$.

3.3 $\mathsf{Sig}_{\mathsf{m}}$: Signature for Disjunctive Relation

Assume that $\Sigma_0 = (\mathsf{Init}_0, \mathsf{Resp}_0, \mathsf{Verify}_0)$ and $\Sigma_1 = (\mathsf{Init}_1, \mathsf{Resp}_1, \mathsf{Verify}_1)$ are Σ-protocols over the challenge space $\mathcal{S} = \mathbb{Z}_p$, designed for relations R_{bb} and $\mathsf{R}_{\mathsf{ddh}}$ specified above, respectively. The signature Sig_m is formally defined in Fig. 7. It is significant that the scheme only uses H_c (with $\mathsf{H}_\mathsf{c} : \{0,1\}^* \to \mathcal{S}$) once throughout the entire signing algorithm, i.e., $c = \mathsf{H}_\mathsf{c}(\mathbf{A}_0, \mathbb{x}_0, \mathbf{A}_1, \mathbb{x}_1, m)$. Furthermore, c_0 and c_1 satisfy $c = c_0 + c_1$, implying that π is a proof of $\mathsf{R}_{\mathsf{bb}} \vee \mathsf{R}_{\mathsf{ddh}}$. Additionally, the first flow $\mathbf{A}_0, \mathbf{A}_1$ can be omitted from the proof π because these values can be recomputed given $(c, c_0, \mathbf{z}_0, \mathbf{z}_1)$.

$\underline{\mathsf{Sig}_\mathsf{m}.\mathsf{SysGen}(1^\lambda)}$
1: $(\mathbb{G}, p, G) \leftarrow\$ \mathsf{GGen}(1^\lambda)$
2: $\mathsf{par}' \leftarrow\$ \mathsf{TLF.Gen}(1^\lambda), \quad [\mathbf{G}] \leftarrow\$ \mathcal{T}$
3: **Select** $\mathsf{H}_\mathsf{m} : \{0,1\}^* \to \mathcal{T}$
4: **Select** $\mathsf{H}_\mathsf{c} : \{0,1\}^* \to \mathcal{S}$
5: **Select** $\mathsf{H}_{\mathsf{ddh}} : \{0,1\}^* \to \mathbb{G}^2$
6: $\mathsf{H}_{\mathsf{all}} := (\mathsf{H}_\mathsf{m}, \mathsf{H}_\mathsf{c}, \mathsf{H}_{\mathsf{ddh}})$
7: **return** $\mathsf{par} = (\mathbb{G}, p, G, \mathsf{par}', \mathsf{H}_{\mathsf{all}}, [\mathbf{G}])$

$\underline{\mathsf{Sig}_\mathsf{m}.\mathsf{KeyGen}(\mathsf{par})}$
1: $[\mathbf{W}] \leftarrow\$ \mathcal{T}, \quad D_1 \leftarrow\$ \mathbb{G}$
2: $\mathsf{SK} = \mathbf{sk} \leftarrow\$ \mathcal{D}, \quad \mathbf{X} := [\mathbf{G}] \circ \mathbf{sk}$
3: $\mathsf{PK} = (\mathbf{X}, [\mathbf{W}], D_1)$
4: **return** $(\mathsf{PK}, \mathsf{SK})$

$\underline{\mathsf{Sig}_\mathsf{m}.\mathsf{Sign}(\mathsf{PK}, \mathsf{SK}, m, \tau, \mathsf{par})}$
1: $(D_2^\tau, D_3^\tau) := \mathsf{H}_{\mathsf{ddh}}(\tau), \mathbf{D}^\tau = (D_1, D_2^\tau, D_3^\tau)$
2: $[\bar{\mathbf{H}}] := \mathsf{H}_\mathsf{m}(m), \quad \mathbf{s} \leftarrow\$ \mathcal{D}$
3: $\mathbf{S} := \phi_0([\bar{\mathbf{H}}], (\mathbf{s}, \mathbf{sk})) = (\mathbf{S}_1, \mathbf{S}_2, \mathbf{S}_3)$
4: $//\mathbf{S} = ([\bar{\mathbf{H}}] \circ \mathbf{sk} + [\mathbf{W}] \circ \mathbf{s}, [\mathbf{G}] \circ \mathbf{s}, \mathbf{X})^\top$
5: $\mathbb{x}_0 = ([\mathbf{G}], [\mathbf{W}], [\bar{\mathbf{H}}], \mathbf{S}), \mathbb{w}_0 = (\mathbf{s}, \mathbf{sk})$
6: $(\mathbf{A}_0, \mathsf{st}_0) := \mathsf{Init}_0(\mathbb{x}_0, \mathbb{w}_0)$
7: $\mathbb{x}_1 = (G, \mathbf{D}^\tau), \quad c_1 \leftarrow\$ \mathcal{S}$
8: $(\mathbf{A}_1, z_1) := \mathsf{Sim}_1(\mathbb{x}_1, c_1)$
9: $c := \mathsf{H}_\mathsf{c}((\mathbb{x}_b, \mathbf{A}_b)_{b \in \{0,1\}}, m)$
10: $c_0 := c - c_1, \quad \mathbf{z}_0 := \mathsf{Resp}_0(\mathsf{st}_0, c_0)$
11: $\pi = (\mathbf{A}_0, \mathbf{A}_1, c, c_0, \mathbf{z}_0, z_1)$
12: **return** $\sigma = (\mathbf{S}_1, \mathbf{S}_2, \pi)$

$\underline{\mathsf{Sig}_\mathsf{m}.\mathsf{Verify}(\mathsf{PK}, m, \tau, \sigma, \mathsf{par})}$
1: **parse** $(\mathbf{S}_1, \mathbf{S}_2, \pi) = \sigma$
2: $(D_2^\tau, D_3^\tau) := \mathsf{H}_{\mathsf{ddh}}(\tau), \mathbf{D}^\tau = (D_1, D_2^\tau, D_3^\tau)$
3: $\mathbf{S} = (\mathbf{S}_1, \mathbf{S}_2, \mathbf{X})^\top, [\bar{\mathbf{H}}] := \mathsf{H}_\mathsf{m}(m)$
4: $c' := \mathsf{H}_\mathsf{c}((\mathbb{x}_b, \mathbf{A}_b)_{b \in \{0,1\}}, m)$
5: $c_1 := c' - c_0$
6: **if** $\mathsf{Verify}_0(\mathbb{x}_0, \mathbf{A}_0, c_0, \mathbf{z}_0) = 0$: **return** 0
7: **if** $\mathsf{Verify}_1(\mathbb{x}_1, \mathbf{A}_1, c_1, z_1) = 0$: **return** 0
8: **return** 1

Fig. 7. Description of the plain signature Sig_m.

The following lemma proves that producing a tuple $(\mathbf{S}_1, \mathbf{S}_2)$ such that $([\mathbf{G}], [\mathbf{W}], [\bar{\mathbf{H}}], \mathbf{S} = (\mathbf{S}_1, \mathbf{S}_2, \mathbf{X})) \in \mathscr{L}_{\mathsf{bb}}$ is computationally hard without secret key $\mathsf{SK} = \mathbf{sk}$. This lemma is analogous to the unforgeability property of $\mathsf{Sig}_{\mathsf{m}}$, except that it ignores the proof π and τ, which can be proved by employing the puncturing strategy in [51]. The detailed proof of Lemma 1 is shown in Appendix C.

Lemma 1. (**Security of** $\mathsf{Sig}_{\mathsf{m}}$) *Assume that for* $\mathsf{TLF} = (\mathsf{Gen}, \circ)$, $\varepsilon_{\mathbf{t}} = \mathsf{negl}(\lambda)$ *and* $\varepsilon_{\mathbf{r}} = \mathsf{negl}(\lambda)$. *For any adversary* $\mathcal{A}$ *(makes at most* $Q_{\mathsf{H}_{\mathsf{m}}}$ *hash queries to* H_{m}*) and game* $\mathsf{Game}^{\mathsf{BB}}_{\mathsf{Sig}}$ *as follows, there is a PPT reduction* $\mathcal{B}$ *with* $T_{\mathcal{A}}(\lambda) \approx T_{\mathcal{B}}(\lambda)$ *and* $\mathsf{Adv}^{\mathsf{sec}}_{\mathcal{A},\mathsf{Sig}_{\mathsf{m}}}(\lambda) \leq Q_{\mathsf{H}_{\mathsf{m}}} \cdot \mathsf{Adv}^{\mathsf{cdh}}_{\mathcal{B},\mathsf{GGen}}(\lambda)$.

- *Sample* $\mathsf{SK} = \mathbf{sk} \leftarrow\$\ \mathcal{D}, [\mathbf{W}], [\mathbf{G}] \leftarrow\$\ \mathcal{T}$ *and set* $\mathbf{X} := [\mathbf{G}] \circ \mathbf{sk}$.
- *Run* $(m^\star, [\bar{\mathbf{H}}]^\star, \mathbf{S}_1^\star, \mathbf{S}_2^\star) \leftarrow \mathcal{A}^{\mathsf{H}_{\mathsf{m}}, \mathcal{O}_{\mathsf{Sig}_{\mathsf{m}}}}([\mathbf{G}], [\mathbf{W}], \mathbf{X})$, *and* H_{m}, $\mathcal{O}_{\mathsf{Sig}_{\mathsf{m}}}$ *as follows:*
  1. $\mathsf{H}_{\mathsf{m}}(m)$*: Sample* $[\bar{\mathbf{H}}] \leftarrow\$\ \mathcal{T}$, *set* $\mathcal{Q}_{\mathsf{H}_{\mathsf{m}}}[m] = [\bar{\mathbf{H}}]$ *and return* $\mathcal{Q}_{\mathsf{H}_{\mathsf{m}}}[m]$ *if no* m *has been queried. Otherwise, return* $\mathcal{Q}_{\mathsf{H}_{\mathsf{m}}}[m]$.
  2. $\mathcal{O}_{\mathsf{Sig}_{\mathsf{m}}}(m)$*: If* $\mathcal{Q}_{\mathsf{H}_{\mathsf{m}}}[m] = \perp$, *query* $\mathsf{H}_{\mathsf{m}}(m)$. *Otherwise, sample* $\mathbf{s} \leftarrow\$\ \mathcal{D}$, *compute* $\mathbf{S} := \phi_0([\bar{\mathbf{H}}], (\mathbf{s}, \mathbf{sk}))$. *Set* $\mathsf{queried} = \mathsf{queried} \cup \{m\}$, *return* $(\mathbf{S}_1, \mathbf{S}_2)$.
- *Set* $\mathbb{x}_0^\star := ([\mathbf{G}], [\mathbf{W}], [\bar{\mathbf{H}}]^\star, \mathbf{S}^\star = (\mathbf{S}_1^\star, \mathbf{S}_2^\star, \mathbf{X}))$, *if* $\mathbb{x}_0^\star \in \mathscr{L}_{\mathsf{bb}} \wedge \mathsf{H}_{\mathsf{m}}(m^\star) = [\bar{\mathbf{H}}]^\star \wedge m^\star \notin \mathsf{queried}$, *output 1.*

4 BS_{m}: Blind Signature with Mixed Extraction

We first design a special NP relation $\mathsf{R}_{\mathsf{ped}}$ for the scalar element (β, φ) and the m-related tag element $[\bar{\mathbf{H}}]$. Based on $\mathsf{R}_{\mathsf{ped}}$, we design Σ_{ped} protocol whose special soundness ensures the computation of $(\beta, \varphi, [\bar{\mathbf{H}}])$. By combining with straight line extraction, we construct a non-interactive proof system $\mathsf{NIPS}_{\mathsf{ped}}$, with an extractor $\mathsf{Ext}_{\mathsf{ped}}$ that can extract witness $(\beta, \varphi, [\bar{\mathbf{H}}])$.

Note that the extractor can only extract group element and scalar. It cannot force the adversary to query random oracle, nor can it rule out the strategy where the adversary does query the random oracle and then performs blinding twice. Consequently, the group elements extracted by the extractor may have no matching entry in the hash list. This, however, does not affect the ability to answer signing queries. Since the OR-composition in the security proof is programmed to the DDH-tuple branch, we can substitute the DLEQ elements with random group elements, which does not require the extracted group elements to appear in the hash list.

Utilizing $\mathsf{NIPS}_{\mathsf{ped}}$, we construct a BS scheme BS_{m}, serving as the foundation for our construction of adaptively secure TBS. Moreover, we prove the one-more unforgeability security of BS_{m} under the CDH assumption in the ROM.

4.1 NP Relation $\mathsf{R}_{\mathsf{ped}}$ and Σ_{ped} Protocol

We define $\mathsf{R}_{\mathsf{ped}}$ with included language $\mathscr{L}_{\mathsf{ped}}$ as follows:

$$\mathsf{R}_{\mathsf{ped}} := \{\mathbb{x}_{\mathsf{ped}}, \mathbb{w}_{\mathsf{ped}} : [\mathbf{E}] = \varphi * [\mathbf{G}] \wedge [\hat{\mathbf{H}}] = \beta * [\bar{\mathbf{H}}] \wedge [\mathbf{H}] - [\hat{\mathbf{H}}] = \beta * [\mathbf{E}]\},$$

where $\mathbb{x}_{\mathsf{ped}} = ([\mathbf{G}], [\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}]) \in \mathcal{T}^4$, $\mathbb{w}_{\mathsf{ped}} = ([\bar{\mathbf{H}}], \beta, \varphi) \in \mathcal{T} \times \mathcal{S}^2$. We instantiate Σ protocol to construct a mixed Σ_{ped} protocol as in Fig. 8 for proving statements with mixed witness. The security properties of Σ_{ped} protocol are shown in Appendix B.

$\mathsf{Init}_{\mathsf{ped}}(\mathbb{x}_{\mathsf{ped}}, \mathbb{w}_{\mathsf{ped}})$:
1: Sample $r_1, r_2 \leftarrow\$ \mathcal{S}$ and set $[\mathbf{A}_1] := r_1 * [\mathbf{G}]$, $[\mathbf{A}_2] := r_2 * [\mathbf{E}]$, $\mathbf{A} = ([\mathbf{A}_1], [\mathbf{A}_2])$
2: Output commitment $[\mathbf{A}_1], [\mathbf{A}_2]$ and state $\mathsf{st}_{\mathsf{ped}} := (\mathbb{w}_{\mathsf{ped}}, r_1, r_2)$
$\mathsf{Resp}_{\mathsf{ped}}(\mathsf{st}_{\mathsf{ped}}, \mathbf{c})$: Output $z_1 := r_1 + c_1 * \varphi$ and $z_2 := r_2 + c_2 * \beta$
$\mathsf{Verify}_{\mathsf{ped}}(\mathbb{x}_{\mathsf{ped}}, \mathbf{A}, \mathbf{c}, \mathbf{z})$: Output 1 if $z_1 * [\mathbf{G}] = [\mathbf{A}_1] + c_1 * [\mathbf{E}] \wedge z_2 * [\mathbf{E}] = [\mathbf{A}_2] +$
$c_2 * [\mathbf{H}] - c_2 * [\hat{\mathbf{H}}]$ and 0 otherwise. ($\mathbf{z} = (z_1, z_2), \mathbf{c} = (c_1, c_2)$)
$\mathsf{Sim}_{\mathsf{ped}}(\mathbb{x}_{\mathsf{ped}}, \mathbf{c})$: Sample $z_1, z_2 \leftarrow\$ \mathcal{S}$ and set $[\mathbf{A}_1] = z_1 * [\mathbf{G}] - c_1 * [\mathbf{E}]$,
$[\mathbf{A}_2] = z_2 * [\mathbf{E}] - c_2 * [\mathbf{H}] + c_2 * [\hat{\mathbf{H}}]$.

Fig. 8. $\Sigma_{\mathsf{ped}} = (\mathsf{Init}_{\mathsf{ped}}, \mathsf{Resp}_{\mathsf{ped}}, \mathsf{Verify}_{\mathsf{ped}})$ for $\mathsf{R}_{\mathsf{ped}}$.

4.2 Non-Interactive Proof System: $\mathsf{NIPS}_{\mathsf{ped}}$

We instantiate a non-interactive proof system $\mathsf{NIPS}_{\mathsf{ped}} = (\mathsf{Prove}, \mathsf{Ver})$ as shown in Fig. 9 from mixed Σ_{ped} protocol by applying the compilation method outlined in [20]. The correctness, zero-knowledge, witness-indistinguishability, and knowledge soundness of $\mathsf{NIPS}_{\mathsf{ped}}$ follow directly from [20].

$\mathsf{NIPS}_{\mathsf{ped}} = (\mathsf{Prove}, \mathsf{Ver})$ for $\mathsf{R}_{\mathsf{ped}}$ constructed from $\Sigma_{\mathsf{ped}} = (\mathsf{Init}_{\mathsf{ped}}, \mathsf{Resp}_{\mathsf{ped}}, \mathsf{Verify}_{\mathsf{ped}})$

$r \in \mathbb{N}$: repetitions; $tt \in \mathbb{N}$: iterations; $b \in \mathbb{N}$: $\mathsf{H}_{\mathsf{ped}} : \{0,1\}^* \to \{0,1\}^b$, k: challenge space of $\mathcal{CH}$, $k : \log(|\mathcal{CH}|)$. Typically, we set $r \geq \lceil \lambda / b \rceil$ for λ bits of security, and $tt \in \Theta(\log(\lambda) b \log(r))$ and $k = \omega(b)$ for negligible correctness error (and zero-knowledge); to ensure polynomial time prover, we need $2^{tt} = \mathsf{poly}(\lambda)$. The proof size only relies on r.

$\mathsf{Prove}^{\mathsf{H}_{\mathsf{ped}}}(\mathbb{x}, \mathbb{w})$:
1. For each $i \in [r]$, compute $(\mathbf{A}_i, \mathsf{st}_i) \leftarrow \Sigma_{\mathsf{ped}}.\mathsf{Init}_{\mathsf{ped}}(\mathbb{x}, \mathbb{w})$.
2. Let $\mathbf{A} = (\mathbf{A}_1, ..., \mathbf{A}_r)$
3. For $i = 1$, until $i > r$, or output $\perp$ if total of tt tries are exceeded:
   (a) Sample $\mathbf{c}_i \leftarrow\$ \mathcal{CH}$ uniformly without replacement within this iteration.
   (b) Compute $\mathbf{z}_i \leftarrow \Sigma_{\mathsf{ped}}.\mathsf{Resp}_{\mathsf{ped}}(\mathsf{st}_i, \mathbf{c}_i)$.
   (c) Query $h_i = \mathsf{H}_{\mathsf{ped}}(\mathbb{x}, \mathbf{A}, i, \mathbf{c}_i, \mathbf{z}_i)$.
   (d) If $h_i = 0^b$, set $i := i + 1$. (Continue to next repetition.)
   (e) Else: Go back to Step 3(a) and try again.
4. Return $\pi := (\mathbf{A}_i, \mathbf{c}_i, \mathbf{z}_i)_{i=1}^r$.

$\mathsf{Ver}^{\mathsf{H}_{\mathsf{ped}}}(\mathbb{x}, \pi)$:
1. Parse $\pi := (\mathbf{A}_i, \mathbf{c}_i, \mathbf{z}_i)_{i=1}^r$.
2. Compute $h_i = \mathsf{H}_{\mathsf{ped}}(\mathbb{x}, \mathbf{A}, i, \mathbf{c}_i, \mathbf{z}_i)$ for all $i \in [r]$.
3. If $h_i \neq 0^b$ for some $i \in [r]$, return 0.
4. If $\Sigma_{\mathsf{ped}}.\mathsf{Verify}_{\mathsf{ped}}(\mathbb{x}, \mathbf{A}_i, \mathbf{c}_i, \mathbf{z}_i) = 0$ for some $i \in [r]$, return 0.
5. Return 1.

Fig. 9. Non-interactive proof system $\mathsf{NIPS}_{\mathsf{ped}} = (\mathsf{Prove}, \mathsf{Ver})$.

4.3 $\mathsf{BS_m}$: Blind Signature with Mixed Extraction

Based on the plain signature scheme $\mathsf{Sig_m}$, utilizing blinding factors and the non-interactive proof system $\mathsf{NIPS_{ped}}$, we get the BS scheme $\mathsf{BS_m}$ by well design. $\mathsf{BS_m} = (\mathsf{BS_m.SysGen}, \mathsf{BS_m.KeyGen}, \mathsf{BS_m.Sign}, \mathsf{BS_m.Verify})$ is shown in Fig. 10.

Correctness. By correctness of $\mathsf{NIPS_{ped}}$, the protocol may fail with a probability bounded by γ_{err}, so $\mathsf{BS_m}$ is correct up to error γ_{err}.

Theorem 1. (**Computational blindness of** $\mathsf{BS_m}$) *For every PPT adversary* $\mathcal{A}$ *against the blindness game* $\mathsf{Game}^{\mathsf{Blind}}_{\mathsf{BS}}$ *of Definition 7, there exist PPT adversaries* $\mathcal{A}_{\mathsf{DDH}}$ *and* $\mathcal{A}_{\mathsf{ZK}}$ *such that* $\mathsf{Adv}^{\mathsf{BS\text{-}Blind}}_{\mathcal{A},\mathsf{BS_m}}(\lambda) \leq \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{A}_{\mathsf{DDH}}}(\lambda) + \mathsf{Adv}^{\mathsf{NIPS}}_{\mathcal{A}_{\mathsf{ZK}}}(\lambda)$.

The detailed proof is presented in Appendix D.1.

By employing relaxed knowledge sound to instantiate $\mathsf{NIPS_{ped}}$, the extractor $\mathsf{Ext_{ped}}$ extracts a witness corresponding to the relaxed knowledge relation:

$$\tilde{\mathsf{R}}_{\mathsf{ped}} := \{(\mathbb{x}, \mathbb{w}) | [\mathbf{G}] \circ \mathbb{w} = \mathbf{X} \vee (\mathbb{x}, \mathbb{w}) \in \mathsf{R_{ped}}\}, \text{ where } \mathbb{x} = ([\mathbf{G}], [\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}]). \quad (1)$$

Theorem 2. (BS-OMUF *of* $\mathsf{BS_m}$) *Assume that for* $\mathsf{TLF} = (\mathsf{Gen}, \circ)$, $\varepsilon_{\mathbf{t}} = \mathsf{negl}(\lambda)$ *and* $\varepsilon_{\mathbf{r}} = \mathsf{negl}(\lambda)$. *For any adversary* $\mathcal{A}$ *for the game* $\mathsf{Game}^{\mathsf{OMUF}}_{\mathsf{BS}}$ *making at most* Q_{S_1} *and* $l = Q_{S_2}$ *queries to* $\mathcal{O}^{\mathsf{ISign}_1}_{\mathsf{BS}}$ *and* $\mathcal{O}^{\mathsf{ISign}_2}_{\mathsf{BS}}$, *respectively, and* Q_{H_*} *queries to* $\mathsf{H}_* \in \{\mathsf{H_m}, \mathsf{H_c}, \mathsf{H_{ddh}}, \mathsf{H_{ped}}\}$, *modeled as random oracles, there are reductions* $\mathcal{A}_{\mathsf{KS}}$, $\mathcal{A}_{\mathsf{DL}}$, $\mathcal{A}_{\mathsf{DDH}}$ *and* $\mathcal{A}_{\mathsf{CDH}}$ *with similar running time to* $\mathcal{A}$ *such that*

$$\mathsf{Adv}^{\mathsf{BS\text{-}OMUF}}_{\mathcal{A},\mathsf{BS_m}}(\lambda) \leq \mathsf{Adv}^{\mathsf{NIPS_{ped}},\tilde{\mathsf{R}}_{\mathsf{ped}}}_{\mathcal{A}_{\mathsf{KS}}}(\lambda) + \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{A}_{\mathsf{DL}}}(\lambda) + \frac{\hat{Q}^2_{\mathsf{H_m}}}{p} + 2\mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{A}_{\mathsf{DDH}}}(\lambda) + \frac{4}{p-1}$$
$$+ \hat{Q}_{\mathsf{H_{ddh}}}\hat{Q}_{\mathsf{H_m}}\big(\frac{2}{p-1} + 3\mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{A}_{\mathsf{DDH}}}(\lambda) + \frac{\hat{Q}_{\mathsf{H_c}}}{p} + \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{A}_{\mathsf{CDH}}}(\lambda)\big)$$

where $\hat{Q}_{\mathsf{H_m}} = Q_{\mathsf{H_m}} + l + 1$, $\hat{Q}_{\mathsf{H_c}} = Q_{\mathsf{H_c}} + l + 1$, $\hat{Q}_{\mathsf{H_{ddh}}} = Q_{\mathsf{H_{ddh}}} + 1$.

The proof consists of a series of games, we give a brief idea: The simulator utilize the extractor $\mathsf{Ext_{ped}}$ of $\mathsf{NIPS_{ped}}$ to extract $[\bar{\mathbf{H}}]$ from the first protocol message $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}], \pi_{\mathsf{ped}})$ sent by the adversary. By programming $\mathsf{H_m}$, the simulator binds the signing session to the message m and hashed message $[\bar{\mathbf{H}}]$. The simulator randomly chooses two values q^*_τ and q^*_m that correspond to the common message τ^* and the special message $[\bar{\mathbf{H}}]^\star$ in "one-more" output by the adversary, respectively. The simulator performs the following two settings for $\mathbf{T}^\diamond$ in $\mathbb{x}_0$ within $\mathcal{O}^{\mathsf{ISign}_1}_{\mathsf{BS}}$: Sampling random elements for $\mathbf{T}^\diamond$ when $\tau \neq \tau^*$ or $[\bar{\mathbf{H}}] = [\bar{\mathbf{H}}]^\star$; Computing $\mathbf{T}^\diamond$ honestly (requires $\mathbf{sk}$) when $\tau = \tau^*$ and $[\bar{\mathbf{H}}] \neq [\bar{\mathbf{H}}]^\star$, the simulator fails but with negligible probability. Then, the simulator programs $\mathsf{H_{ddh}}$ making $\mathbb{x}_1$ is a valid statement for the NP relation $\mathsf{R_{ddh}}$. At this point the simulator can produce a valid proof for the disjunctive relation $\mathsf{R_{bb}} \vee \mathsf{R_{ddh}}$ by honestly computing Σ_1 and simulating Σ_0. Finally the simulator outputs the particular adversary's forgery to the security game of $\mathsf{Sig_m}$, thus reducing to the CDH assumption. The detailed proof is shown in Appendix D.2.

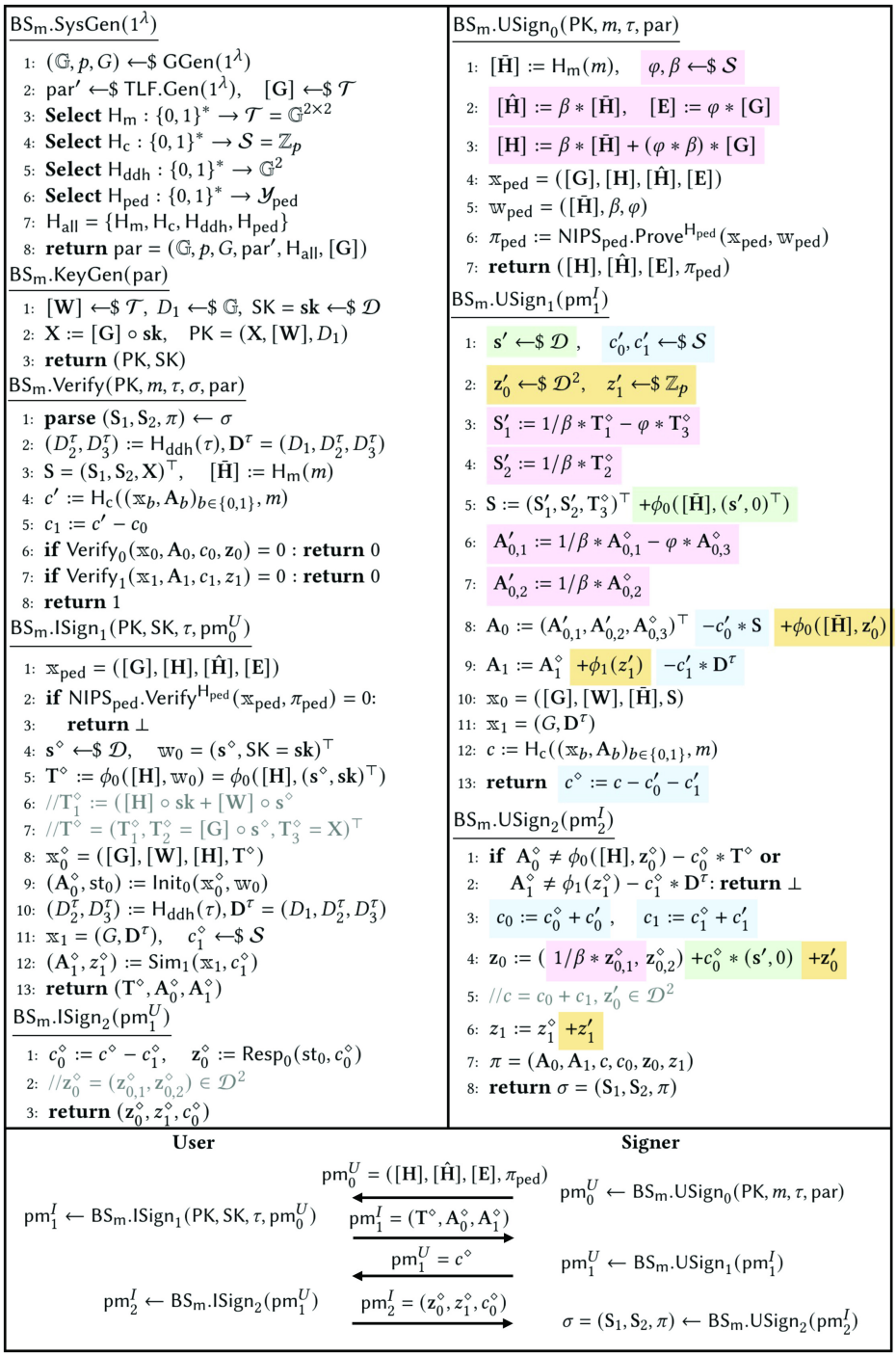

Fig. 10. Blind signature scheme BS_m and its interaction process between signer and user. The blinding factors are highlighted with different colors.

5 Rainblind: Fully Adaptive TBS Scheme

We propose Rainblind, the first TBS scheme achieving fully adaptive security without AGM. Rainblind generalizes single-party blind signature scheme BS_m to the multi-issuer framework. The user acts as the protocol's coordinator; every issuer in set $\mathcal{SS}$ communicates with the user, who relays messages among issuers over three signing rounds. Finally, the user collects and combines the signature shares provided by issuers, then publishes the aggregated signature. Specifically, we utilize a centralized key generation mechanism, though alternatively, a distributed key generation protocol (n issuers jointly generate PK and $\{\mathsf{PK}_i\}_1^n$ while each issuer obtains only SK_i) could be employed in its place. Suppose an outside mechanism identifies the issuers set $\mathcal{SS} \subseteq \{1, ..., n\}$, where $t+1 \leq |\mathcal{SS}| \leq n$ and $\mathcal{SS}$ is arranged for consistency. Rainblind integrates a EUF-CMA-secure signature scheme DS, authenticating messages exchanged in the signing rounds.

5.1 Construction of Rainblind

$\mathsf{Rainblind} = (\mathsf{Rainblind.SysGen}, \mathsf{Rainblind.KeyGen}, \mathsf{Rainblind.Sign}, \mathsf{Rainblind.Verify})$ is shown in Fig. 11.

Correctness. Rainblind is correct with error γ_{err} (the correctness error of $\mathsf{NIPS}_{\mathsf{ped}}$).

5.2 Security Proof of Rainblind

We defer the computational blindness of Rainblind to Appendix E.

Theorem 3. (TBS-OMUF-SB *of* Rainblind) *Assume that for* $\mathsf{TLF} = (\mathsf{Gen}, \circ)$, $\varepsilon_{\mathbf{t}} = \mathsf{negl}(\lambda)$ *and* $\varepsilon_{\mathbf{r}} = \mathsf{negl}(\lambda)$. *For an adversary* $\mathcal{A}$ *against the game* $\mathsf{Game}^{\mathsf{OMUF\text{-}SB}}_{\mathsf{TBS}}$ *with running time* $T_{\mathcal{A}}$, *making at most* Q_{S_i} *queries to* $\mathcal{O}^{\mathsf{ISign}_i}_{\mathsf{TBS}}$ $(i \in \{1,2,3\})$ *and* Q_{H_*} *queries to* $\mathsf{H}_* \in \{\mathsf{H}_\mathsf{m}, \mathsf{H}_\mathsf{c}, \mathsf{H}_{\mathsf{ped}}, \mathsf{H}_{\mathsf{ddh}}, \mathsf{H}_{\mathsf{cm}}\}$, *modeled as random oracles, there is a reduction* $\mathcal{C}$ *with* $T_{\mathcal{C}} \approx T_{\mathcal{A}}$ *for the game* $\mathsf{Game}^{t\text{-A-TRAN-RES}}_{\mathsf{TLF}}$ *making at most* t *queries to* $\mathcal{O}^{\mathsf{Inv}}_{\mathsf{TLF}}$, *a reduction* $\mathcal{F}$ *with* $T_{\mathcal{F}} \approx T_{\mathcal{A}}$ *for the game* $\mathsf{Game}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{DS}}$ *making at most* Q_{S_2} *queries to* $\mathcal{O}^{\mathsf{Sign}}_{\mathsf{DS}}$, *and reductions* $\mathcal{A}_{\mathsf{KS}}$, $\mathcal{A}_{\mathsf{DL}}$, $\mathcal{A}_{\mathsf{DDH}}$ *and* $\mathcal{A}_{\mathsf{CDH}}$ *that have running time comparable to* $\mathcal{A}$ *such that*

$$\mathsf{Adv}^{\mathsf{TBS\text{-}OMUF\text{-}SB}}_{\mathcal{A},\mathsf{Rainblind}}(\lambda) \leq \mathsf{Adv}^{\mathsf{NIPS}_{\mathsf{ped}},\tilde{\mathsf{R}}_{\mathsf{ped}}}_{\mathcal{A}_{\mathsf{KS}}}(\lambda) + \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{A}_{\mathsf{DL}}}(\lambda) + \frac{4}{p-1} + +3n \cdot \hat{Q}_{\mathsf{H}_{\mathsf{ddh}}} \hat{Q}_{\mathsf{H}_\mathsf{m}} \mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathcal{F},\mathsf{DS}}$$

$$\hat{Q}_{\mathsf{H}_{\mathsf{ddh}}} \hat{Q}_{\mathsf{H}_\mathsf{m}} \left(\mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{A}_{\mathsf{DDH}}}(\lambda) + \frac{2}{p-1} + \frac{\hat{Q}_{\mathsf{H}_\mathsf{c}}}{p} + \mathsf{Adv}^{t\text{-A-TRAN-RES}}_{\mathcal{C},\mathsf{TLF}}(\lambda) \right) + 2\mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{A}_{\mathsf{DDH}}}(\lambda) + \frac{\hat{Q}^2_{\mathsf{H}_\mathsf{m}} + Q_{S_3}}{p},$$

where $\hat{Q}_{\mathsf{H}_\mathsf{m}} = Q_{\mathsf{H}_\mathsf{m}} + l + 1$, $\hat{Q}_{\mathsf{H}_\mathsf{c}} = Q_{\mathsf{H}_\mathsf{c}} + l + 1$, $\hat{Q}_{\mathsf{H}_{\mathsf{ddh}}} = Q_{\mathsf{H}_{\mathsf{ddh}}} + 1$.

Proof. The proof proceeds via a game sequence $\mathbf{G}^{\mathsf{tbs}}_0$-$\mathbf{G}^{\mathsf{tbs}}_{13}$. Firstly, $\mathbf{G}^{\mathsf{tbs}}_1$-$\mathbf{G}^{\mathsf{tbs}}_{12}$ answer the signing queries without secret keys. Secondly, $\mathbf{G}^{\mathsf{tbs}}_{13}$ excludes the case where the signatures $\{\sigma_j\}_{j \in \mathcal{SS}}$ in $\mathcal{O}^{\mathsf{ISign}_3}_{\mathsf{TBS}}$ are forged. We reduce $\mathbf{G}^{\mathsf{tbs}}_{13}$ to the t-A-TRAN-RES assumption (implies DDH assumption [22]) of TLF, using its inversion oracle $\mathcal{O}^{\mathsf{Inv}}_{\mathsf{TLF}}$ to answer the adaptive secret key corruption queries.

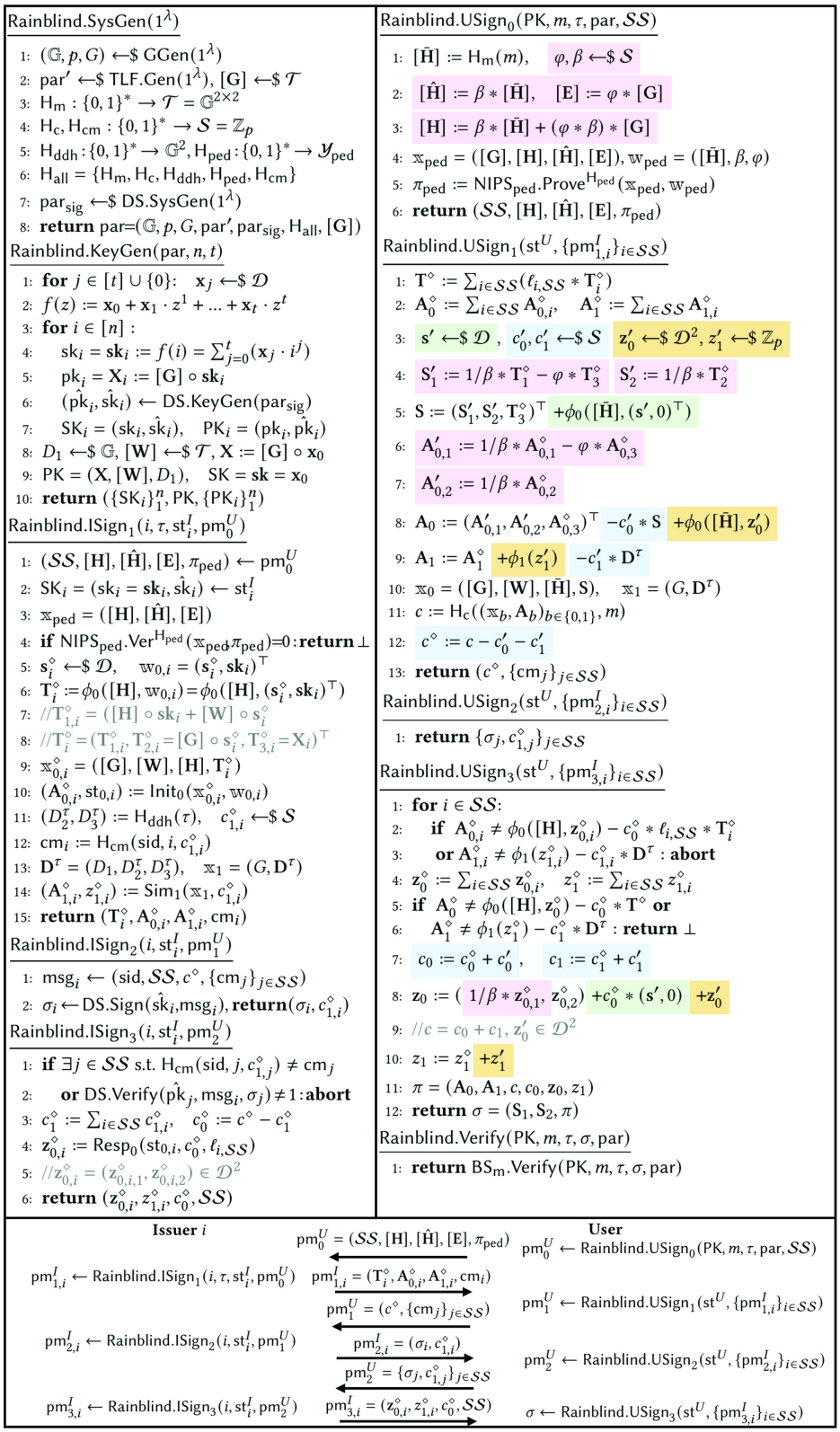

Fig. 11. Threshold blind signature scheme Rainblind and its interaction process between issuers and the user. The blinding factors are highlighted with different colors.

Game $\mathbf{G}_0^{\mathsf{tbs}}$: $\mathbf{G}_0^{\mathsf{tbs}}$ is shown in Fig. 12. It runs $\mathsf{par} \leftarrow \mathsf{Rainblind.SysGen}(1^\lambda)$, then interacts with an adversary $\mathcal{A}(\mathsf{par})$ to get $(n, t, \mathsf{Corrupted})$. The game runs $(\{\mathsf{SK}_i\}_1^n, \mathsf{PK}, \{\mathsf{PK}_i\}_1^n) \leftarrow \mathsf{Rainblind.KeyGen}(\mathsf{par}, n, t)$. Next, the game interacts with an adversary $\mathcal{A}(\mathsf{par}, \mathsf{PK}, \{\mathsf{PK}_j\}_1^n, \{\mathsf{SK}_i\}_{i\in\mathsf{Corrupted}})$ with access to the signing oracles $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_1}, \mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_2}, \mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_3}$, and random oracles $\mathsf{H_m}$, $\mathsf{H_c}$, $\mathsf{H_{ddh}}$, $\mathsf{H_{ped}}$, $\mathsf{H_{cm}}$ (by lazy sampling), and corruption oracle $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{Corr}}$. Among them, the number of accessing are Q_{S_1}, Q_{S_2}, Q_{S_3}, and Q_{H_*} for $\mathsf{H}_* \in \{\mathsf{H_m}, \mathsf{H_c}, \mathsf{H_{ddh}}, \mathsf{H_{ped}}, \mathsf{H_{cm}}\}$. Finally, adversary $\mathcal{A}$ outputs $(\tau^*, \{(m_k^*, \sigma_k^*)\}_{k\in[l+1]})$. If for all $k_1 \neq k_2$, $m_{k_1}^* \neq m_{k_2}^*$, for all $k \in [l+1]$, $\mathsf{Rainblind.Verify}(\mathsf{PK}, m_k^*, \tau^*, \sigma_k^*, \mathsf{par}) = 1$, the number of corrupted issuers $|\mathsf{Corrupted}| \leq t$, and the number of legitimate signatures against common message τ^* is at most l, i.e., $|\mathsf{ES}_{\mathsf{SB}}[\tau^*]| \leq l$, then $\mathcal{A}$ succeeds.

We make several purely conceptual changes to the game. We define an initial empty set Cot_0. If issuer j is corrupted before $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_1}$, add j to the set Cot_0. For all values $(\cdot)^{\mathsf{sid}}$, we omit their superscript sid. For any value $(\cdot)$, we default the value is in some session sid. We suppose that the adversary is honest but curious. Assume that while verifying the forgeries, the adversary $\mathcal{A}$ also performs queries to random oracles, i.e., $\hat{Q}_{\mathsf{H_m}} = Q_{\mathsf{H_m}} + l + 1$, $\hat{Q}_{\mathsf{H_c}} = Q_{\mathsf{H_c}} + l + 1$, $\hat{Q}_{\mathsf{H_{ddh}}} = Q_{\mathsf{H_{ddh}}} + 1$ for $\mathsf{H_m}$,$\mathsf{H_c}$ and $\mathsf{H_{ddh}}$, respectively. We suppose the adversary submits exactly t (distinct) corruption queries. We have $\mathsf{Adv}_{\mathcal{A},\mathsf{Rainblind}}^{\mathsf{TBS\text{-}OMUF\text{-}SB}}(\lambda) = \Pr[\mathbf{G}_0^{\mathsf{tbs}} \Rightarrow 1]$.

Game $\mathbf{G}_1^{\mathsf{tbs}}$: (**Change the response of $\mathsf{H_m}$ and ensure the observability of $\mathsf{H_{cm}}$**). We modify the way $\mathsf{H_m}$ responds, from $[\bar{\mathbf{H}}] \leftarrow\$\ \mathcal{T}$ to $([\bar{\mathbf{H}}], \mathsf{td} = \mathbf{r}) \leftarrow \mathsf{Shift}(\mathsf{par}', [\mathbf{G}])$. Additionally, if there are collisions in $\mathsf{H_m}$, the game aborts. Moreover, we abort if $\mathcal{A}$ breaks the observability of the random oracle $\mathsf{H_{cm}}$. More precisely, the game aborts if for any signing session sid and $j \in \mathsf{Corrupted}$, $\mathcal{A}$ manages to provide a value $c_{1,j}^\diamond$ in $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_3}$ for its commitment cm_j in $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_2}$ such that $\mathsf{cm}_j = \mathsf{H_{cm}}(\mathsf{sid}, j, c_{1,j}^\diamond)$, without having queried the random oracle $\mathsf{H_{cm}}$ on $(\mathsf{sid}, j, c_{1,j}^\diamond)$. This will help to extract the adversary's own sampled $c_{1,j}^\diamond$ from the transcripts of $\mathsf{H_{cm}}$, by utilizing the adversary's submitted cm_j in $\mathbf{G}_5^{\mathsf{tbs}}$ and $\mathbf{G}_{10}^{\mathsf{tbs}}$. Note that for each $j \in \mathsf{Corrupted}$, unless $\mathsf{H_{cm}}(\cdot, j, \cdot) \neq \perp$, the game outputs $\mathsf{H_{cm}}(\cdot, j, \cdot)$ with uniformly random values in $\mathbb{Z}_p$. Therefore, the probability of $\mathsf{H_{cm}}(\mathsf{sid}, j, c_{1,j}^\diamond) = \mathsf{cm}_j$ for each $(\mathsf{sid}, j, c_{1,j}^\diamond)$ is $1/p$. Thus, we have $\left|\Pr[\mathbf{G}_0^{\mathsf{tbs}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathsf{tbs}} \Rightarrow 1]\right| \leq (\hat{Q}_{\mathsf{H_m}}^2 + Q_{S_3})/p$.

Game $\mathbf{G}_2^{\mathsf{tbs}}$: (**Extract** $([\bar{\mathbf{H}}], \beta, \varphi)$ **from** $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$). In this game, we change $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_1}$: we utilize the extractor $\mathsf{Ext_{ped}}$ of $\mathsf{NIPS_{ped}}$ to extract $([\bar{\mathbf{H}}], \beta, \varphi) \in \mathcal{T} \times \mathcal{S}^2$ from $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}], \pi_{\mathsf{ped}})$ sent by the adversary. If π_{ped} passes validation but extraction fails, the game aborts. Same as $\mathbf{G}_2^{\mathsf{bs}}$, we have that $\left|\Pr[\mathbf{G}_1^{\mathsf{tbs}} \Rightarrow 1] - \Pr[\mathbf{G}_2^{\mathsf{tbs}} \Rightarrow 1]\right| \leq \mathsf{Adv}_{\mathcal{A}_{\mathsf{KS}}}^{\mathsf{NIPS_{ped}},\tilde{\mathsf{R}}_{\mathsf{ped}}}(\lambda) + \mathsf{Adv}_{\mathcal{A}_{\mathsf{DL}}}^{\mathsf{GGen}}(\lambda) + 4/(p-1) + \sum_{i\in[2]} \mathsf{Adv}_{\mathcal{A}_{\mathsf{DDH}}^i}^{\mathsf{GGen}}(\lambda)$.

Game $\mathbf{G}_3^{\mathsf{tbs}}$ (**Guess** τ^*). We guess the first query to $\mathsf{H_{ddh}}$ with τ^* as input. The game randomly choose $q_\tau^* \leftarrow\$\ [\hat{Q}_{\mathsf{H_{ddh}}}]$ at the beginning. The game also determines whether τ^* was first queried to $\mathsf{H_{ddh}}$ on the q_τ^*-th query to $\mathsf{H_{ddh}}$ after the adversary produces its forgeries with the common message τ^* finally. If not, the game aborts. It is easy to obtain that $\Pr[\mathbf{G}_2^{\mathsf{tbs}} \Rightarrow 1] \leq \hat{Q}_{\mathsf{H_{ddh}}} \cdot \Pr[\mathbf{G}_3^{\mathsf{tbs}} \Rightarrow 1]$.

Fig. 12. Game $\mathbf{G}_0^{\mathsf{tbs}}$. It is the same as $\mathsf{Game}_{\mathsf{TBS}}^{\mathsf{OMUF-SB}}$.

Game $\mathbf{G}_4^{\mathsf{tbs}}$ (Guess unsigned $[\bar{\mathbf{H}}]^*$ in forgery). Here, we guess the first query q_m^* to H_{m} that satisfies both of the below conditions:

1. The adversary $\mathcal{A}$'s forgeries include the input $m_{q_m^*}$ of the q_m^*-th $\mathsf{H_m}$ query.
2. If $[\bar{\mathbf{H}}] = \mathsf{H_m}(m_{q_m^*})$ can be extracted from the commitment $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$ (in Game $\mathbf{G}_2^{\mathsf{tbs}}$), then no session about common message τ^* is finished.

If the guess is wrong then the game aborts. Adversary $\mathcal{A}$'s forgeries $(m_k^*, \sigma_k^*)_{k\in[l+1]}$ with common message τ^* consist of $l+1$ different messages if $\mathcal{A}$ succeeds. The hashed messages $([\bar{\mathbf{H}}]_k^*)_{k\in[l+1]}$ for $[\bar{\mathbf{H}}]_k^* = \mathsf{H_m}(m_k^*)$ also differ because Game $\mathbf{G}_1^{\mathsf{tbs}}$ has excluded $\mathsf{H_m}$ collisions. Moreover, with respect to common message τ^*, there are at most l completed sessions, each of which can extract a hashed message $[\bar{\mathbf{H}}] \in \mathcal{T}$ from the commitment $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$ via $\mathsf{Ext_{ped}}$. Thus, one of the $l+1$ different hashed messages $([\bar{\mathbf{H}}]_k^*)_{k\in[l+1]}$ has never been extracted from $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$ in a completed session. With the game's queries included, there is a $j \in [l+1]$ such that $m_{q_m^*} = m_j^*$ that satisfies both of the conditions mentioned above. Note that q_m^* is concealed from the adversary, thus the probability that the adversary correctly guesses q_m^* is at most $1/\hat{Q}_{\mathsf{H_m}}$. Thus, we have $\Pr[\mathbf{G}_3^{\mathsf{tbs}} \Rightarrow 1] \leq \hat{Q}_{\mathsf{H_m}} \cdot \Pr[\mathbf{G}_4^{\mathsf{tbs}} \Rightarrow 1]$.

For the query m_i to $\mathsf{H_m}$, if $i = q_m^*$, we set $m_{q_m^*} = m_i$ and $[\bar{\mathbf{H}}]^\star := \mathsf{H_m}(m_{q_m^*})$. If $\mathcal{A}$ succeeds, we suppose the game knows $[\bar{\mathbf{H}}]^\star$ from the beginning. The game can choose $[\bar{\mathbf{H}}]^\star$ at the start and output $[\bar{\mathbf{H}}]^\star$ in the q_m^*-th query against $\mathsf{H_m}$. We emphasize that the game aborts only when a signing interaction is completed, i.e., the signing oracles $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_1}$, $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_2}$ and $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_3}$ have all been executed with extracted $[\bar{\mathbf{H}}]^\star$ and τ^*. This is because, $[\bar{\mathbf{H}}]^\star$ can be extracted and executed only in the signing oracle $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_1}$, at which point the session does not get completed.

Game $\mathbf{G}_5^{\mathsf{tbs}}$ (**Simulate Σ_0 if $\tau \neq \tau^*$**). For $\tau \neq \tau^*$, we simulate the transcript $(\mathbf{A}_{i,0}^\diamond, c_0^\diamond, \mathbf{z}_{i,0}^\diamond)$ of Σ_0 by the HVZK of Σ_0. Within $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_1}$ for $\tau \neq \tau^*$, the game samples $\mathsf{cm}_i \leftarrow\$\ \mathcal{S}$. Moreover, the game resamples cm_i if it has already been sampled, i.e., $\exists\ \mathsf{H_{cm}}[\cdot,\cdot,\cdot] = \mathsf{cm}_i$. Then the game computes $\mathbf{A}_{0,i}^\diamond$ via $c_0^\diamond \leftarrow\$\ \mathcal{S}, (\mathbf{A}_{0,i}^\diamond, \mathbf{z}_{0,i}^\diamond) := \mathsf{Sim}_0(\mathbb{x}_{0,i}^\diamond, c_0^\diamond, \ell_{i,\mathcal{SS}})$. Within $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_2}$ for $\tau \neq \tau^*$: after receiving $(c^\diamond, \{\mathsf{cm}_j\}_{j\in\mathcal{SS}})$, for $j \in \mathsf{Cot}_0 \cap \mathcal{SS}$, the game utilizes the observability of $\mathsf{H_{cm}}$ (which is guaranteed in $\mathbf{G}_1^{\mathsf{tbs}}$) to extract $c_{1,j}^\diamond$ based on cm_j. Then, the game samples $c_{1,j}^\diamond$ for $j \in \mathcal{SS}\backslash\mathsf{Cot}_0$ such that $c_0^\diamond + \sum_{j\in\mathcal{SS}\cap\mathsf{Cot}_0} c_{1,j}^\diamond + \sum_{j\in\mathcal{SS}\backslash\mathsf{Cot}_0} c_{1,j}^\diamond = c^\diamond$ and sets $\mathsf{H_{cm}}[\mathsf{sid}, j, c_{1,j}^\diamond] = \mathsf{cm}_j$ (if $\exists \mathsf{H_{cm}}[\cdot,\cdot,c_{1,j}^\diamond] = \mathsf{cm}_j$, resamples $c_{1,j}^\diamond$). Within $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_3}$ for $\tau \neq \tau^*$, the game sets $c_1^\diamond := c^\diamond - c_0^\diamond = \sum_{j\in\mathcal{SS}} c_{1,j}^\diamond$ and returns $\mathbf{z}_{0,i}^\diamond$ from $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_1}$. Therefore, in Game $\mathbf{G}_4^{\mathsf{tbs}}$ and Game $\mathbf{G}_5^{\mathsf{tbs}}$, the challenges $(c_0^\diamond, \{c_{1,j}^\diamond\}_{j\in\mathcal{SS}})$ have the same distribution. Note that the Σ_0 transcript in Game $\mathbf{G}_4^{\mathsf{tbs}}$ is computed honestly. Consequently, using Σ_0's perfect HVZK, we have that $\Pr[\mathbf{G}_4^{\mathsf{tbs}} \Rightarrow 1] = \Pr[\mathbf{G}_5^{\mathsf{tbs}} \Rightarrow 1]$.

Game $\mathbf{G}_6^{\mathsf{tbs}}$ (**Sample DDH tuples if $\tau \neq \tau^*$**). In this game, for $\mathsf{H_{ddh}}$, we sample the real DDH tuple, except in the q_τ^*-th query. When $\mathsf{H_{ddh}}$ is queried with τ and the hash value remains undefined, the game randomly chooses $d_2 \leftarrow\$\ \mathbb{Z}_p$, sets $(D_2^\tau, D_3^\tau) := (d_2 \cdot G, d_2 \cdot D_1)$ rather than $(D_2, D_3) \leftarrow\$\ \mathbb{G}^2$. Moreover, the game stores the witness d_2 in a table, i.e., $\mathsf{T_{ddh}}[\tau] := d_2$. Notice that the results (i.e., $(D_2^{\tau^*}, D_3^{\tau^*})$) of the q_τ^*-th query against $\mathsf{H_{ddh}}$ and further queries using the same input remain unchanged. Thus, for $\tau \neq \tau^*$, we have $\mathbf{D}^\tau \in \mathscr{L}_{\mathsf{ddh}}$. We can

construct a reduction $\mathcal{B}_1$ for Q-DDH, where $Q = \hat{Q}_{\mathsf{ddh}}$, with $T_{\mathcal{A}}(\lambda) \approx T_{\mathcal{B}_1}(\lambda)$ and $\left|\Pr[\mathbf{G}_5^{\mathsf{tbs}} \Rightarrow 1] - \Pr[\mathbf{G}_6^{\mathsf{tbs}} \Rightarrow 1]\right| \leq \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{B}_1,\mathsf{Q\text{-}DDH}}(\hat{Q}_{\mathsf{ddh}}, \lambda)$.

Game $\mathbf{G}_7^{\mathsf{tbs}}$ (**Use DDH witness for Σ_1 if $\tau \neq \tau^*$**). For $\tau \neq \tau^*$, we compute Σ_1 transcript $(\mathbf{A}^{\diamond}_{1,i}, z^{\diamond}_{1,i})$ using the witness $\mathsf{T}_{\mathsf{ddh}}[\tau]$. Specifically, within $\mathcal{O}^{\mathsf{ISign}_1}_{\mathsf{TBS}}$ for $\tau \neq \tau^*$, the game generates $(\mathbf{A}^{\diamond}_{1,i}, \mathsf{st}^{\diamond}_{1,i}) := \mathsf{Init}_1(\mathbb{x}_1, \mathbb{w}_1)$, where $\mathbb{w}_1$ is given by $\mathsf{T}_{\mathsf{ddh}}[\tau]$ and $\mathbb{x}_1$ equals $(G, \mathbf{D}^{\tau})$. Recall that in Game $\mathbf{G}_5^{\mathsf{tbs}}$, the game samples $c^{\diamond}_{1,j}$ for $j \in \mathcal{SS}$ such that $c^{\diamond}_0 + \sum_{j\in\mathcal{SS}} c^{\diamond}_{1,j} = c^{\diamond}$ after receiving $(c^{\diamond}, \{\mathsf{cm}_j\}_{j\in\mathcal{SS}})$ and sets $\mathsf{H}_{\mathsf{cm}}(\mathsf{sid}, i, c^{\diamond}_{1,i}) = \mathsf{cm}_i$ in $\mathcal{O}^{\mathsf{ISign}_2}_{\mathsf{TBS}}$ with $\tau \neq \tau^*$. Within $\mathcal{O}^{\mathsf{ISign}_3}_{\mathsf{TBS}}$ for $\tau \neq \tau^*$, the game generates $z^{\diamond}_{1,i} := \mathsf{Resp}_1(\mathsf{st}_{1,i}, c^{\diamond}_{1,i})$. From the perfect HVZK of Σ_1, we can conclude that the distribution of transcript $(\mathbf{A}^{\diamond}_{1,i}, z^{\diamond}_{1,i})$ of Σ_1 is the identical between Game $\mathbf{G}_6^{\mathsf{tbs}}$ and Game $\mathbf{G}_7^{\mathsf{tbs}}$. Thus, we have $\Pr[\mathbf{G}_6^{\mathsf{tbs}} \Rightarrow 1] = \Pr[\mathbf{G}_7^{\mathsf{tbs}} \Rightarrow 1]$.

We divide Game $\mathbf{G}_7^{\mathsf{tbs}}$ into two cases, (1) $\tau \neq \tau^*$, or (2) $\tau = \tau^*$ and $[\bar{\mathbf{H}}] = [\bar{\mathbf{H}}]^{\star}$ has been extracted from $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$. Remember that in $\mathcal{O}^{\mathsf{ISign}_1}_{\mathsf{TBS}}$, the game computes $\mathbf{T}^{\diamond}_i := \phi_0([\bar{\mathbf{H}}], \mathbb{w}_{0,i})$, where $\mathbb{w}_{0,i} = (\mathbf{s}^{\diamond}_i, \mathbf{sk}_i)$ and $\mathbf{s}^{\diamond}_i \in \mathcal{D}$ is random. Note that when case (1) occurs, the game uses $\mathbb{w}_{0,i} = (\mathbf{s}^{\diamond}_i, \mathbf{sk}_i)$ only to sample $\mathbf{T}^{\diamond}_i$ by the change in Game $\mathbf{G}_7^{\mathsf{tbs}}$, and nothing else in the oracle. Similarly, if case (2) occurs, the game uses $\mathbb{w}_{0,i} = (\mathbf{s}^{\diamond}_i, \mathbf{sk}_i)$ to sample $\mathbf{T}^{\diamond}_i$ and compute $\mathbf{z}^{\diamond}_{0,i}$ in $\mathcal{O}^{\mathsf{ISign}_3}_{\mathsf{TBS}}$. The case (2) doesn't happen in $\mathcal{O}^{\mathsf{ISign}_3}_{\mathsf{TBS}}$ due to the setting in Game $\mathbf{G}_4^{\mathsf{tbs}}$. In short, the game only utilizes $\mathbb{w}_{0,i}$ to sample $\mathbf{T}^{\diamond}_i$ when case (1) or (2) occurs.

Game $\mathbf{G}_8^{\mathsf{tbs}}$ (**Output random $\mathbf{T}^{\diamond}_i$ in some sessions**). For the specific cases (1) and (2) discussed above, we modify the signing oracle $\mathcal{O}^{\mathsf{ISign}_1}_{\mathsf{TBS}}$. In the previous game, the signing oracle computes $\mathbf{T}^{\diamond}_i := \phi_0([\bar{\mathbf{H}}], \mathbb{w}_{0,i})$. In this game, if case (1) or case (2) happens, the game randomly samples $(\mathbf{T}^{\diamond}_{i,1}, \mathbf{T}^{\diamond}_{i,2}) \leftarrow \mathcal{R}^2$ and sets $\mathbf{T}^{\diamond}_{i,3} = \mathbf{X}_i$. Note that if $[\bar{\mathbf{H}}] = [\bar{\mathbf{H}}]^{\star}$, the oracle $\mathcal{O}^{\mathsf{ISign}_3}_{\mathsf{TBS}}$ aborts, as introduced in $\mathbf{G}_4^{\mathsf{tbs}}$. At an intuitive level, since $([\mathbf{W}], [\mathbf{G}]\circ\mathbf{s}^{\diamond}_i, [\mathbf{W}]\circ\mathbf{s}^{\diamond}_i)$ within $\mathbf{T}^{\diamond}_i$ implies a DDH tuple, replacing $[\mathbf{W}] \circ \mathbf{s}^{\diamond}_i$ with a random value should be indistinguishable, and this would cause the first part of $\mathbf{T}^{\diamond}_i$ to be random. Same as $\mathbf{G}_8^{\mathsf{bs}}$, there exists a reduction $\mathcal{B}_2$ that breaks Q-DDH given that $\mathcal{A}$ distinguishes between Game $\mathbf{G}_7^{\mathsf{tbs}}$ and Game $\mathbf{G}_8^{\mathsf{tbs}}$. We have $\left|\Pr[\mathbf{G}_7^{\mathsf{tbs}} \Rightarrow 1] - \Pr[\mathbf{G}_8^{\mathsf{tbs}} \Rightarrow 1]\right| \leq \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{B}_2,\mathsf{Q\text{-}DDH}}(Q_S, \lambda)$.

Game $\mathbf{G}_9^{\mathsf{tbs}}$ (**Abort if forgeries not in $\mathscr{L}_{\mathsf{bb}}$**). In this game, if any of the adversary's forgeries $\sigma^*_k = (\mathbf{S}^*_{1,k}, \mathbf{S}^*_{2,k}, \pi^*_k)$ for message m^*_k fulfills $([\mathbf{G}], [\mathbf{W}], [\bar{\mathbf{H}}]^*_k, \mathbf{S}^*_{1,k}, \mathbf{S}^*_{2,k}, \mathbf{X}) \notin \mathscr{L}_{\mathsf{bb}}$ with $[\bar{\mathbf{H}}]^*_k = \mathsf{H}_{\mathsf{m}}(m^*_k)$, the game aborts. This is achieved efficiently by the following: The game computes $([\mathbf{W}], \mathsf{td}_w = \mathbf{r}_w) \leftarrow \mathsf{Shift}(\mathsf{par}', [\mathbf{G}])$ for the public key. The game parses $\sigma^*_k = (\mathbf{S}^*_{1,k}, \mathbf{S}^*_{2,k}, \pi^*_k)$ after $\mathcal{A}$ provides its forgery $(\tau^*, \{m^*_k, \sigma^*_k\}_{k\in[l+1]})$. Next, the game verifies that for each $k \in [l+1]$, the condition holds

$$\mathbf{S}^*_{1,k} = [\mathbf{W}] \circ \mathbf{s}^{\diamond} + [\bar{\mathbf{H}}]^*_k \circ \mathbf{sk} = \mathsf{Tran}(\mathsf{td}_w, \mathbf{S}^*_{2,k}) + [\bar{\mathbf{H}}]^*_k \circ \mathbf{sk}.$$

Using td_w and $\mathbf{sk}$, the check is efficiently conducted. The game aborts when the check fails, proceeding normally otherwise. Same as $\mathbf{G}_9^{\mathsf{bs}}$, we obtain that $\left|\Pr[\mathbf{G}_8^{\mathsf{tbs}} \Rightarrow 1] - \Pr[\mathbf{G}_9^{\mathsf{tbs}} \Rightarrow 1]\right| \leq \frac{\hat{Q}_{\mathsf{H_c}}}{p}$.

Game $\mathbf{G}_{10}^{\mathsf{tbs}}$ (**Simulate Σ_0 if $\tau = \tau^*$**). When $\tau = \tau^*$, we simulate Σ_0. Specifically, in this case we program the signing oracles $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_1}$, $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_2}$, $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_3}$ as in game $\mathbf{G}_5^{\mathsf{tbs}}$. Thus, same as $\mathbf{G}_5^{\mathsf{tbs}}$, we have $\Pr[\mathbf{G}_9^{\mathsf{tbs}} \Rightarrow 1] = \Pr[\mathbf{G}_{10}^{\mathsf{tbs}} \Rightarrow 1]$.
Game $\mathbf{G}_{11}^{\mathsf{tbs}}$ (**Sample DDH tuple if $\tau = \tau^*$**). The game chooses a real DDH tuple in $\mathsf{H}_{\mathsf{ddh}}$ for the q_τ^* query. The game randomly chooses $d_2 \leftarrow\$\ \mathbb{Z}_p$, sets $(D_2^{\tau^*}, D_3^{\tau^*}) := (d_2 \cdot G, d_2 \cdot D_1)$ instead of $(D_2^{\tau^*}, D_3^{\tau^*}) \leftarrow\$\ \mathbb{G}^2$, stores d_2 in $\mathsf{T}_{\mathsf{ddh}}[\tau^*]$. It is easy to devise a reduction $\mathcal{A}_{\mathsf{DDH}}^3$ against DDH with $T_{\mathcal{A}}(\lambda) \approx T_{\mathcal{A}_{\mathsf{DDH}}^3}(\lambda)$ and $\left|\Pr[\mathbf{G}_{10}^{\mathsf{tbs}} \Rightarrow 1] - \Pr[\mathbf{G}_{11}^{\mathsf{tbs}} \Rightarrow 1]\right| \leq \mathsf{Adv}_{\mathcal{A}_{\mathsf{DDH}}^3}^{\mathsf{GGen}}(\lambda)$.

Observe that now $\mathbf{D}^\tau \in \mathscr{L}_{\mathsf{ddh}}$ for every common message τ.
Game $\mathbf{G}_{12}^{\mathsf{tbs}}$ (**Utilize DDH witness for Σ_1 if $\tau = \tau^*$**). Within $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_1}$ for $\tau = \tau^*$, the game executes $(\mathbf{A}_{1,i}^\diamond, \mathsf{st}_{1,i}^\diamond) := \mathsf{Init}_1(\mathbb{x}_1, \mathbb{w}_1)$, where $\mathbb{w}_1 = \mathsf{T}_{\mathsf{ddh}}[\tau^*]$ and $\mathbb{x}_1 = (G, \mathbf{D}^{\tau^*})$. Recall that in Game $\mathbf{G}_{10}^{\mathsf{tbs}}$, the game samples $c_{1,j}^\diamond$ for $j \in \mathcal{SS}$ such that $c_0^\diamond + \sum_{j \in \mathcal{SS}} c_{1,j}^\diamond = c^\diamond$ after receiving $(c^\diamond, \{\mathsf{cm}_j\}_{j \in \mathcal{SS}})$ and sets $\mathsf{H}_{\mathsf{cm}}(\mathsf{sid}, i, c_{1,i}^\diamond) = \mathsf{cm}_i$ in $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_2}$ with $\tau = \tau^*$. Within $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_3}$ for $\tau = \tau^*$, the game executes $z_{1,i}^\diamond := \mathsf{Resp}_1(\mathsf{st}_{1,i}, c_{1,i}^\diamond)$. From the HVZK of Σ_1, we know that Σ_1 transcripts $(\mathbf{A}_{1,i}^\diamond, z_{1,i}^\diamond)$ for $\tau = \tau^*$ have the same distribution in Game $\mathbf{G}_{11}^{\mathsf{tbs}}$ and Game $\mathbf{G}_{12}^{\mathsf{tbs}}$. Thus, $\Pr[\mathbf{G}_{11}^{\mathsf{tbs}} \Rightarrow 1] = \Pr[\mathbf{G}_{12}^{\mathsf{tbs}} \Rightarrow 1]$.
Game $\mathbf{G}_{13}^{\mathsf{tbs}}$ (**Ensure the unforgeability of** DS). In this game, we abort if $\mathcal{A}$ breaks the unforgeability of the single-party signature scheme DS. More precisely, the game aborts if for any signing session sid and $j \in [n] \backslash \mathsf{Corrupted}$, $\mathcal{A}$ manages to provide a partial signature $\sigma_{k,\mathsf{sid}}$ in $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{ISign}_3}(\mathsf{sid}, i, \{\sigma_j, c_{1,j}^\diamond\}_{j \in \mathcal{SS}_{\mathsf{sid}}})$ for $\mathsf{msg}_{k,\mathsf{sid}} \neq \mathsf{msg}_{i,\mathsf{sid}}$ or $\mathsf{msg}_{k,\mathsf{sid}} = \perp$ such that $\mathsf{DS.Verify}(\hat{\mathsf{pk}}_k, \mathsf{msg}_{i,\mathsf{sid}}, \sigma_{k,\mathsf{sid}}) = 1$. We define this abort event as $\mathsf{ForgeSig}$. When $\mathsf{ForgeSig}$ happens, it means that $\mathcal{A}$ breaks the $\mathsf{EUF\text{-}CMA}$ security of DS for some public key $\hat{\mathsf{pk}}_k$. Thus, we are able to design an adversary $\mathcal{F}$ for the game $\mathsf{Game}_{\mathsf{DS}}^{\mathsf{EUF\text{-}CMA}}$ outlined below. First, $\mathcal{F}$ receives a public key $\hat{\mathsf{pk}}^*$ from $\mathsf{Game}_{\mathsf{DS}}^{\mathsf{EUF\text{-}CMA}}$ and runs $\mathcal{A}$ by simulating the game $\mathsf{Game}_{\mathsf{DS}}^{\mathsf{EUF\text{-}CMA}}$ honestly except $\mathcal{F}$ randomly selects $k^* \leftarrow [n] \backslash \mathsf{Corrupted}$ and assigns $\hat{\mathsf{pk}}_{k^*} = \hat{\mathsf{pk}}^*$. To produce a signature for public key $\hat{\mathsf{pk}}_{k^*}$, $\mathcal{F}$ makes a query to $\mathcal{O}_{\mathsf{DS}}^{\mathsf{Sign}}$ whenever necessary. For $\mathsf{msg}_{i,\mathsf{sid}}$, as defined in the construction of $\mathcal{A}$, is also maintained by $\mathcal{F}$. Next, if $\mathsf{ForgeSig}$ takes place, there exists some $k \in [n] \backslash \mathsf{Corrupted}$ where the adversary $\mathcal{A}$ produces a signature for m_k (i.e., $\mathsf{msg}_{k,\mathsf{sid}}$) and $\hat{\mathsf{pk}}^*$, without having obtained a signature for m_k previously. $\mathcal{F}$ is able to win the game $\mathsf{Game}_{\mathsf{DS}}^{\mathsf{EUF\text{-}CMA}}$ for $k = k^*$. Clearly, $\mathcal{F}$ issues at most Q_{S_2} query to $\mathcal{O}_{\mathsf{DS}}^{\mathsf{Sign}}$. Because the probability of $k = k^*$ is at least $1/n$, we derive that $\left|\Pr[\mathbf{G}_{12}^{\mathsf{tbs}} \Rightarrow 1] - \Pr[\mathbf{G}_{13}^{\mathsf{tbs}} \Rightarrow 1]\right| \leq n \cdot \mathsf{Adv}_{\mathcal{F},\mathsf{DS}}^{\mathsf{EUF\text{-}CMA}}(\lambda)$.
Reduce to TLF. We describe a reduction $\mathcal{C}$ against the security game $\mathsf{Game}_{\mathsf{TLF}}^{t\text{-}\mathsf{A\text{-}TRAN\text{-}RES}}$ of the TLF. The reduction $\mathcal{C}$ gets par', $[\mathbf{G}]$, $[\mathbf{H}_\#]$, $\mathbf{X}_0, ..., \mathbf{X}_t \in \mathcal{R}$. The reduction $\mathcal{C}$ simulates game $\mathbf{G}_{13}^{\mathsf{tbs}}$ for $\mathcal{A}$, with some changes shown in Fig. 13.

- $\mathcal{C}$ samples $D_1 \leftarrow\$\ \mathbb{G}$, computes $([\mathbf{W}], \mathsf{td}_w = \mathbf{r}_w) \leftarrow \mathsf{Shift}(\mathsf{par}', [\mathbf{G}])$, sets $\mathsf{PK} := (\mathbf{X}_0, [\mathbf{W}], D_1)$. For each $i \in [n]$, the reduction $\mathcal{C}$ computes $(\hat{\mathsf{pk}}_i, \hat{\mathsf{sk}}_i) \leftarrow \mathsf{DS.KeyGen}(\mathsf{par}_{\mathsf{sig}})$. For each $i \in [t]$, the reduction $\mathcal{C}$ computes $\mathsf{pk}_i := \mathbf{X}_i$. For every $i \in [n] \backslash \mathcal{SS}_0$, $\mathcal{C}$ sets $\mathsf{pk}_i := \sum_{j \in \mathcal{SS}_0} \ell_{j,\mathcal{SS}_0}(i) * \mathsf{pk}_j$, where $\mathcal{SS}_0 := \{0\} \cup [t]$.

Fig. 13. Reduction $\mathcal{C}$ simulates the game $\mathbf{G}_{13}^{\mathsf{tbs}}$ for $\mathcal{A}$.

- $\mathcal{C}$ provides all the random oracles in $\mathbf{G}_{13}^{\mathsf{tbs}}$, except for H_{m}. For H_{m}, if $\mathsf{ctr}_m = q_m^*$, $\mathcal{C}$ sets $[\bar{\mathbf{H}}]^\star = \mathsf{H}_{\mathsf{m}}(m_{q_m^*}) = [\mathbf{H}_\#]$ ($[\mathbf{H}_\#]$ is from game $\mathsf{Game}_{\mathsf{TLF}}^{t\text{-A-TRAN-RES}}$). Otherwise, $\mathcal{C}$ computes $([\bar{\mathbf{H}}], \mathsf{td} = \mathbf{r}) \leftarrow \mathsf{Shift}([\mathbf{G}])$, sets $\mathcal{Q}_{\mathsf{H}_{\mathsf{m}}}[m] = [\bar{\mathbf{H}}]$.
- $\mathcal{C}$ provides all the signing oracles in $\mathbf{G}_{13}^{\mathsf{tbs}}$ without relying on any secret key.
- When $\mathcal{A}$ makes a query $\mathcal{O}_{\mathsf{TBS}}^{\mathsf{Corr}}(i)$, the reduction $\mathcal{C}$ queries $\mathbf{x}_i := \mathcal{O}_{\mathsf{TLF}}^{\mathsf{Inv}}(\ell_{0,\mathcal{SS}_0}(i), ..., \ell_{t,\mathcal{SS}_0}(i))$, sets $\mathsf{sk}_i := \mathbf{x}_i$ and returns $\mathsf{SK}_i = (\mathsf{sk}_i, \hat{\mathsf{sk}}_i)$. Plainly, sk_i maintains the correct distribution. $\mathcal{C}$ makes queries to its oracle exactly t times because $\mathcal{A}$ corrupts exactly t parties (see $\mathbf{G}_0^{\mathsf{tbs}}$).
- When $\mathcal{A}$ returns the final forgery $\{(m_k^*, \sigma_k^*)\}_{k\in[l+1]}$, we find out the special forgery $(m^\star, \sigma^\star)$ with $[\bar{\mathbf{H}}]^\star = \mathsf{H}_{\mathsf{m}}(m^\star)$ and computes $\mathbf{X}_0' := \mathbf{S}_1^\star - \mathsf{InvTran}(\mathsf{td}_w, \mathbf{S}_2^\star)$.
- Let $\mathcal{SS}^* := \mathsf{Corrupted} \cup \{0\}$ with $|\mathcal{SS}^*| = t + 1$. For $i \in \mathsf{Corrupted}$, set $\mathbf{X}_i' := [\mathbf{H}_\#] \circ \mathbf{x}_i$. For every $i \in [n] \backslash \mathsf{Corrupted}$, set $\mathbf{X}_i' := \sum_{j\in\mathcal{SS}^*} \ell_{j,\mathcal{SS}^*}(i) * \mathbf{X}_j'$. Reduction $\mathcal{C}$ submits $\mathbf{X}_0', ..., \mathbf{X}_t'$ to the game $\mathsf{Game}_{\mathsf{TLF}}^{t\text{-A-TRAN-RES}}$.

We show that $\mathbf{X}_0', ..., \mathbf{X}_t'$ is a valid solution of the game $\mathsf{Game}_{\mathsf{TLF}}^{t\text{-A-TRAN-RES}}$. We must contend that there exists a $\theta_i \in \mathcal{D}$ such that $[\mathbf{G}] \circ \theta_i = \mathbf{X}_i$ and $[\mathbf{H}_\#] \circ \theta_i = \mathbf{X}_i'$ for every $i \in \{0\} \cup [t]$. Firstly, this holds by building for any $i \in [t]$ with $i \in \mathsf{Corrupted}$. In the case of $i = 0$, we are aware that there is a $\mathbf{x}_0$ such that $[\mathbf{G}] \circ \mathbf{x}_0 = \mathbf{X}_0$ and $\mathbf{S}_1^\star = [\mathbf{W}] \circ (\sum_{i\in\mathcal{SS}} \ell_{i,\mathcal{SS}} * \mathbf{s}_i) + [\bar{\mathbf{H}}]^\star \circ \mathbf{x}_0, \mathbf{S}_2^\star = [\mathbf{G}] \circ (\sum_{i\in\mathcal{SS}} \ell_{i,\mathcal{SS}} * \mathbf{s}_i)$. By translation completeness of TLF, we know that $\mathbf{X}_0' := \mathbf{S}_1^\star - \mathsf{InvTran}(\mathsf{td}_w, \mathbf{S}_2^\star)$. For every $i \in [t]$ with $i \notin \mathsf{Corrupted}$, we have

$$\mathbf{X}_i' = \sum_{j\in\mathcal{SS}^*} \ell_{j,\mathcal{SS}^*}(i) * \mathbf{X}_j' = \sum_{j\in\mathcal{SS}^*} \ell_{j,\mathcal{SS}^*}(i) * ([\mathbf{H}_\#] \circ \mathbf{x}_i) = [\mathbf{H}_\#] \circ (\sum_{j\in\mathcal{SS}^*} \ell_{j,\mathcal{SS}^*}(i) * \mathbf{x}_i),$$

$$[\mathbf{G}] \circ (\sum_{j\in\mathcal{SS}^*} \ell_{j,\mathcal{SS}^*}(i) * \mathbf{x}_i) = \sum_{j\in\mathcal{SS}^*} \ell_{j,\mathcal{SS}^*}(i) * ([\mathbf{G}] \circ \mathbf{x}_i) = \sum_{j\in\mathcal{SS}^*} \ell_{j,\mathcal{SS}^*}(i) * \mathsf{pk}_i = \mathbf{X}_i.$$

The difference between the view of $\mathcal{A}$ in game $\mathbf{G}_{13}^{\mathsf{tbs}}$ and the view of $\mathcal{A}$ in the simulation provided by $\mathcal{C}$ lies in the distribution of H_{m}. By the $\varepsilon_{\mathbf{t}}$-translatability of the TLF ($\varepsilon_{\mathbf{t}} = \mathsf{negl}(\lambda)$) and the union bound for all queries, we have $\Pr[\mathbf{G}_{13}^{\mathsf{tbs}} \Rightarrow 1] \leq Q_{\mathsf{H}_{\mathsf{m}}}\varepsilon_{\mathsf{t}} + \mathsf{Adv}_{\mathcal{C},\mathsf{TLF}}^{t\text{-A-TRAN-RES}}(\lambda)$.

6 Conclusion

Motivated by the existing margin in fully adaptive security of threshold blind signatures without AGM, we propose Rainblind. Due to the inherent unlinkability between blinded protocol messages and hash queries, achieving fully adaptive security without AGM is challenging. We first design two Σ protocols and construct a plain signature scheme $\mathsf{Sig}_{\mathsf{m}}$ for the proof of a disjunctive relation, which is specifically between a discrete logarithm equality statement and a DDH-tuple statement. We then design a mixed Σ_{ped} protocol with accordingly a non-interactive proof system $\mathsf{NIPS}_{\mathsf{ped}}$ to support mixed extraction for both group and scalar elements, and further construct a blind signature scheme BS_{m}. We propose the first adaptively secure threshold blind signature scheme Rainblind without AGM, and prove its security under TBS-OMUF-SB security model in the DDH assumption in the ROM.

References

1. D. Chaum, "Blind Signatures for Untraceable Payments," in *CRYPTO 1982*, D. Chaum, R. L. Rivest, and A. T. Sherman, Eds. Plenum Press, New York, 1982, pp. 199–203.
2. A. Fujioka, T. Okamoto, and K. Ohta, "A Practical Secret Voting Scheme for Large Scale Elections," in *AUSCRYPT 1992*, ser. LNCS, J. Seberry and Y. Zheng, Eds., vol. 718. Springer, 1992, pp. 244–251.
3. F. Baldimtsi and A. Lysyanskaya, "Anonymous credentials light," in *CCS 2013*, A. Sadeghi, V. D. Gligor, and M. Yung, Eds. ACM, 2013, pp. 1087–1098.
4. S. Brands, "Untraceable Off-line Cash in Wallets with Observers (Extended Abstract)," in *CRYPTO 1993*, ser. LNCS, D. R. Stinson, Ed., vol. 773. Springer, 1993, pp. 302–318.
5. I. Karantaidou, O. Renawi, F. Baldimtsi, N. Kamarinakis, J. Katz, and J. Loss, "Blind Multisignatures for Anonymous Tokens with Decentralized Issuance," in *CCS 2024*, B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie, Eds. ACM, 2024, pp. 1508–1522.
6. Apple Inc., "iCloud Private Relay Overview," https://www.apple.com/privacy/docs/iCloud_Private_Relay_Overview_Dec2021.PDF, 2021.
7. Google LLC, "VPN by Google One, explained." https://one.google.com/about/vpn/howitworks, 2020.
8. E. C. Crites, C. Komlo, M. Maller, S. Tessaro, and C. Zhu, "Snowblind: A Threshold Blind Signature in Pairing-Free Groups," in *CRYPTO 2023*, ser. LNCS, H. Handschuh and A. Lysyanskaya, Eds., vol. 14081. Springer, 2023, pp. 710–742.
9. A. Rial and A. M. Piotrowska, "Compact and Divisible E-Cash with Threshold Issuance," *Proc. Priv. Enhancing Technol.*, vol. 2023, no. 4, pp. 381–415, 2023.
10. A. Sonnino, M. Al-Bassam, S. Bano, S. Meiklejohn, and G. Danezis, "Coconut: Threshold Issuance Selective Disclosure Credentials with Applications to Distributed Ledgers," in *NDSS 2019*. The Internet Society, 2019.
11. J. Doerner, Y. Kondi, E. Lee, A. Shelat, and L. Tyner, "Threshold BBS+ Signatures for Distributed Anonymous Credential Issuance," in *SP 2023*. IEEE, 2023, pp. 773–789.
12. A. Lehmann, P. Nazarian, and C. Özbay, "Stronger Security for Threshold Blind Signatures," *IACR Cryptol. ePrint Arch.*, p. 353, 2025.
13. S. Faller, G. Niot, and M. Reichle, "Lattice-based Threshold Blind Signatures," *IACR Cryptol. ePrint Arch.*, p. 1566, 2025.
14. V. Shoup and R. Gennaro, "Securing Threshold Cryptosystems against Chosen Ciphertext Attack," in *EUROCRYPT 1998*, ser. LNCS, K. Nyberg, Ed., vol. 1403. Springer, 1998, pp. 1–16.
15. S. Jarecki and P. Nazarian, "Adaptively Secure Threshold Blind BLS Signatures and Threshold Oblivious PRF," *IACR Cryptol. ePrint Arch.*, p. 483, 2025.
16. G. Fuchsbauer, F. Regen, and H. Wee, "Round-Optimal Threshold Blind Signatures without Random Oracles," *IACR Cryptol. ePrint Arch.*, vol. 2026, p. 433, 2026.
17. G. Fuchsbauer, E. Kiltz, and J. Loss, "The Algebraic Group Model and its Applications," in *CRYPTO 2018*, ser. LNCS, H. Shacham and A. Boldyreva, Eds., vol. 10992. Springer, 2018, pp. 33–62.
18. U. M. Maurer, "Abstract Models of Computation in Cryptography," in *IMACC 2005*, ser. LNCS, N. P. Smart, Ed., vol. 3796. Springer, 2005, pp. 1–12.

19. C. Zhang, H. Zhou, and J. Katz, "An Analysis of the Algebraic Group Model," in *ASIACRYPT 2022*, ser. LNCS, S. Agrawal and D. Lin, Eds., vol. 13794. Springer, 2022, pp. 310–322.
20. M. Klooß, M. Reichle, and B. Wagner, "Practical Blind Signatures in Pairing-Free Groups," in *ASIACRYPT 2024*, ser. LNCS, K. Chung and Y. Sasaki, Eds., vol. 15484. Springer, 2024, pp. 363–395.
21. Y. Kondi and A. Shelat, "Improved Straight-Line Extraction in the Random Oracle Model with Applications to Signature Aggregation," in *ASIACRYPT 2022*, ser. LNCS, S. Agrawal and D. Lin, Eds., vol. 13792. Springer, 2022, pp. 279–309.
22. R. Bacho, J. Loss, S. Tessaro, B. Wagner, and C. Zhu, "Twinkle: Threshold Signatures from DDH with Full Adaptive Security," in *EUROCRYPT 2024*, ser. LNCS, M. Joye and G. Leander, Eds., vol. 14651. Springer, 2024, pp. 429–459.
23. D. R. Stinson and R. Strobl, "Provably Secure Distributed Schnorr Signatures and a (t, n) Threshold Scheme for Implicit Certificates," in *ACISP 2001*, ser. LNCS, V. Varadharajan and Y. Mu, Eds., vol. 2119. Springer, 2001, pp. 417–434.
24. C. Komlo and I. Goldberg, "FROST: Flexible Round-Optimized Schnorr Threshold Signatures," in *SAC 2020*, ser. LNCS, O. Dunkelman, M. J. J. Jr., and C. O'Flynn, Eds., vol. 12804. Springer, 2020, pp. 34–65.
25. E. C. Crites, C. Komlo, and M. Maller, "Fully Adaptive Schnorr Threshold Signatures," in *CRYPTO 2023*, ser. LNCS, H. Handschuh and A. Lysyanskaya, Eds., vol. 14081. Springer, 2023, pp. 678–709.
26. R. Bacho, S. Das, J. Loss, and L. Ren, "Glacius: Threshold Schnorr Signatures from DDH with Full Adaptive Security," in *EUROCRYPT 2025*, ser. LNCS, S. Fehr and P. Fouque, Eds., vol. 15602. Springer, 2025, pp. 304–334.
27. S. Katsumata, M. Reichle, and K. Takemure, "Adaptively Secure 5 Round Threshold Signatures from MLWE/MSIS and DL with Rewinding," in *CRYPTO 2024*, ser. LNCS, L. Reyzin and D. Stebila, Eds., vol. 14926. Springer, 2024, pp. 459–491.
28. Y. Chen, "Dazzle: Improved adaptive threshold signatures from DDH," *IACR Cryptol. ePrint Arch.*, p. 264, 2025.
29. R. Bacho and B. Wagner, "Tightly Secure Threshold Signatures over Pairing-Free Groups," *IACR Cryptol. ePrint Arch.*, p. 1557, 2024.
30. B. Libert, M. Joye, and M. Yung, "Born and raised distributively: Fully distributed non-interactive adaptively-secure threshold signatures with short shares," *Theor. Comput. Sci.*, vol. 645, pp. 1–24, 2016.
31. R. Bacho and J. Loss, "On the Adaptive Security of the Threshold BLS Signature Scheme," in *ACM CCS 2022*, H. Yin, A. Stavrou, C. Cremers, and E. Shi, Eds. ACM, 2022, pp. 193–207.
32. S. Das and L. Ren, "Adaptively Secure BLS Threshold Signatures from DDH and co-CDH," in *CRYPTO 2024*, ser. LNCS, L. Reyzin and D. Stebila, Eds., vol. 14926. Springer, 2024, pp. 251–284.
33. A. Mitrokotsa, S. Mukherjee, M. Sedaghat, D. Slamanig, and J. Tomy, "Threshold Structure-Preserving Signatures: Strong and Adaptive Security Under Standard Assumptions," in *PKC 2024*, ser. LNCS, Q. Tang and V. Teague, Eds., vol. 14601. Springer, 2024, pp. 163–195.
34. V. Shoup, "Practical Threshold Signatures," in *EUROCRYPT 2000*, ser. LNCS, B. Preneel, Ed., vol. 1807. Springer, 2000, pp. 207–220.
35. J. F. Almansa, I. Damgård, and J. B. Nielsen, "Simplified Threshold RSA with Adaptive and Proactive Security," in *EUROCRYPT 2006*, ser. LNCS, S. Vaudenay, Ed., vol. 4004. Springer, 2006, pp. 593–611.

36. R. Gennaro, S. Halevi, H. Krawczyk, and T. Rabin, "Threshold RSA for Dynamic and Ad-Hoc Groups," in *EUROCRYPT 2008*, ser. LNCS, N. P. Smart, Ed., vol. 4965. Springer, 2008, pp. 88–107.
37. S. Tessaro and C. Zhu, "Threshold and Multi-signature Schemes from Linear Hash Functions," in *EUROCRYPT 2023*, ser. LNCS, C. Hazay and M. Stam, Eds., vol. 14008. Springer, 2023, pp. 628–658.
38. M. Abe, "A Secure Three-Move Blind Signature Scheme for Polynomially Many Signatures," in *EUROCRYPT 2001*, ser. LNCS, B. Pfitzmann, Ed., vol. 2045. Springer, 2001, pp. 136–151.
39. S. Tessaro and C. Zhu, "Short Pairing-Free Blind Signatures with Exponential Security," in *EUROCRYPT 2022*, ser. LNCS, O. Dunkelman and S. Dziembowski, Eds., vol. 13276. Springer, 2022, pp. 782–811.
40. R. Chairattana-Apirom, S. Tessaro, and C. Zhu, "Pairing-Free Blind Signatures from CDH Assumptions," in *CRYPTO 2024*, ser. LNCS, L. Reyzin and D. Stebila, Eds., vol. 14920. Springer, 2024, pp. 174–209.
41. N. Brandt, D. Hofheinz, M. Klooß, and M. Reichle, "Tightly-Secure Blind Signatures in Pairing-Free Groups," *IACR Cryptol. ePrint Arch.*, p. 2075, 2024.
42. C. Schnorr, "Security of Blind Discrete Log Signatures against Interactive Attacks," in *ICICS 2001, Xian*, ser. LNCS, S. Qing, T. Okamoto, and J. Zhou, Eds., vol. 2229. Springer, 2001, pp. 1–12.
43. R. Lu, Z. Cao, and Y. Zhou, "Proxy blind multi-signature scheme without a secure channel," *Appl. Math. Comput.*, vol. 164, no. 1, pp. 179–187, 2005.
44. R. Tso, Z. Liu, and Y. Tseng, "Identity-Based Blind Multisignature From Lattices," *IEEE Access*, vol. 7, pp. 182 916–182 923, 2019.
45. R. Gay, D. Hofheinz, E. Kiltz, and H. Wee, "Tightly CCA-Secure Encryption Without Pairings," in *EUROCRYPT 2016*, ser. LNCS, M. Fischlin and J. Coron, Eds., vol. 9665. Springer, 2016, pp. 1–27.
46. D. Catalano, D. Fiore, R. Gennaro, and E. Giunta, "On the Impossibility of Algebraic Vector Commitments in Pairing-Free Groups," in *TCC 2022*, ser. LNCS, E. Kiltz and V. Vaikuntanathan, Eds., vol. 13748. Springer, 2022, pp. 274–299.
47. J. Pan and B. Wagner, "Chopsticks: Fork-free Two-Round Multi-signatures from Non-interactive Assumptions," in *EUROCRYPT 2023*, ser. LNCS, C. Hazay and M. Stam, Eds., vol. 14008. Springer, 2023, pp. 597–627.
48. E. Hauck, E. Kiltz, and J. Loss, "A Modular Treatment of Blind Signatures from Identification Schemes," in *EUROCRYPT 2019*, ser. LNCS, Y. Ishai and V. Rijmen, Eds., vol. 11478. Springer, 2019, pp. 345–375.
49. J. Katz, J. Loss, and M. Rosenberg, "Boosting the Security of Blind Signature Schemes," in *ASIACRYPT 2021*, ser. LNCS, M. Tibouchi and H. Wang, Eds., vol. 13093. Springer, 2021, pp. 468–492.
50. R. Chairattana-Apirom, L. Hanzlik, J. Loss, A. Lysyanskaya, and B. Wagner, "PI-Cut-Choo and Friends: Compact Blind Signatures via Parallel Instance Cut-and-Choose and More," in *CRYPTO 2022*, ser. LNCS, Y. Dodis and T. Shrimpton, Eds., vol. 13509. Springer, 2022, pp. 3–31.
51. D. Boneh and X. Boyen, "Efficient Selective-ID Secure Identity-Based Encryption Without Random Oracles," in *EUROCRYPT 2004*, ser. LNCS, C. Cachin and J. Camenisch, Eds., vol. 3027. Springer, 2004, pp. 223–238.
52. A. Escala, G. Herold, E. Kiltz, C. Ràfols, and J. L. Villar, "An Algebraic Framework for Diffie-Hellman Assumptions," in *CRYPTO 2013*, ser. LNCS, R. Canetti and J. A. Garay, Eds., vol. 8043. Springer, 2013, pp. 129–147.

A Deferred Definitions

A.1 NP-Relation

Definition 12. (**NP-Relation and Language**). *A binary relation* $\mathsf{R} \subseteq \{0,1\}^* \times \{0,1\}^*$ *is an NP-relation if* R *can be efficiently determined. The language generated by* R *is* $\mathscr{L}_{\mathsf{R}} = \{\mathbb{x} \in \{0,1\}^* \mid \exists \mathbb{w} \text{ such that } (\mathbb{x}, \mathbb{w}) \in \mathsf{R}\}$.

For R, the $\Sigma = (\mathsf{Init}, \mathsf{Resp}, \mathsf{Verify})$ protocol with challenge space $\mathcal{CH}$ is as follows:

- $(A, \mathsf{st}) \leftarrow \mathsf{Init}(\mathbb{x}, \mathbb{w})$ outputs a commitment A and a state st after accepting a statement $\mathbb{x} \in \mathscr{L}_{\mathsf{R}}$ and a witness $\mathbb{w}$.
- $z \leftarrow \mathsf{Resp}(\mathsf{st}, c)$ returns response z after accepting state st, challenge $c \in \mathcal{CH}$.
- $b \leftarrow \mathsf{Verify}(\mathbb{x}, A, c, z)$ returns a bit $b \in \{0,1\}$ after accepting a statement $\mathbb{x}$, a commitment A, a challenge c, and a response z.

Special HVZK. Σ satisfies special honest-verifier zero-knowledge (HVZK) if a PPT simulator Sim exists, and $\mathcal{D}_{\mathsf{real}} = \mathcal{D}_{\mathsf{sim}}$ for

$$\mathcal{D}_{\mathsf{real}} := \left\{(A, c, z) \middle| A \leftarrow \mathsf{Init}(\mathbb{x}, \mathbb{w}), z \leftarrow \mathsf{Resp}(\mathsf{st}, c)\right\},$$
$$\mathcal{D}_{\mathsf{sim}} := \left\{(A, c, z) \middle| (A, z) \leftarrow \mathsf{Sim}(\mathbb{x}, c)\right\}.$$

Special Soundness. Σ satisfies (2-)special soundness if a deterministic PT extractor Ext exists whereby provided with two valid transcripts $\{(A, c_b, z_b)\}_{b \in [2]}$ with $c_1 \neq c_2$, the extractor can generate a witness $\mathbb{w}$ satisfying $(\mathbb{x}, \mathbb{w}) \in \mathsf{R}$.

Definition 13. (**Non-Interactive Proof System**). *The non-interactive proof system* $\mathsf{NIPS} = (\mathsf{Prove}, \mathsf{Ver})$ *for* R *with hash function* H *is:*

$-\pi \leftarrow \mathsf{Prove}^{\mathsf{H}}(\mathbb{x}, \mathbb{w})$ *returns a proof* π *after accepting* $(\mathbb{x}, \mathbb{w}) \in \mathsf{R}$.
$-0/1 \leftarrow \mathsf{Ver}^{\mathsf{H}}(\mathbb{x}, \pi)$ *returns* $0/1$ *after accepting* π *and* $\mathbb{x}$.

Correctness. NIPS is correct with correctness error γ_{err} such that

$$\Pr[\pi \leftarrow \mathsf{Prove}^{\mathsf{H}}(\mathbb{x}, \mathbb{w}) : \mathsf{Ver}^{\mathsf{H}}(\mathbb{x}, \pi) = 1] \geq 1 - \gamma_{\mathsf{err}}(\lambda).$$

Zero-Knowledge (Simulatability). NIPS is zero-knowledge if there exists a PPT simulator Sim such that for every statement-witness pair $(\mathbb{x}, \mathbb{w}) \in \mathsf{R}$, the following two distributions are computationally indistinguishable:

$$\mathcal{D}_{\mathsf{real}} := \left\{\pi \;\middle|\; \pi \leftarrow \mathsf{Prove}^{\mathsf{H}}(\mathbb{x}, \mathbb{w})\right\}, \text{ and } \mathcal{D}_{\mathsf{sim}} := \left\{\pi \;\middle|\; \pi \leftarrow \mathsf{Sim}^{\mathsf{H}}(\mathbb{x})\right\}.$$

That is, for every PPT distinguisher $\mathcal{A}$,

$$\mathsf{Adv}^{\mathsf{ZK},\mathsf{NIPS}}_{\mathcal{A}}(\lambda) := \left|\Pr\left[\mathcal{A}^{\mathsf{H}}(\mathcal{D}_{\mathsf{real}}) = 1\right] - \Pr\left[\mathcal{A}^{\mathsf{H}}(\mathcal{D}_{\mathsf{sim}}) = 1\right]\right| \leq \mathsf{negl}(\lambda).$$

Relaxed Knowledge Soundness. NIPS is knowledge sound, given any algorithm $\mathcal{A}$ there exists PPT extractor algorithm Ext such that

$$\mathsf{Real}_{\mathcal{A}}(\lambda) = \Pr[b \leftarrow \mathcal{A}^{\mathsf{H},\mathcal{O}_{\mathsf{Ver}}}(1^\lambda) : b = 1], \mathsf{Ideal}_{\mathcal{A}}(\lambda) = \Pr[b \leftarrow \mathcal{A}^{\mathsf{H},\mathcal{O}_{\mathsf{Ext}}}(1^\lambda) : b = 1],$$
$$\mathsf{Adv}^{\mathsf{NIPS},\mathsf{R}}_{\mathcal{A}_{\mathsf{KS}}}(\lambda) := |\mathsf{Real}_{\mathcal{A}}(\lambda) - \mathsf{Ideal}_{\mathcal{A}}(\lambda)| \leq \mathsf{negl}(\lambda), \text{ where}$$

- $\mathcal{O}_{\mathsf{Ver}}(\mathbb{x}, \pi)$ returns $\mathsf{Ver}(\mathbb{x}, \pi)$.
- $\mathcal{O}_{\mathsf{Ext}}(\mathbb{x}, \pi)$ returns 0 if $\mathsf{Ver}(\mathbb{x}, \pi) = 1$ and $(\mathbb{x}, \mathbb{w}) \notin \mathsf{R}$ for $\mathbb{w} \leftarrow \mathsf{Ext}(\mathcal{Q}, \mathbb{x}, \pi)$, otherwise returns 1. The notation $\mathcal{Q}$ represents the H query set of $\mathcal{A}$.

B Security Properties of Σ-Protocols

We provide proofs of honest-verifier zero-knowledge (HVZK) and special soundness for our three Σ-protocols.

For Σ_0: HVZK is guaranteed by the indistinguishability of Sim_0's output from real interactions. For special soundness: Given two valid transcripts $(\mathbf{A}_0, c_0, \mathbf{z}_0)$ and $(\mathbf{A}_0, c_0', \mathbf{z}_0')$ with $c_0 \neq c_0'$, we have $\phi_0([\bar{\mathbf{H}}], \mathbf{z}_0) - c_0 * \mathbf{S} = \phi_0([\bar{\mathbf{H}}], \mathbf{z}_0') - c_0' * \mathbf{S}$, thus $\phi_0([\bar{\mathbf{H}}], \mathbf{z}_0 - \mathbf{z}_0') = (c_0 - c_0') * \mathbf{S} = (c_0 - c_0') * \phi_0([\bar{\mathbf{H}}], \mathbb{w}_0)$, $\mathbb{w}_0 = (\mathbf{z}_0 - \mathbf{z}_0')/(c_0 - c_0')$.

For Σ_1: HVZK is guaranteed by the indistinguishability of Sim_1's output from real interactions. For special soundness: Given two valid transcripts $(\mathbf{A}_1, c_1, z_1)$, $(\mathbf{A}_1, c_1', z_1')$ with $c_1 \neq c_1'$, we have $\phi_1(z_1) - c_1 * (D_2, D_3)^\top = \phi_1(z_1') - c_1' * (D_2, D_3)^\top$, thus $\phi_1(z_1 - z_1') = (c_1 - c_1') * (D_2, D_3)^\top = (c_1 - c_1') * \phi_1(\mathbb{w}_1)$, $\mathbb{w}_1 = (z_1 - z_1')/(c_1 - c_1')$.

For Σ_{ped}: HVZK is guaranteed by the indistinguishability of $\mathsf{Sim}_{\mathsf{ped}}$'s output from real interactions. For special soundness, given two valid transcripts $([\mathbf{A}_1], [\mathbf{A}_2], c_1, c_2, z_1, z_2)$ and $([\mathbf{A}_1], [\mathbf{A}_2], c_1', c_2', z_1', z_2')$ with $c_1 \neq c_1', c_2 \neq c_2', z_2 \neq z_2'$, we can extract $\varphi = (z_1 - z_1')/(c_1 - c_1')$, $\beta = (z_2 - z_2')/(c_2 - c_2')$ and compute $[\bar{\mathbf{H}}] = (1/\beta) * [\hat{\mathbf{H}}]$. It can be seen that $[\mathbf{H}] = \beta * [\bar{\mathbf{H}}] + \beta * \varphi * [\mathbf{G}]$, such a design considers $[\mathbf{H}]$ as a blinded value of $[\bar{\mathbf{H}}]$. Due to the fact that $[\bar{\mathbf{H}}]$ cannot be made public and $[\bar{\mathbf{H}}]$ needs to be included in $[\mathbf{H}]$, some of the following intuitive designs are flawed: $[\mathbf{H}] = \beta * [\bar{\mathbf{H}}]$, $[\mathbf{H}] = \beta * [\mathbf{G}]$, $[\mathbf{H}] = \varphi * \beta * [\bar{\mathbf{H}}]$, $[\mathbf{H}] = \beta * [\bar{\mathbf{H}}] + \varphi * [\mathbf{G}]$.

C Security of $\mathsf{Sig}_{\mathsf{m}}$

Proof. (Lemma 1) Suppose that $\mathcal{A}$ is a PPT adversary for the game specified in Lemma 1. From the CDH game, $\mathcal{B}$ obtains the instance $(G, a \cdot G, b \cdot G)$ and proceeds as outlined below. $\mathcal{B}$ samples $e_1, e_2, e_3, d \in \mathbb{Z}_p$ and sets

$$[\mathbf{G}] = \begin{bmatrix} G & e_1 \cdot G \\ e_2 \cdot G & e_3 \cdot G \end{bmatrix}, \mathbf{X} = \begin{bmatrix} a \cdot G + d \cdot e_1 \cdot G \\ e_2 \cdot a \cdot G + d \cdot e_3 \cdot G \end{bmatrix}, ([\mathbf{W}], \mathsf{td}_w = \mathbf{r}_w) \leftarrow \mathsf{Shift}([\mathbf{G}]),$$

where $\mathbf{sk}$ is implicitly defined as (a, d). $\mathcal{B}$ picks an integer $i^* \in [1, Q_{\mathsf{H}_{\mathsf{m}}}]$ at random. Then, $\mathcal{B}$ invokes $\mathcal{A}$ and gets $(m^\star, [\bar{\mathbf{H}}]^\star, \mathbf{S}_1^\star, \mathbf{S}_2^\star) \leftarrow \mathcal{A}^{\mathsf{H}_{\mathsf{m}}, \mathcal{O}_{\mathsf{Sig}_{\mathsf{m}}}}([\mathbf{G}], [\mathbf{W}], \mathbf{X})$, where H_{m} and $\mathcal{O}_{\mathsf{Sig}_{\mathsf{m}}}$ are simulated by $\mathcal{B}$ as follows:

$\mathsf{H}_{\mathsf{m}}(m)$: The i-th hash query is denoted as m_i. If $\mathcal{Q}_{\mathsf{H}_{\mathsf{m}}}[m_i] \neq \perp$, $\mathcal{B}$ returns the stored value. Otherwise, $\mathcal{B}$ samples $([\bar{\mathbf{H}}], \mathbf{r}) \leftarrow \mathsf{Shift}([\mathbf{G}])$. If $i \neq i^*$, $\mathcal{B}$ sets trapdoor $\mathsf{td}[m_i] = \mathbf{r}$ and returns $\mathcal{Q}_{\mathsf{H}_{\mathsf{m}}}[m_i] = [\bar{\mathbf{H}}]$. Otherwise,

$$\text{for } \mathbf{r}^\star = \begin{bmatrix} r_{1,1}^\star & r_{1,2}^\star \\ r_{2,1}^\star & r_{2,2}^\star \end{bmatrix}, [\bar{\mathbf{H}}] = [\mathbf{r}^\star \mathbf{G}] = \begin{bmatrix} r_{1,1}^\star \cdot G_{1,1} + r_{1,2}^\star \cdot G_{2,1} & r_{1,1}^\star \cdot G_{1,2} + r_{1,2}^\star \cdot G_{2,2} \\ r_{2,1}^\star \cdot G_{1,1} + r_{2,2}^\star \cdot G_{2,1} & r_{2,1}^\star \cdot G_{1,2} + r_{2,2}^\star \cdot G_{2,2} \end{bmatrix},$$

$\mathcal{B}$ sets $[\bar{\mathbf{H}}]^\star = \begin{bmatrix} r_{1,1}^\star \cdot G_{1,1} + r_{1,2}^\star \cdot G_{2,1} + b \cdot G & r_{1,1}^\star \cdot G_{1,2} + r_{1,2}^\star \cdot G_{2,2} \\ r_{2,1}^\star \cdot G_{1,1} + r_{2,2}^\star \cdot G_{2,1} & r_{2,1}^\star \cdot G_{1,2} + r_{2,2}^\star \cdot G_{2,2} \end{bmatrix}$, returns $\mathcal{Q}_{\mathsf{H}_{\mathsf{m}}}[m_{i^*}] = [\bar{\mathbf{H}}]^\star$. ($\bar{\mathbf{H}}_{1,1}^\star = r_{1,1}^\star \cdot G_{1,1} + r_{1,2}^\star \cdot G_{2,1} + b \cdot G = r_{1,1}^\star \cdot G + r_{1,2}^\star \cdot e_1 \cdot G + b \cdot G$)

$\mathcal{O}_{\mathsf{Sig}_{\mathsf{m}}}(m)$: If $\mathcal{Q}_{\mathsf{H}_{\mathsf{m}}}[m] = \perp$, $\mathcal{B}$ first queries $\mathsf{H}_{\mathsf{m}}(m)$ to obtain $[\bar{\mathbf{H}}]$. If m corresponds to the i^*-th entry in the hash list, $\mathcal{B}$ aborts. Otherwise, $\mathcal{B}$ samples $\mathbf{s} \in \mathcal{D}$,

computes $\mathbf{S}_2 := [\mathbf{G}] \circ \mathbf{s}$, $\mathbf{S}_1 := [\mathbf{W}] \circ \mathbf{s} + [\bar{\mathbf{H}}] \circ \mathbf{sk} = [\mathbf{W}] \circ \mathbf{s} + \mathsf{Tran}(\mathsf{td}[m], \mathbf{X})$ and returns $(\mathbf{S}_1, \mathbf{S}_2)$. $\mathbf{S}_1$ is valid by the translatability of TLF.

Finally, the adversary returns $(m^\star, [\bar{\mathbf{H}}]^\star, \mathbf{S}_1^\star, \mathbf{S}_2^\star)$ where $m^\star$ was not queried via $\mathcal{O}_{\mathsf{Sig_m}}$. If $m^\star$ is not the i^*-th queried message in the hash list, $\mathcal{B}$ aborts. Otherwise, $\mathcal{B}$ outputs the CDH solution $(ab) \cdot G$. If $\mathbb{x}_0^\star \in \mathscr{L}_{\mathsf{bb}}$, there exists a $\mathbb{w}_0^\star = (\mathbf{s}^\star, \mathbf{sk})$ satisfying $(\mathbb{x}_0^\star, \mathbb{w}_0^\star) \in \mathsf{R}_{\mathsf{bb}}$, so

$$\mathbf{S}_1^\star = [\mathbf{W}] \circ \mathbf{s}^\star + [\bar{\mathbf{H}}]^\star \circ \mathbf{sk}, \mathbf{S}_2^\star = [\mathbf{G}] \circ \mathbf{s}^\star, [\bar{\mathbf{H}}]^\star \circ \mathbf{sk} = \mathbf{S}_1^\star - [\mathbf{W}] \circ \mathbf{s}^\star = \mathbf{S}_1^\star - \mathsf{Tran}(\mathsf{td}_w, \mathbf{S}_2^\star),$$

$$[\bar{\mathbf{H}}]^\star \circ \mathbf{sk} = \begin{bmatrix} r_{1,1}^\star \cdot G_{1,1} + r_{1,2}^\star \cdot G_{2,1} + b \cdot G & r_{1,1}^\star \cdot G_{1,2} + r_{1,2}^\star \cdot G_{2,2} \\ r_{2,1}^\star \cdot G_{1,1} + r_{2,2}^\star \cdot G_{2,1} & r_{2,1}^\star \cdot G_{1,2} + r_{2,2}^\star \cdot G_{2,2} \end{bmatrix} \circ \begin{bmatrix} a \\ d \end{bmatrix}.$$

The $(1,1)$ entry is $a \cdot r_{1,1}^\star \cdot G_{1,1} + a \cdot r_{1,2}^\star \cdot G_{2,1} + a \cdot b \cdot G + d \cdot r_{1,1}^\star \cdot G_{1,2} + d \cdot r_{1,2}^\star \cdot G_{2,2} = a \cdot r_{1,1}^\star \cdot G + a \cdot r_{1,2}^\star \cdot e_1 \cdot G + a \cdot b \cdot G + d \cdot r_{1,1}^\star \cdot e_1 \cdot G + d \cdot r_{1,2}^\star \cdot e_3 \cdot G$.

Thus, denote $\mathbf{S}' = \mathbf{S}_1^\star - \mathsf{Tran}(\mathsf{td}_w, \mathbf{S}_2^\star)$, $\mathcal{B}$ outputs $\mathbf{S}'_{1,1} - r_{1,1}^\star \cdot a \cdot G - r_{1,2}^\star \cdot e_1 \cdot a \cdot G - d \cdot r_{1,1}^\star \cdot e_1 \cdot G - d \cdot r_{1,2}^\star \cdot e_3 \cdot G = a \cdot b \cdot G$. Notice that i^* is randomly selected and not disclosed to the adversary $\mathcal{A}$. Thus the probability of solving the CDH instance is $1/Q_{\mathsf{H_m}}$.

D Security of $\mathsf{BS_m}$

D.1 Proof of the Theorem 1

Proof. (Theorem 1) We stress that the blindness achieved by our construction is computational, not statistical. Indeed, the first user-to-signer message contains the tuple $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$, and an unbounded adversary may recover the blinding relation from this tuple. We prove Theorem 1 via a sequence of games.

Game $\mathbf{G}_0$: $\mathbf{G}_0$ is exactly the real blindness experiment of Definition 7. Hence, $\mathsf{Adv}^{\mathsf{BS\text{-}Blind}}_{\mathcal{A},\mathsf{BS_m}}(\lambda) = \left|\Pr[\mathbf{G}_0 \Rightarrow 1] - \frac{1}{2}\right|$

Game $\mathbf{G}_1$: In this game, we modify only the way the group elements $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$ are generated in the user's first-round message. Instead of computing them from $[\bar{\mathbf{H}}] = \mathsf{H}_m(m_b)$, the challenger samples $[\hat{\mathbf{H}}] \leftarrow\$\ \mathcal{T}, [\mathbf{E}] \leftarrow\$\ \mathcal{T}, \beta \leftarrow\$\ \mathcal{S}$, and sets $[\mathbf{H}] := [\hat{\mathbf{H}}] + \beta * [\mathbf{E}]$. The proof π_{ped} is still generated honestly by $\pi_{\mathsf{ped}} \leftarrow \mathsf{NIPS_{ped}.Prove}^{\mathsf{H_{ped}}}(\mathbb{x}_{\mathsf{ped}}, \mathbb{w}_{\mathsf{ped}})$, for the corresponding valid statement $\mathbb{x}_{\mathsf{ped}} = ([\mathbf{G}], [\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$. Therefore, the only difference between Game $\mathbf{G}_0$ and Game $\mathbf{G}_1$ is the distribution of the tuple $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$. By the DDH assumption in $\mathbb{G}$, these two distributions are computationally indistinguishable. The reduction is carried out in essentially the same way as in the proof of Lemma 1. Intuitively, since the adversary is computationally bounded (cannot compute discrete logarithm), the tuple $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$ does not reveal any useful information about $[\bar{\mathbf{H}}]$ to the adversary. Hence, there exists a PPT adversary $\mathcal{A}_{\mathsf{DDH}}$ such that $\left|\Pr[\mathbf{G}_0 \Rightarrow 1] - \Pr[\mathbf{G}_1 \Rightarrow 1]\right| \leq \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{A}_{\mathsf{DDH}}}(\lambda)$.

Game $\mathbf{G}_2$: In this game, we keep the generation of $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$ exactly as in Game $\mathbf{G}_1$, but replace the real proof $\mathsf{NIPS_{ped}.Prove}$ with the simulator of $\mathsf{NIPS_{ped}}$. Namely, the challenger computes $\pi_{\mathsf{ped}} \leftarrow \mathsf{Sim_{ped}}(\mathbb{x}_{\mathsf{ped}})$, and sends $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}], \pi_{\mathsf{ped}})$

to the adversary. By the zero-knowledge (simulatability) property of $\mathsf{NIPS}_{\mathsf{ped}}$, there exists a PPT adversary $\mathcal{A}_{\mathsf{ZK}}$ such that $\left|\Pr[\mathbf{G}_1 \Rightarrow 1] - \Pr[\mathbf{G}_2 \Rightarrow 1]\right| \leq \mathsf{Adv}^{\mathsf{NIPS}}_{\mathcal{A}_{\mathsf{ZK}}}(\lambda)$. Finally, in Game $\mathbf{G}_2$, the tuple $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$ is generated independently of the message m_b, and the simulated proof π_{ped} depends only on the statement $\mathbb{x}_{\mathsf{ped}} = ([\mathbf{G}], [\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$, which is itself independent of m_b. Therefore, the entire first-round protocol message sent by the user is independent of m_b, and thus $\Pr[\mathbf{G}_2 \Rightarrow 1] = \frac{1}{2}$.

Hence, the user's first-round protocol message is computationally independent of the message m, and BS_{m} satisfies blindness.

D.2 Proof of Theorem 2

Proof. (Theorem 2) We use a sequence of games to gradually approach the target.

Game $\mathbf{G}_0^{\mathsf{bs}}$: A full description of Game $\mathbf{G}_0^{\mathsf{bs}}$ is provided in Fig. 14. The game first runs $\mathsf{par} \leftarrow \mathsf{BS}_{\mathsf{m}}.\mathsf{SysGen}(1^\lambda), (\mathsf{SK}, \mathsf{PK}) \leftarrow \mathsf{BS}_{\mathsf{m}}.\mathsf{KeyGen}(\mathsf{par})$, and then conducts interactions with an adversary $\mathcal{A}(\mathsf{par}, \mathsf{PK})$ capable of accessing to $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}, \mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$ and random oracles H_{m}, H_{c}, $\mathsf{H}_{\mathsf{ddh}}$, $\mathsf{H}_{\mathsf{ped}}$ (implemented using lazy sampling). Here, $\mathcal{A}$ makes Q_{S_1} queries to $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$, $l = Q_{S_2}$ queries to $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$, and Q_{H_*} queries to each $\mathsf{H}_* \in \{\mathsf{H}_{\mathsf{m}}, \mathsf{H}_{\mathsf{c}}, \mathsf{H}_{\mathsf{ddh}}, \mathsf{H}_{\mathsf{ped}}\}$. Finally, adversary $\mathcal{A}$ outputs $(\tau^*, \{(m_k^*, \sigma_k^*)\}_{k\in[l+1]})$. If for all $k_1 \neq k_2$, $m_{k_1}^* \neq m_{k_2}^*$, for all $k \in [l+1]$, $\mathsf{BS}_{\mathsf{m}}.\mathsf{Verify}(\mathsf{PK}, m_k^*, \tau^*, \sigma_k^*, \mathsf{par}) = 1$ and $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$ was accessed at most l times with τ^*, then $\mathcal{A}$ succeeds.

Game $\mathbf{G}_0^{\mathsf{bs}}$

$\forall \mathsf{H}_* \in \{\mathsf{H}_{\mathsf{ddh}}, \mathsf{H}_{\mathsf{c}}, \mathsf{H}_{\mathsf{m}}, \mathsf{H}_{\mathsf{ped}}\}, Q_{\mathsf{H}_*}[\cdot] := \perp$
$l := 0, \quad Q_{\mathsf{cm}}[\cdot] := 0, \quad \mathcal{S}_1, \mathcal{S}_2 := \emptyset$
$(\mathbb{G}, p, G) \leftarrow\$ \mathsf{GGen}(1^\lambda)$
$\mathsf{par}' \leftarrow\$ \mathsf{TLF.Gen}(1^\lambda)$
$[\mathbf{G}], [\mathbf{W}] \leftarrow\$ \mathcal{T}, \quad D_1 \leftarrow\$ \mathbb{G}$
$\mathsf{SK} = \mathbf{sk} \leftarrow\$ \mathcal{D}$
$\mathbf{X} := [\mathbf{G}] \circ \mathbf{sk}$
$\mathsf{PK} = (\mathbf{X}, [\mathbf{W}], D_1)$
$\mathsf{par} := (\mathbb{G}, p, G, \mathsf{par}', [\mathbf{G}])$
$\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISIGN}} := (\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}, \mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2})$
$(\tau^*, (m_k^*, \sigma_k^*)_{k\in[l+1]}) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISIGN}}}(\mathsf{par}, \mathsf{PK})$
if $Q_{\mathsf{cm}}[\tau^*] \geq l+1$: **abort**
for $\forall k, i \in [l+1], k \neq i$:
  if $m_k^* = m_i^*$: **return** 0
  if $\mathsf{BS}_{\mathsf{m}}.\mathsf{Verify}(\mathsf{par}, \sigma_k^*, \mathsf{PK}, m_k^*, \tau^*) \neq 1$:
    return 0
return 1

$\mathsf{H}_{\mathsf{ddh}}(\tau)$
if $Q_{\mathsf{H}_{\mathsf{ddh}}}[\tau] = \perp$:
  $(D_2^\tau, D_3^\tau) \leftarrow\$ \mathbb{G}^2, Q_{\mathsf{H}_{\mathsf{ddh}}}[\tau] := (D_2^\tau, D_3^\tau)$
return $Q_{\mathsf{H}_{\mathsf{ddh}}}[\tau]$

$\mathsf{H}_{\mathsf{c}}(x)$
if $Q_{\mathsf{H}_{\mathsf{c}}}[x] = \perp$:
  $c \leftarrow\$ \mathbb{Z}_p, \quad Q_{\mathsf{H}_{\mathsf{c}}}[x] := c$
return $Q_{\mathsf{H}_{\mathsf{c}}}[x]$

$\mathsf{H}_{\mathsf{m}}(m)$
if $Q_{\mathsf{H}_{\mathsf{m}}}[m] = \perp$:
  $[\hat{\mathbf{H}}] \leftarrow\$ \mathcal{T}, \quad Q_{\mathsf{H}_{\mathsf{m}}}[m] := [\hat{\mathbf{H}}]$
return $Q_{\mathsf{H}_{\mathsf{m}}}[m]$

$\mathsf{H}_{\mathsf{ped}}(x)$
if $Q_{\mathsf{H}_{\mathsf{ped}}}[x] = \perp$:
  $y \leftarrow\$ \mathcal{Y}_{\mathsf{ped}}, \quad Q_{\mathsf{H}_{\mathsf{ped}}}[x] := y$
return $Q_{\mathsf{H}_{\mathsf{ped}}}[x]$

$\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}(\mathsf{sid}, \tau, [\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}], \pi_{\mathsf{ped}})$
if $\mathsf{sid} \in \mathcal{S}_1$: **return** $\perp$ **else:** $\mathcal{S}_1 := \mathcal{S}_1 \cup \{\mathsf{sid}\}$
$\mathbb{x}_{\mathsf{ped}} := ([\mathbf{G}], [\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$
if $\mathsf{NIPS}_{\mathsf{ped}}.\mathsf{Ver}^{\mathsf{H}_{\mathsf{ped}}}(\mathbb{x}_{\mathsf{ped}}, \pi_{\mathsf{ped}}) = 0$: **return** $\perp$
$\mathbf{s}^\diamond \leftarrow\$ \mathcal{D}, \quad \mathbb{w}_0 := (\mathbf{s}^\diamond, \mathbf{sk})^\top$
$\mathbf{T}^\diamond := \phi_0([\mathbf{H}], \mathbb{w}_0) = \phi_0([\mathbf{H}], (\mathbf{s}^\diamond, \mathbf{sk})^\top)$
$\mathbb{x}_0^\diamond = ([\mathbf{G}], [\mathbf{W}], [\mathbf{H}], \mathbf{T}^\diamond)$
$(\mathbf{A}_0^\diamond, \mathsf{st}_0) := \mathsf{Init}_0(\mathbb{x}_0^\diamond, \mathbb{w}_0)$
$(D_2^\tau, D_3^\tau) := \mathsf{H}_{\mathsf{ddh}}(\tau), \quad \mathbf{D}^\tau = (D_1, D_2^\tau, D_3^\tau)$
$\mathbb{x}_1 = (G, \mathbf{D}^\tau), \quad c_1^\diamond \leftarrow\$ \mathcal{S}$
$(\mathbf{A}_1^\diamond, z_1^\diamond) := \mathsf{Sim}_1(\mathbb{x}_1, c_1^\diamond)$
return $(\mathbf{T}^\diamond, \mathbf{A}_0^\diamond, \mathbf{A}_1^\diamond)$

$\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}(\mathsf{sid}, c^\diamond)$
if $\mathsf{sid} \in \mathcal{S}_2$ **or** $\mathsf{sid} \notin \mathcal{S}_1$: **return** $\perp$
$\mathcal{S}_2 := \mathcal{S}_2 \cup \{\mathsf{sid}\}$
$c_0^\diamond := c^\diamond - c_1^\diamond, \quad z_0^\diamond := \mathsf{Resp}_0(\mathsf{st}_0, c_0^\diamond)$
$l := l+1, \quad Q_{\mathsf{cm}}[\tau] := Q_{\mathsf{cm}}[\tau] + 1$
return $(z_0^\diamond, z_1^\diamond, c_0^\diamond)$

Fig. 14. Game $\mathbf{G}_0^{\mathsf{bs}}$. It is the same as $\mathsf{Game}_{\mathsf{BS}}^{\mathsf{OMUF}}$.

Each signing session is uniquely identified by sid, which is provided as input to both $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$ and $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$. The oracles $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$ and $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$ execute as follows:

$\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}(\mathsf{sid}, [\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}], \pi_{\mathsf{ped}})$: If $\mathsf{NIPS}_{\mathsf{ped}}.\mathsf{Ver}^{\mathsf{H}_{\mathsf{ped}}}(\mathbb{x}_{\mathsf{ped}}, \mathbb{w}_{\mathsf{ped}}) = 0$ for $\mathbb{x}_{\mathsf{ped}} = ([\mathbf{G}], [\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$, abort. Randomly choose $\mathbf{s}^{\diamond} \in \mathcal{D}$ and set $\mathbb{w}_0 = (\mathbf{s}^{\diamond}, \mathbf{sk})$. Then, compute $\mathbf{T}^{\diamond} := \phi_0([\mathbf{H}], \mathbb{w}_0)$, $\mathbf{D}^{\tau} := (D_1, \mathsf{H}_{\mathsf{ddh}}(\tau))$, and set $\mathbb{x}_0^{\diamond} := ([\mathbf{G}], [\mathbf{W}], [\mathbf{H}], \mathbf{T}^{\diamond})$, $\mathbb{x}_1 := (G, \mathbf{D}^{\tau})$. For Σ_1 protocol, randomly choose $c_1^{\diamond} \in \mathcal{S}$ and compute $(\mathbf{A}_1^{\diamond}, z_1^{\diamond}) := \mathsf{Sim}_1(\mathbb{x}_1, c_1^{\diamond})$. For Σ_0 protocol, compute $(\mathbf{A}_0^{\diamond}, \mathsf{st}_0) := \mathsf{Init}_0(\mathbb{x}_0^{\diamond}, \mathbb{w}_0)$. For session sid, store $(z_1^{\diamond}, c_1^{\diamond}, \mathsf{st}_0)$ as part of the state and output $(\mathbf{T}^{\diamond}, \mathbf{A}_0^{\diamond}, \mathbf{A}_1^{\diamond})$.

$\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}(\mathsf{sid}, c^{\diamond})$: Obtain $(z_1^{\diamond}, c_1^{\diamond}, \mathsf{st}_0)$ using the state of sid, or abort if it can't. Then calculate $c_0^{\diamond} := c^{\diamond} - c_1^{\diamond}$ and $\mathbf{z}_0^{\diamond} := \mathsf{Resp}_0(\mathsf{st}_0, c_0^{\diamond})$ for Σ_0, output $(\mathbf{z}_0^{\diamond}, z_1^{\diamond}, c_0^{\diamond})$.

Here are some changes that do not affect the probability. For all values $(\cdot)^{\mathsf{sid}}$ in the security proof, we omit their superscript sid. Unless explicitly stated otherwise, all values $(\cdot)$ are implicitly assumed to belong to some session sid. Suppose that the adversary is honest but curious. Assume that while verifying the forgeries, the adversary $\mathcal{A}$ also performs queries to random oracles, i.e., $\hat{Q}_{\mathsf{H_m}} = Q_{\mathsf{H_m}} + l + 1$, $\hat{Q}_{\mathsf{H_c}} = Q_{\mathsf{H_c}} + l + 1$, $\hat{Q}_{\mathsf{H_{ddh}}} = Q_{\mathsf{H_{ddh}}} + 1$ for $\mathsf{H_m}$, $\mathsf{H_c}$ and $\mathsf{H_{ddh}}$, respectively. We have $\mathsf{Adv}^{\mathsf{BS\text{-}OMUF}}_{\mathcal{A},\mathsf{BS_m}}(\lambda) = \Pr[\mathbf{G}_0^{\mathsf{bs}} \Rightarrow 1]$.

Game $\mathbf{G}_1^{\mathsf{bs}}$ (Change the response of $\mathsf{H_m}$). We modify the response of $\mathsf{H_m}$ from $[\bar{\mathbf{H}}] \leftarrow\$\ \mathcal{T}$ to $([\bar{\mathbf{H}}], \mathsf{td} = \mathbf{r}) \leftarrow \mathsf{Shift}([\mathbf{G}])$. Moreover, if there exist collisions in $\mathsf{H_m}$, the game aborts. This means if there are queries $x \neq x'$ such that $\mathsf{H_m}(x) = \mathsf{H_m}(x')$, the game aborts. By the typical birthday-bound argument, we have $\left|\Pr[\mathbf{G}_0^{\mathsf{bs}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathsf{bs}} \Rightarrow 1]\right| \leq \frac{\hat{Q}_{\mathsf{H_m}}^2}{|\mathcal{T}|}$.

Game $\mathbf{G}_2^{\mathsf{bs}}$ (Extract $([\bar{\mathbf{H}}], \beta, \varphi)$ from $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$). In this game, we modify $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$: we utilize the extractor $\mathsf{Ext}_{\mathsf{ped}}$ of $\mathsf{NIPS}_{\mathsf{ped}}$ to extract $([\bar{\mathbf{H}}], \beta, \varphi) \in \mathcal{T} \times \mathcal{S}^2$ from $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}], \pi_{\mathsf{ped}})$ sent by the adversary. If π_{ped} passes validation but extraction fails, the game aborts. Specifically, if $\mathsf{NIPS}_{\mathsf{ped}}.\mathsf{Ver}^{\mathsf{H}_{\mathsf{ped}}}(\mathbb{x}_{\mathsf{ped}}, \pi_{\mathsf{ped}}) = 1$, then game sets $\mathbb{w}_{\mathsf{ped}} \leftarrow \mathsf{Ext}_{\mathsf{ped}}(\mathcal{Q}_{\mathsf{H}_{\mathsf{ped}}}, \mathbb{x}_{\mathsf{ped}}, \pi_{\mathsf{ped}})$ and parses $([\bar{\mathbf{H}}], \beta, \varphi) := \mathbb{w}_{\mathsf{ped}}$. If parsing $\mathbb{w}_{\mathsf{ped}}$ fails or $(\mathbb{x}_{\mathsf{ped}}, \mathbb{w}_{\mathsf{ped}}) \notin \mathsf{R}_{\mathsf{ped}}$, the game aborts.

For Game $\mathbf{G}_2^{\mathsf{bs}}$, we ensure that the extraction is successful and the extracted witness $\mathbb{w}_{\mathsf{ped}}$ is an opening for $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$, i.e., $(\mathbb{x}_{\mathsf{ped}}, \mathbb{w}_{\mathsf{ped}}) \in \mathsf{R}_{\mathsf{ped}}$. For relation $\tilde{\mathsf{R}}_{\mathsf{ped}}$, the extracted witness $\mathbb{w}_{\mathsf{ped}}$ may be the preimage $\mathbf{sk}$ of $\mathbf{X}$ (Equation (1)), since the soundness relation is *relaxed*. The probability of this is negligible since $\mathcal{A}$ provides π_{ped} and $\mathbf{sk}$ is hidden from $\mathcal{A}$. However, since the simulation of the signing oracles still requires $\mathbf{sk}$, we cannot immediately rely on the DL assumption to construct a reduction. We introduce intermediate games between Game $\mathbf{G}_1^{\mathsf{bs}}$ and Game $\mathbf{G}_2^{\mathsf{bs}}$, formalized in Lemma 2. The proof is given in Appendix D.3.

Lemma 2. *We can construct reductions $\mathcal{A}_{\mathsf{KS}}$, $\mathcal{A}^i_{\mathsf{DDH}}$ (for $i \in [2]$), and $\mathcal{A}_{\mathsf{DL}}$ with running times comparable to $\mathcal{A}$'s, ensuring that*

$$\left|\Pr[\mathbf{G}_1^{\mathsf{bs}} \Rightarrow 1] - \Pr[\mathbf{G}_2^{\mathsf{bs}} \Rightarrow 1]\right| \leq \mathsf{Adv}^{\mathsf{NIPS}_{\mathsf{ped}}, \tilde{\mathsf{R}}_{\mathsf{ped}}}_{\mathcal{A}_{\mathsf{KS}}}(\lambda) + \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{A}_{\mathsf{DL}}}(\lambda) + \frac{4}{p-1} + \sum_{i \in [2]} \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{A}^i_{\mathsf{DDH}}}(\lambda).$$

Game $\mathbf{G}_3^{\mathsf{bs}}$ (**Guess** τ^*). We guess the first query to $\mathsf{H}_{\mathsf{ddh}}$ with τ^* as input. The game randomly chooses $q_\tau^* \leftarrow\$\ [\hat{Q}_{\mathsf{H}_{\mathsf{ddh}}}]$ at the beginning. The game also determines whether τ^* was first queried to $\mathsf{H}_{\mathsf{ddh}}$ on the q_τ^*-th query to $\mathsf{H}_{\mathsf{ddh}}$ after the adversary produces its forgeries with the common message τ^* finally. If not, the game aborts. Since $\mathsf{H}_{\mathsf{ddh}}$ query is also performed when verifying the adversary's forgery, i.e., $(D_2^{\tau^*}, D_3^{\tau^*}) := \mathsf{H}_{\mathsf{ddh}}(\tau^*)$, such a query exists. Notice that q_τ^* is concealed from the adversary $\mathcal{A}$, thus the probability that the adversary correctly guesses q_τ^* is at most $1/\hat{Q}_{\mathsf{H}_{\mathsf{ddh}}}$. We get $\Pr[\mathbf{G}_2^{\mathsf{bs}} \Rightarrow 1] \leq \hat{Q}_{\mathsf{H}_{\mathsf{ddh}}} \cdot \Pr[\mathbf{G}_3^{\mathsf{bs}} \Rightarrow 1]$.
Game $\mathbf{G}_4^{\mathsf{bs}}$ (**Guess unsigned $[\bar{\mathbf{H}}]^*$ in forgery**). We guess the first query q_m^* to H_{m} that satisfies both of the below conditions:

- The adversary $\mathcal{A}$'s forgeries include the input $m_{q_m^*}$ of the q_m^*-th H_{m} query.
- If $[\bar{\mathbf{H}}] = \mathsf{H}_{\mathsf{m}}(m_{q_m^*})$ can be extracted from commitment $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$ (in Game $\mathbf{G}_2^{\mathsf{bs}}$), then no session about common message τ^* is finished.

If the guess is wrong, then the game aborts. Adversary $\mathcal{A}$'s forgeries $(m_k^*, \sigma_k^*)_{k\in[l+1]}$ with common message τ^* consist of $l+1$ different messages if $\mathcal{A}$ succeeds. The hashed messages $([\bar{\mathbf{H}}]_k^*)_{k\in[l+1]}$ for $[\bar{\mathbf{H}}]_k^* = \mathsf{H}_{\mathsf{m}}(m_k^*)$ also differ because Game $\mathbf{G}_1^{\mathsf{bs}}$ has excluded H_{m} collisions. Moreover, concerning common message τ^*, there are at most l completed sessions, each of which can extract a hashed message $[\bar{\mathbf{H}}] \in \mathcal{T}$ from the commitment $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$ via $\mathsf{Ext}_{\mathsf{ped}}$. Thus, one of the $l+1$ different hashed messages $([\bar{\mathbf{H}}]_k^*)_{k\in[l+1]}$ has never been extracted from $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$ in a completed session. With the game's queries included, there is a $j \in [l+1]$ such that $m_{q_m^*} = m_j^*$ satisfies both of the conditions mentioned above. Note that q_m^* is concealed from the adversary, thus the probability that the adversary correctly guesses q_m^* is at most $1/\hat{Q}_{\mathsf{H}_{\mathsf{m}}}$. Thus, $\Pr[\mathbf{G}_3^{\mathsf{bs}} \Rightarrow 1] \leq \hat{Q}_{\mathsf{H}_{\mathsf{m}}} \cdot \Pr[\mathbf{G}_4^{\mathsf{bs}} \Rightarrow 1]$.

For the query m_i to H_{m}, if $i = q_m^*$, we denote by $m_{q_m^*} = m_i$ and $[\bar{\mathbf{H}}]^\star := \mathsf{H}_{\mathsf{m}}(m_{q_m^*})$. If $\mathcal{A}$ succeeds, we suppose that the game knows $[\bar{\mathbf{H}}]^\star$ from the beginning. The game can randomly choose $[\bar{\mathbf{H}}]^\star$ at the start and output $[\bar{\mathbf{H}}]^\star$ in the q_m^*-th query to H_{m}. We emphasize that the game aborts only when a signing interaction is completed, i.e., the signing oracles $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$ and $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$ have both been executed with extracted $[\bar{\mathbf{H}}]^\star$ and common message τ^*. This is because, $[\bar{\mathbf{H}}]^\star$ can be extracted and executed only in the signing oracle $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$, at which point the session fails to conclude.
Game $\mathbf{G}_5^{\mathsf{bs}}$ (**Sample DDH tuples if** $\tau \neq \tau^*$). For $\mathsf{H}_{\mathsf{ddh}}$, we sample a real DDH tuple, excluding the q_τ^*-th query. When $\mathsf{H}_{\mathsf{ddh}}$ is queried with τ and the hash value remains undefined, the game randomly chooses $d_2 \leftarrow\$\ \mathbb{Z}_p$, sets $(D_2^\tau, D_3^\tau) := (d_2 \cdot G, d_2 \cdot D_1)$ rather than $(D_2, D_3) \leftarrow\$\ \mathbb{G}^2$. Moreover, the game stores the witness d_2 in a table, i.e., $\mathsf{T}_{\mathsf{ddh}}[\tau] := d_2$. Notice that the results (i.e., $(D_2^{\tau^*}, D_3^{\tau^*})$) of the q_τ^*-th query against $\mathsf{H}_{\mathsf{ddh}}$ and further queries using the same input stay unchanged. Thus, for $\tau \neq \tau^*$, we have $\mathbf{D}^\tau \in \mathscr{L}_{\mathsf{ddh}}$. It is clear that we can construct a reduction $\mathcal{B}_1$ for Q-DDH assumption (the Q-DDH assumption requires indistinguishability for Q independent DDH challenges), where $Q = \hat{Q}_{\mathsf{ddh}}$, with $T_{\mathcal{A}}(\lambda) \approx T_{\mathcal{B}_1}(\lambda)$ and $\left|\Pr[\mathbf{G}_4^{\mathsf{bs}} \Rightarrow 1] - \Pr[\mathbf{G}_5^{\mathsf{bs}} \Rightarrow 1]\right| \leq \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{B}_1,\mathsf{Q\text{-}DDH}}(\hat{Q}_{\mathsf{ddh}}, \lambda)$.
Game $\mathbf{G}_6^{\mathsf{bs}}$ (**Utilize DDH witness for** Σ_1 **if** $\tau \neq \tau^*$). For $\tau \neq \tau^*$, we compute Σ_1 transcript $(\mathbf{A}_1^\diamond, c_1^\diamond, z_1^\diamond)$ using the witness $\mathsf{T}_{\mathsf{ddh}}[\tau]$. Specifically, within $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$

for $\tau \neq \tau^*$, the game randomly selects $c_1^\diamond \in \mathcal{S}$ and computes $(\mathbf{A}_1^\diamond, \mathsf{st}_1) := \mathsf{Init}_1(\mathbb{x}_1, \mathbb{w}_1)$, where $\mathbb{w}_1$ is retrieved from $\mathsf{T}_{\mathsf{ddh}}[\tau]$ and $\mathbb{x}_1$ equals $(G, \mathbf{D}^\tau)$. Within $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$ for $\tau \neq \tau^*$, the game generates $z_1^\diamond := \mathsf{Resp}_1(\mathsf{st}_1, c_1^\diamond)$. Remember that in Game $\mathbf{G}_5^{\mathsf{bs}}$, the game randomly selects $c_1^\diamond \leftarrow\$\ \mathcal{S}$ and computes $(\mathbf{A}_1^\diamond, z_1^\diamond) := \mathsf{Sim}_1(\mathbb{x}_1, c_1^\diamond)$. From the perfect HVZK of Σ_1, it follows that the distribution of transcript $(\mathbf{A}_1^\diamond, c_1^\diamond, z_1^\diamond)$ of Σ_1 is the same in Game $\mathbf{G}_5^{\mathsf{bs}}$ and Game $\mathbf{G}_6^{\mathsf{bs}}$. Thus, $\Pr[\mathbf{G}_5^{\mathsf{bs}} \Rightarrow 1] = \Pr[\mathbf{G}_6^{\mathsf{bs}} \Rightarrow 1]$.

Game $\mathbf{G}_7^{\mathsf{bs}}$ (**Simulate Σ_0 if $\tau \neq \tau^*$**). In this game, for $\tau \neq \tau^*$, we simulate the transcript $(\mathbf{A}_0^\diamond, c_0^\diamond, \mathbf{z}_0^\diamond)$ of Σ_0 through the HVZK of Σ_0. Within $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$ for $\tau \neq \tau^*$, the game generates $\mathbf{A}_0^\diamond$ by $c_0^\diamond \leftarrow\$\ \mathcal{S}, (\mathbf{A}_0^\diamond, \mathbf{z}_0^\diamond) := \mathsf{Sim}_0(\mathbb{x}_0^\diamond, c_0^\diamond)$. Within $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$ for $\tau \neq \tau^*$, the game sets $c_1^\diamond := c^\diamond - c_0^\diamond$ and returns $\mathbf{z}_0^\diamond$ from $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$. The computation of $z_1^\diamond$ utilizes $\mathbb{w}_1$, as presented in Game $\mathbf{G}_6^{\mathsf{bs}}$. Remember that in Game $\mathbf{G}_6^{\mathsf{bs}}$, the game sets $c_1^\diamond \leftarrow\$\ \mathcal{S}$ and $c_0^\diamond := c^\diamond - c_1^\diamond$. Therefore, in Game $\mathbf{G}_6^{\mathsf{bs}}$ and Game $\mathbf{G}_7^{\mathsf{bs}}$, the challenges $(c_0^\diamond, c_1^\diamond)$ have the identical distribution. Note that the Σ_0 transcript in Game $\mathbf{G}_6^{\mathsf{bs}}$ is computed honestly. Consequently, using Σ_0's perfect HVZK, we obtain that $\Pr[\mathbf{G}_6^{\mathsf{bs}} \Rightarrow 1] = \Pr[\mathbf{G}_7^{\mathsf{bs}} \Rightarrow 1]$.

We divide Game $\mathbf{G}_7^{\mathsf{bs}}$ into two specific cases, (1) $\tau \neq \tau^*$, or (2) $\tau = \tau^*$ and $[\bar{\mathbf{H}}] = [\bar{\mathbf{H}}]^\star$ has been extracted from $([\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$ by the game. Remember that in $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$, the game computes $\mathbf{T}^\diamond := \phi_0([\mathbf{H}], \mathbb{w}_0) = ([\mathbf{H}] \circ \mathbf{sk} + [\mathbf{W}] \circ \mathbf{s}^\diamond, [\mathbf{G}] \circ \mathbf{s}^\diamond, \mathbf{X})^\top$, where $\mathbb{w}_0 = (\mathbf{s}^\diamond, \mathbf{sk})$ and $\mathbf{s}^\diamond \in \mathcal{D}$ is random. It can be noticed that when (1) occurs, the game uses $\mathbb{w}_0 = (\mathbf{s}^\diamond, \mathbf{sk})$ only to sample $\mathbf{T}^\diamond$ due to the change in Game $\mathbf{G}_7^{\mathsf{bs}}$. In the same way, if (2) occurs, the game uses $\mathbb{w}_0 = (\mathbf{s}^\diamond, \mathbf{sk})$ to sample $\mathbf{T}^\diamond$ and compute $\mathbf{z}_0^\diamond$ in $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$. The case (2) doesn't happen in $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$ due to the setting in Game $\mathbf{G}_4^{\mathsf{bs}}$. In short, the game only utilizes $\mathbb{w}_0$ to sample $\mathbf{T}^\diamond$ in the signing oracles if case (1) or case (2) occurs.

Game $\mathbf{G}_8^{\mathsf{bs}}$ (**Output random $\mathbf{T}^\diamond$ in some sessions**). For the specific cases (1) and (2) discussed above, we modify the signing oracle $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$. In the previous game, the signing oracle computes $\mathbf{T}^\diamond := \phi_0([\mathbf{H}], \mathbb{w}_0)$. In this game, if case (1) or case (2) happens, the game randomly chooses $(\mathbf{T}_1^\diamond, \mathbf{T}_2^\diamond) \leftarrow \mathcal{R}^2$ and sets $\mathbf{T}_3^\diamond = \mathbf{X}$. At an intuitive level, since $([\mathbf{W}], [\mathbf{G}] \circ \mathbf{s}^\diamond, [\mathbf{W}] \circ \mathbf{s}^\diamond)$ within $\mathbf{T}^\diamond$ implies a DDH tuple, replacing $[\mathbf{W}] \circ \mathbf{s}^\diamond$ with a random value should be indistinguishable, and this makes the first part of $\mathbf{T}^\diamond$ to be random.

If adversary $\mathcal{A}$ can distinguish between Game $\mathbf{G}_7^{\mathsf{bs}}$ and Game $\mathbf{G}_8^{\mathsf{bs}}$, we can build a reduction $\mathcal{B}_2$ breaking Q-DDH assumption. The reduction $\mathcal{B}_2$ receives input $(G, H_1, (H_{2,i}, H_{3,i})_{i \in [Q_s]})$ from Q-DDH game and randomly chooses $D_1 \in \mathbb{G}$, $e_{1,1}, e_{1,2}, e_{2,1}, e_{2,2}, w_{1,1}, w_{1,2}, w_{2,1}, w_{2,2} \in \mathbb{Z}_p$, sets

$$[\mathbf{G}] = \begin{bmatrix} e_{1,1} \cdot G & e_{1,2} \cdot G \\ e_{2,1} \cdot G & e_{2,2} \cdot G \end{bmatrix}, [\mathbf{W}] = \begin{bmatrix} H_1 & w_{1,2} \cdot e_{1,2} \cdot G \\ w_{2,1} \cdot e_{2,1} \cdot G & w_{2,2} \cdot e_{2,2} \cdot G \end{bmatrix}, \mathbf{X} := [\mathbf{G}] \circ \mathbf{sk},$$

where D_1 and $\mathbf{sk}$ are defined as in Game $\mathbf{G}_7^{\mathsf{bs}}$. Next, $\mathcal{B}_2$ begins simulating Game $\mathbf{G}_7^{\mathsf{bs}}$ for $\mathcal{A}$ with parameters $(\mathbf{X}, [\mathbf{W}], D_1, [\mathbf{G}], G)$, except during the i-th call to $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$. Additionally, $\mathcal{B}_2$ verifies whether neither case (1) nor case (2) arises. If this holds, $\mathcal{B}_2$ sets $\mathbf{T}^\diamond = \phi_0([\mathbf{H}], \mathbb{w}_0)$ for $\mathbb{w}_0 = (\mathbf{s}^\diamond, \mathbf{sk})$ and randomly chooses $\mathbf{s}^\diamond = (s_1^\diamond, s_2^\diamond) \leftarrow\$\ \mathcal{D}$ as before. Otherwise, for $\mathbf{T}_1^\diamond = [\mathbf{W}] \circ \mathbf{s}^\diamond + (\beta * [\bar{\mathbf{H}}] +$

$(\varphi * \beta) * [\mathbf{G}]) \circ \mathbf{sk} = (T^{\diamond}_{1,1}, T^{\diamond}_{1,2})$, $[\bar{\mathbf{H}}] = [\mathbf{rG}]$ where $([\bar{\mathbf{H}}], \mathsf{td} = \mathbf{r}) \leftarrow \mathsf{Shift}([\mathbf{G}])$,

$$
\begin{aligned}
T^{\diamond}_{1,1} &= s^{\diamond}_1 H_1 + w_{1,2} \cdot e_{1,2} \cdot s^{\diamond}_2 \cdot G + sk_1\big(\beta \cdot r_{1,1} \cdot e_{1,1} + \beta \cdot r_{1,2} \cdot e_{2,1} + \beta \cdot \varphi \cdot e_{1,1}\big) \cdot G \\
&\quad + sk_2\big(\beta \cdot r_{1,1} \cdot e_{1,2} + \beta \cdot r_{1,2} \cdot e_{2,2} + \beta \cdot \varphi \cdot e_{1,2}\big) \cdot G \\
T^{\diamond}_{1,2} &= s^{\diamond}_1 \cdot w_{2,1} \cdot e_{2,1} \cdot G + s^{\diamond}_2 \cdot w_{2,2} \cdot e_{2,2} \cdot G + sk_1\big(\beta \cdot r_{2,1} \cdot e_{1,1} + \beta \cdot r_{2,2} \cdot e_{2,1} + \beta \cdot \varphi \cdot e_{2,1}\big) \cdot G \\
&\quad + sk_2\big(\beta \cdot r_{2,1} \cdot e_{1,2} + \beta \cdot r_{2,2} \cdot e_{2,2} + \beta \cdot \varphi \cdot e_{2,2}\big) \cdot G \\
\mathbf{T}^{\diamond}_2 &= [\mathbf{G}] \circ \mathbf{s}^{\diamond} = \begin{bmatrix} e_{1,1,} \cdot s^{\diamond}_1 \cdot G + e_{1,2} \cdot s^{\diamond}_2 \cdot G \\ e_{2,1} \cdot s^{\diamond}_1 \cdot G + e_{2,2} \cdot s^{\diamond}_2 \cdot G \end{bmatrix}
\end{aligned}
$$

$\mathcal{B}$ sets $T^{\diamond'}_{1,1} = T^{\diamond}_{1,1} - s^{\diamond}_1 \cdot H_1 + H_{3,i}$, $T^{\diamond'}_{2,1} = T^{\diamond}_{2,1} - e_{1,1} \cdot s^{\diamond}_1 \cdot G + H_{2,i}$,

$$
\mathbf{T}^{\diamond'} = \begin{bmatrix} \mathbf{T}^{\diamond'}_1 = (T^{\diamond'}_{1,1}, T^{\diamond}_{1,2}) \\ \mathbf{T}^{\diamond'}_2 = (T^{\diamond'}_{2,1}, T^{\diamond}_{2,2}) \\ \mathbf{T}^{\diamond}_3 \end{bmatrix}.
$$

$\mathcal{B}_2$ generates $\mathbf{A}^{\diamond}_0$ and $\mathbf{A}^{\diamond}_1$ in the same way as in Game $\mathbf{G}^{\mathsf{bs}}_7$ and outputs $(\mathbf{T}^{\diamond'}, \mathbf{A}^{\diamond}_0, \mathbf{A}^{\diamond}_1)$. Moreover, $\mathcal{B}_2$ simulates $\mathcal{O}^{\mathsf{ISign}_2}_{\mathsf{BS}}$ as specified in Game $\mathbf{G}^{\mathsf{bs}}_7$. $\mathcal{B}_2$ returns $b' := 1$ when $\mathcal{A}$ succeeds, and $b' := 0$ otherwise after $\mathcal{A}$ yields its forgeries.

Observe that the parameter $(G, [\mathbf{G}])$ and the public key PK output by $\mathcal{B}_2$ are identically distributed to $(G, [\mathbf{G}], \mathsf{PK})$ in Game $\mathbf{G}^{\mathsf{bs}}_7$ and Game $\mathbf{G}^{\mathsf{bs}}_8$. Moreover, assuming $H_1 = h \cdot G$, $H_{2,i} = s_i \cdot G$, and $H_{3,i} = (h \cdot s_i) \cdot G$ hold for every $i \in [Q_S]$, if case (1) or (2) happens in the i-th invocation of $\mathcal{O}^{\mathsf{ISign}_1}_{\mathsf{BS}}$, then $\mathbf{T}^{\diamond'}$ is distributed identically to $\mathbf{T}^{\diamond}$ in $\mathbf{G}^{\mathsf{bs}}_7$. Otherwise, for every $i \in [Q_S]$, we obtain $H_1 = h \cdot G$, $H_{2,i} = s_i \cdot G$, and $H_{3,i} \leftarrow\$ \mathbb{G}$. If case (1) or (2) happens during the i-th invocation of $\mathcal{O}^{\mathsf{ISign}_1}_{\mathsf{BS}}$, $\mathbf{T}^{\diamond}$ is distributed identically to $\mathbf{T}^{\diamond'}$ in Game $\mathbf{G}^{\mathsf{bs}}_8$, as $H_{3,i}$ operates as a one-time pad. When neither case (1) nor (2) occurs, $\mathbf{T}^{\diamond}$ follows the distribution specified in both Game $\mathbf{G}^{\mathsf{bs}}_7$ and Game $\mathbf{G}^{\mathsf{bs}}_8$ for $\mathcal{O}^{\mathsf{ISign}_1}_{\mathsf{BS}}$ by its construction. Thus, $\big|\Pr[\mathbf{G}^{\mathsf{bs}}_7 \Rightarrow 1] - \Pr[\mathbf{G}^{\mathsf{bs}}_8 \Rightarrow 1]\big| \leq \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{B}_2, \mathsf{Q\text{-}DDH}}(Q_S, \lambda)$.

Game $\mathbf{G}^{\mathsf{bs}}_9$ (Abort if forgeries not in $\mathscr{L}_{\mathsf{bb}}$). In this game, if any of the adversary's forgeries $\sigma^*_k = (\mathbf{S}^*_{1,k}, \mathbf{S}^*_{2,k}, \pi^*_k)$ for message m^*_k fulfills $([\mathbf{G}], [\mathbf{W}], [\bar{\mathbf{H}}]^*_k, \mathbf{S}^*_{1,k}, \mathbf{S}^*_{2,k}, \mathbf{X}) \notin \mathscr{L}_{\mathsf{bb}}$ where $[\bar{\mathbf{H}}]^*_k = \mathsf{H}_{\mathsf{m}}(m^*_k)$, the game aborts. This is achieved efficiently by the following: The game computes $([\mathbf{W}], \mathsf{td}_w = \mathbf{r}_w) \leftarrow \mathsf{Shift}([\mathbf{G}])$ for the public key. The game parses $\sigma^*_k = (\mathbf{S}^*_{1,k}, \mathbf{S}^*_{2,k}, \pi^*_k)$ after $\mathcal{A}$ provides its forgery $(\tau^*, \{m^*_k, \sigma^*_k\}_{k \in [l+1]})$. Next, the game verifies that for each $k \in [l+1]$, the condition holds

$$
\mathbf{S}^*_{1,k} = [\mathbf{W}] \circ \mathbf{s}^{\diamond} + [\bar{\mathbf{H}}]^*_k \circ \mathbf{sk} = \mathsf{Tran}(\mathsf{td}_w, \mathbf{S}^*_{2,k}) + [\bar{\mathbf{H}}]^*_k \circ \mathbf{sk}. \tag{2}
$$

Utilizing the knowledge of td_w and $\mathbf{sk}$, this check can be executed efficiently. If the verification fails, the game aborts; otherwise, it continues as previously.

With probability $1/p$, we have $\mathbb{x}^*_1 \notin \mathscr{L}_{\mathsf{ddh}}$, where $\mathbb{x}^*_1 := (G, \mathbf{D}^{\tau^*})$. The soundness of π^*_k guarantees that $\mathbb{x}^*_{0,k} \in \mathscr{L}_{\mathsf{bb}}$ which is identical to Equation (2), except with negligible probability, where $\mathbb{x}^*_{0,k} = ([\mathbf{G}], [\mathbf{W}], [\bar{\mathbf{H}}]^*_k, \mathbf{S}^*_{1,k}, \mathbf{S}^*_{2,k}, \mathbf{X})$.

Let's analyze the probability that Equation (2) fails to hold for some $j \in [l+1]$. We start by proving two propositions. The first relies on the soundness of the Fiat-Shamir transformation, while the second binds Equation (2) to $\mathscr{L}_{\mathsf{bb}}$.

Proposition 1. *Whenever an* H_{c} *query on* $((\mathbb{x}_b, \mathbf{A}_b)_{b\in\{0,1\}}, m)$ *with* $\mathbb{x}_0 \notin \mathscr{L}_{\mathsf{bb}}$ *occurs, there is* $(c_0, c_1, \mathbf{z}_0, z_1)$ *such that*

$$\mathsf{tr}_0 := (\mathbf{A}_0, c_0, \mathbf{z}_0) \text{ is valid for } \mathbb{x}_0 \tag{3}$$

$$\mathsf{tr}_1 := (\mathbf{A}_1, c_1, z_1) \text{ is valid for } \mathbb{x}_1 \tag{4}$$

$$c^* := \mathsf{H}_{\mathsf{c}}((\mathbb{x}_b, \mathbf{A}_b)_{b\in\{0,1\}}, m_j^*) = c_0 + c_1 \tag{5}$$

with probability bounded by $1/p$.

Proof. Σ_0 possesses special soundness, implying that given $\mathbb{x}_0 \notin \mathscr{L}_{\mathsf{bb}}$, there exists at most one challenge $c_0 \in \mathcal{S} = \mathbb{Z}_p$ for which a $\mathbf{z}_0$ exists, making $\mathsf{tr}_0 := (\mathbf{A}_0, c_0, \mathbf{z}_0)$ a valid transcript. In the same way, for $\mathbb{x}_1 \notin \mathscr{L}_{\mathsf{ddh}}$, there exists at most one challenge $c_1 \in \mathcal{S} = \mathbb{Z}_p$ for which a z_1 exists, making $\mathsf{tr}_1 = (\mathbf{A}_1, c_1, z_1)$ a valid transcript. Hence, (c_0, c_1) is uniquely determined by $(\mathbb{x}_b, \mathbf{A}_b)_{b\in\{0,1\}}$ owing to Equations (3) and (4). Since $(\mathbb{x}_b, \mathbf{A}_b)_{b\in\{0,1\}}$ is a component of the input to the H_{c} query that defines c^*, we conclude that c^* is independent of (c_0, c_1). Thus, the probability that Equation (5) holds is bounded by $1/p$.

Proposition 2. *Equation (2) holds iff* $([\mathbf{G}], [\mathbf{W}], [\bar{\mathbf{H}}]_k^*, \mathbf{S}_{1,k}^*, \mathbf{S}_{2,k}^*, \mathbf{X}) \in \mathscr{L}_{\mathsf{bb}}$.

Proof. This is guaranteed by the translatability of the TLF. Assuming $(G, \mathbf{D}^{\tau^*}) \notin \mathscr{L}_{\mathsf{ddh}}$, which is valid with the exception of a probability $1/p$. Since all $l+1$ forgeries of the adversary are valid, Equations (3) to (5) hold. Thus for all $k \in [l+1]$, $\mathbb{x}_{0,k}^* \in \mathscr{L}_{\mathsf{bb}}$, except with probability $\hat{Q}_{\mathsf{H}_{\mathsf{c}}}/p$ (Proposition 1). For all $k \in [l+1]$, Equation (2) holds (Proposition 2). Thus, $\left|\Pr[\mathbf{G}_8^{\mathsf{bs}} \Rightarrow 1] - \Pr[\mathbf{G}_9^{\mathsf{bs}} \Rightarrow 1]\right| \leq \hat{Q}_{\mathsf{H}_{\mathsf{c}}}/p$.
Game $\mathbf{G}_{10}^{\mathsf{bs}}$ (Sample DDH tuple if $\tau = \tau^*$). The game chooses a real DDH tuple for $\mathsf{H}_{\mathsf{ddh}}$ on the q_τ^* query. The game randomly choose $d_2 \leftarrow\$\ \mathbb{Z}_p$, sets $(D_2^{\tau^*}, D_3^{\tau^*}) := (d_2 \cdot G, d_2 \cdot D_1)$ rather than $(D_2^{\tau^*}, D_3^{\tau^*}) \leftarrow\$\ \mathbb{G}^2$, stores d_2 in $\mathsf{T}_{\mathsf{ddh}}[\tau^*]$. It is easy to devise a reduction $\mathcal{A}_{\mathsf{DDH}}^3$ against DDH with $T_{\mathcal{A}}(\lambda) \approx T_{\mathcal{A}_{\mathsf{DDH}}^3}(\lambda)$ and $\left|\Pr[\mathbf{G}_9^{\mathsf{bs}} \Rightarrow 1] - \Pr[\mathbf{G}_{10}^{\mathsf{bs}} \Rightarrow 1]\right| \leq \mathsf{Adv}_{\mathcal{A}_{\mathsf{DDH}}^3}^{\mathsf{GGen}}(\lambda)$.

Observe that, we have $\mathbf{D}^{\tau} \in \mathscr{L}_{\mathsf{ddh}}$ for all common messages τ.
Game $\mathbf{G}_{11}^{\mathsf{bs}}$ (Utilize DDH witness for Σ_1 if $\tau = \tau^*$). In $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$ for $\tau = \tau^*$, the game uniformly chooses $c_1^\diamond$ from $\mathbb{Z}_p$ and executes $(\mathbf{A}_1^\diamond, \mathsf{st}_1) := \mathsf{Init}_1(\mathbb{x}_1, \mathbb{w}_1)$, where $\mathbb{w}_1$ is $\mathsf{T}_{\mathsf{ddh}}[\tau^*]$ and $\mathbb{x}_1$ is $(G, \mathbf{D}^{\tau^*})$. In $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$ for $\tau = \tau^*$, the game obtains $z_1^\diamond$ by executing $\mathsf{Resp}_1(\mathsf{st}_1, c_1^\diamond)$. From the HVZK of Σ_1, we know that Σ_1 transcripts $(\mathbf{A}_1^\diamond, c_1^\diamond, z_1^\diamond)$ for $\tau = \tau^*$ have the identical distribution in Game $\mathbf{G}_{10}^{\mathsf{bs}}$ and Game $\mathbf{G}_{11}^{\mathsf{bs}}$. Therefore, we obtain $\Pr[\mathbf{G}_{10}^{\mathsf{bs}} \Rightarrow 1] = \Pr[\mathbf{G}_{11}^{\mathsf{bs}} \Rightarrow 1]$.
Game $\mathbf{G}_{12}^{\mathsf{bs}}$ (Simulate Σ_0 if $\tau = \tau^*$). Within $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$ for $\tau = \tau^*$, the game randomly chooses $c_0^\diamond \leftarrow\$\ \mathcal{S} = \mathbb{Z}_p$ and computes $(\mathbf{A}_0^\diamond, \mathbf{z}_0^\diamond) := \mathsf{Sim}_0(\mathbb{x}_0^\diamond, c_0^\diamond)$ for $\mathbb{x}_0^\diamond = ([\mathbf{G}], [\mathbf{W}], [\mathbf{H}], \mathbf{T}^\diamond)$. Within $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$ for $\tau = \tau^*$, the game sets $c_1^\diamond := c^\diamond - c_0^\diamond$ and

returns $\mathbf{z}_0^\diamond$ from $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$. The remaining branch (i.e., $z_1^\diamond$) is derived from $\mathbb{w}_1$ similar to the process in Game $\mathbf{G}_{11}^{\mathsf{bs}}$. For both Game $\mathbf{G}_{11}^{\mathsf{bs}}$ and Game $\mathbf{G}_{12}^{\mathsf{bs}}$, the challenges $c_0^\diamond$ and $c_1^\diamond$ exhibit identical distribution, and the Σ_0 transcripts $(\mathbf{A}_0^\diamond, c_0^\diamond, \mathbf{z}_0^\diamond)$ exhibit the same distribution due to the HVZK of Σ_0. Therefore, we obtain $\Pr[\mathbf{G}_{11}^{\mathsf{bs}} \Rightarrow 1] = \Pr[\mathbf{G}_{12}^{\mathsf{bs}} \Rightarrow 1]$.

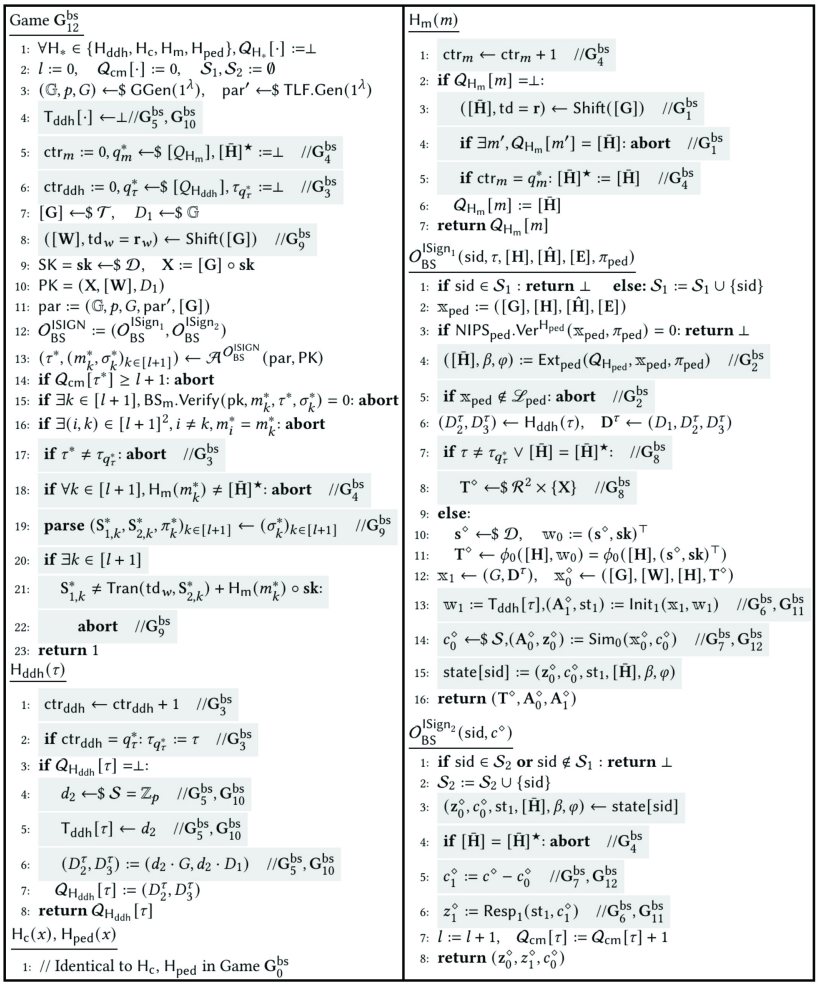

Fig. 15. Game $\mathbf{G}_{12}^{\mathsf{bs}}$. The changes to Game $\mathbf{G}_0^{\mathsf{bs}}$ are grey.

Game $\mathbf{G}_{12}^{\mathsf{bs}}$ is shown in Fig. 15. The public key is initialized similarly as in $\mathsf{BS}_{\mathsf{m}}.\mathsf{KeyGen}$, except $([\mathbf{W}], \mathsf{td}_w = \mathbf{r}_w) \leftarrow \mathsf{Shift}([\mathbf{G}])$. The game guesses the first $\mathsf{H}_{\mathsf{ddh}}$ query of τ^*. The game guesses the first H_{m} query on $m_{q_m^*}$, i.e., $[\bar{\mathbf{H}}]^\star = \mathsf{H}_{\mathsf{m}}(m_{q_m^*})$ and there is no completed signing session with common message τ^* if $[\bar{\mathbf{H}}]^\star$ can be derived from π_{ped}. The game's simulation of the signing oracles is as

follows: In $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$, if $\tau = \tau^*$ and $[\bar{\mathbf{H}}] \neq [\bar{\mathbf{H}}]^\star$, game outputs the honestly computed $\mathbf{T}^\diamond = (\mathbf{T}_1^\diamond, \mathbf{T}_2^\diamond, \mathbf{X})$. Otherwise the first two parts of $\mathbf{T}^\diamond$ are randomly chosen from $\mathcal{R}$. The Σ_1 transcripts $(\mathbf{A}_1^\diamond, c_1^\diamond, z_1^\diamond)$ in $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$ and $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$ are calculated by DDH witness for $\mathbf{D}^\tau$. The Σ_0 transcripts $(\mathbf{A}_0^\diamond, c_0^\diamond, \mathbf{z}_0^\diamond)$ are obtained through HVZK simulation of Σ_0. Eventually, if the forgeries submitted by the adversary are not in $\mathscr{L}_{\mathsf{bb}}$ (Equation (2) with td_w), the game aborts. Note that td_w is only used to perform the Equation (2) check in Game $\mathbf{G}_{12}^{\mathsf{bs}}$, which is only performed after adversary outputs forgeries.

Reduce to CDH. We can build a reduction $\mathcal{A}_{\mathsf{CDH}}$ such that $\Pr[\mathbf{G}_{12}^{\mathsf{bs}} \Rightarrow 1] \leq \mathsf{Adv}_{\mathcal{A}_{\mathsf{CDH}}}^{\mathsf{GGen}}(\lambda)$, which follows Lemma 1. We construct a reduction $\mathcal{B}_3$ which produces $\mathbb{x}_0^\star \in \mathscr{L}_{\mathsf{bb}}$ for the game (Game $\mathsf{Game}_{\mathsf{Sig}}^{\mathsf{BB}}$) in Lemma 1. Notice that $\mathcal{B}_3$ has access to oracle $\mathcal{O}_{\mathsf{Sig}_\mathsf{m}}(m)$ that outputs values $(\mathbf{S}_1, \mathbf{S}_2)$ and random oracle $\mathsf{H}_\mathsf{m}(m)$ that outputs value $[\bar{\mathbf{H}}]$. First, $\mathcal{B}_3$ obtains $([\mathbf{G}], [\mathbf{W}], \mathbf{X})$ from Game $\mathsf{Game}_{\mathsf{Sig}}^{\mathsf{BB}}$. Next, $\mathcal{B}_3$ samples $D_1 \leftarrow\$\ \mathbb{G}$ and sets $\mathsf{PK} = (\mathbf{X}, [\mathbf{W}], D_1)$. Additionally, $\mathcal{B}_3$ sets $\tau_{q_\tau^*} := \perp$, chooses q_m^* and q_τ^* at random, and sets the counters $\mathsf{ctr}_{\mathsf{ddh}}$ and ctr_m to 0. Subsequently, $\mathcal{A}$ is invoked on input PK, and oracles from Game $\mathbf{G}_{12}^{\mathsf{bs}}$ are simulated for $\mathcal{A}$ as follows.

- $\mathsf{H}_\mathsf{m}, \mathsf{H}_{\mathsf{ddh}}, \mathsf{H}_\mathsf{c}, \mathsf{H}_{\mathsf{ped}}, \mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$: They are the same as in Game $\mathbf{G}_{12}^{\mathsf{bs}}$. For H_m, return $[\bar{\mathbf{H}}]^\star$ on the q_m^*-th query. If $[\bar{\mathbf{H}}]^\star$ is extracted from $([\mathbf{G}], [\mathbf{H}], [\hat{\mathbf{H}}], [\mathbf{E}])$ in $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$, the game $\mathbf{G}_{12}^{\mathsf{bs}}$ aborts.
- $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$: If the verification of π_{ped} fails, output $\perp$. Otherwise, extract $([\bar{\mathbf{H}}], \beta, \varphi)$ such that $[\mathbf{E}] = \varphi * [\mathbf{G}] \wedge [\hat{\mathbf{H}}] = \beta * [\bar{\mathbf{H}}] \wedge [\mathbf{H}] - [\hat{\mathbf{H}}] = \beta * [\mathbf{E}]$. In the case where $\tau \neq \tau_{q_\tau^*}$, randomly choose $\mathbf{T}_1^\diamond, \mathbf{T}_2^\diamond$ from $\mathcal{R}$ and configure $\mathbf{T}^\diamond = (\mathbf{T}_1^\diamond, \mathbf{T}_2^\diamond, \mathbf{X})$ following Game $\mathbf{G}_{12}^{\mathsf{bs}}$; if not, invoke $\mathcal{O}_{\mathsf{Sig}_\mathsf{m}}(m)$ to get $(\mathbf{S}_1, \mathbf{S}_2)$ and set $\mathbf{T}^\diamond$ to $(\beta * \mathbf{S}_1 + (\beta * \varphi) * \mathbf{X}, \beta * \mathbf{S}_2, \mathbf{X})$. Subsequently, continue as per the protocol of Game $\mathbf{G}_{12}^{\mathsf{bs}}$.

Once $\mathcal{A}$ generates its forgeries $(m_k^*, \sigma_k^*)_{k \in [l+1]}$ on τ^*, $\mathcal{B}_3$ determines whether there exists a message m_j^* ($m^\star$) satisfying $\mathsf{H}_\mathsf{m}(m_j^*) = [\bar{\mathbf{H}}]^\star$. In the end, $\mathcal{B}_3$ parses $\sigma_j^* = \sigma^\star = (\mathbf{S}_1^\star, \mathbf{S}_2^\star, \pi^\star)$ and returns $(m^\star, [\bar{\mathbf{H}}]^\star, \mathbf{S}_1^\star, \mathbf{S}_2^\star)$ to Game $\mathsf{Game}_{\mathsf{Sig}}^{\mathsf{BB}}$.

The simulated PK is consistent with the distribution in Game $\mathbf{G}_{12}^{\mathsf{bs}}$. Crucially, the distribution of $\mathbf{T}^\diamond$ remains consistent when $\tau = \tau_{q_\tau^*}$. The output $(\mathbf{S}_1, \mathbf{S}_2)$ of $\mathcal{O}_{\mathsf{Sig}_\mathsf{m}}(m)$ in Lemma 1 satisfies $\mathbf{S}_1 = [\mathbf{W}] \circ \mathbf{s} + [\bar{\mathbf{H}}] \circ \mathbf{sk}$ and $\mathbf{S}_2 = [\mathbf{G}] \circ \mathbf{s}$, where $\mathbf{s} \leftarrow\$\ \mathcal{D}$. The simulated $\mathbf{T}_3^\diamond$ is consistent with the distribution in Game $\mathbf{G}_{12}^{\mathsf{bs}}$, due to the fact that

$$\begin{aligned}\mathbf{T}_1^\diamond &= \beta * \mathbf{S}_1 + (\beta * \varphi) * \mathbf{X} = \beta * [\mathbf{W}] \circ \mathbf{s} + \beta * [\bar{\mathbf{H}}] \circ \mathbf{sk} + \beta * \varphi * \mathbf{X} \\ &= [\mathbf{W}] \circ (\beta * \mathbf{s}) + \beta * [\bar{\mathbf{H}}] \circ \mathbf{sk} + \beta * \varphi * [\mathbf{G}] \circ \mathbf{sk} = [\mathbf{W}] \circ (\beta * \mathbf{s}) + [\mathbf{H}] \circ \mathbf{sk},\end{aligned}$$

$$\mathbf{T}_2^\diamond = \beta * \mathbf{S}_2 = \beta * [\mathbf{G}] \circ \mathbf{s} = [\mathbf{G}] \circ (\beta * \mathbf{s}),$$

the simulated $(\mathbf{T}_1^\diamond, \mathbf{T}_2^\diamond)$ is consistent with the distribution in Game $\mathbf{G}_{12}^{\mathsf{bs}}$. Thus, $\mathcal{A}$ observes the same view as in Game $\mathbf{G}_{12}^{\mathsf{bs}}$, and with probability at least $\Pr[\mathbf{G}_{12}^{\mathsf{bs}} \Rightarrow 1]$, there exists a message m_j^* such that $\mathsf{H}_\mathsf{m}(m_j^*) = [\bar{\mathbf{H}}]^\star$ and Equation (2) holds.

As presented in Game $\mathbf{G}_9^{\mathsf{bs}}$, this means $([\mathbf{G}], [\mathbf{W}], [\tilde{\mathbf{H}}]^{\star}, \mathbf{S}_1^{\star}, \mathbf{S}_2^{\star}, \mathbf{X}) \in \mathscr{L}_{\mathsf{bb}}$. According to Lemma 1, there exists an adversary $\mathcal{A}_{\mathsf{CDH}}$ with $T_{\mathcal{A}}(\lambda) \approx T_{\mathcal{A}_{\mathsf{CDH}}}(\lambda)$ and $\Pr[\mathbf{G}_{12}^{\mathsf{bs}} \Rightarrow 1] \leq \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{A}_{\mathsf{CDH}}}$.

By aggregating all the bounds and using the tight reduction [52] between the DDH assumption and the Q-DDH assumption, i.e., $\mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{F}_{\mathsf{Q\text{-}DDH}}}(Q, \lambda) \leq \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{F}_{\mathsf{DDH}}}(\lambda) + 1/(p+1)$, we derive the statement.

D.3 Proof of Lemma 2

Proof. (Lemma 2) Within the proof of Theorem 2, we employ a series of intermediate games between Game $\mathbf{G}_1^{\mathsf{bs}}$ and Game $\mathbf{G}_2^{\mathsf{bs}}$. Game $\mathbf{G}_{1.1}^{\mathsf{bs}}$ employs the knowledge extractor to extract a witness $\mathbb{w}_{\mathsf{ped}}$ from π_{ped} in $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$. From Game $\mathbf{G}_{1.2}^{\mathsf{bs}}$ to Game $\mathbf{G}_{1.5}^{\mathsf{bs}}$, we simulate the signing oracle without $\mathsf{sk} = \mathbf{sk}$. In Game $\mathbf{G}_{1.6}^{\mathsf{bs}}$, if $\mathbb{w}_{\mathsf{ped}}$ is the preimage of $\mathbf{X}$ (which means the DL assumption stating that given $(\mathbb{G}, p, G, a \cdot G)$, it is computationally infeasible to compute a), the game aborts. We then revert the changes from Game $\mathbf{G}_{1.2}^{\mathsf{bs}}$ to Game $\mathbf{G}_{1.5}^{\mathsf{bs}}$ and preserve the abort conditions in $\mathbf{G}_{1.6}^{\mathsf{bs}}$. This gives the same game as Game $\mathbf{G}_2^{\mathsf{bs}}$.

Game $\mathbf{G}_{1.1}^{\mathsf{bs}}$ (Extract $\mathbb{w}_{\mathsf{ped}}$ from π_{ped}). In this game, we change $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$: At first, it follows the same process as Game $\mathbf{G}_1^{\mathsf{bs}}$ until it performs the check $\mathsf{NIPS}_{\mathsf{ped}}.\mathsf{Ver}^{\mathsf{H}_{\mathsf{ped}}}(\mathbb{x}_{\mathsf{ped}}, \mathbb{w}_{\mathsf{ped}}) = 1$. In case the check turns out to be a failure, it outputs $\perp$ in the same way as before. On the other hand, if the check is successful, the game sets $\mathbb{w}_{\mathsf{ped}} \leftarrow \mathsf{Ext}_{\mathsf{ped}}(\mathcal{Q}_{\mathsf{H}_{\mathsf{ped}}}, \mathbb{x}_{\mathsf{ped}}, \pi_{\mathsf{ped}})$, where $\mathcal{Q}_{\mathsf{H}_{\mathsf{ped}}}$ denotes the queries to $\mathsf{H}_{\mathsf{ped}}$. If $(\mathbb{x}_{\mathsf{ped}}, \mathbb{w}_{\mathsf{ped}}) \notin \tilde{\mathsf{R}}_{\mathsf{ped}}$, the game aborts. We can construct a reduction $\mathcal{A}_{\mathsf{KS}}$ with $T_{\mathcal{A}}(\lambda) \approx T_{\mathcal{A}_{\mathsf{KS}}}(\lambda)$, ensuring that $\left|\Pr[\mathbf{G}_1^{\mathsf{bs}} \Rightarrow 1] - \Pr[\mathbf{G}_{1.1}^{\mathsf{bs}} \Rightarrow 1]\right| \leq \mathsf{Adv}^{\mathsf{NIPS}_{\mathsf{ped}}, \tilde{\mathsf{R}}_{\mathsf{ped}}}_{\mathcal{A}_{\mathsf{KS}}}(\lambda)$.

Game $\mathbf{G}_{1.2}^{\mathsf{bs}}$ (Sample DDH tuples). In this game, for $\mathsf{H}_{\mathsf{ddh}}$, we sample the real DDH tuple. When $\mathsf{H}_{\mathsf{ddh}}$ is queried with τ and the hash value remains undefined, the game randomly chooses $d_2 \leftarrow\$\ \mathbb{Z}_p$, sets $(D_2^{\tau}, D_3^{\tau}) := (d_2 \cdot G, d_2 \cdot D_1)$ rather than $(D_2, D_3) \leftarrow\$\ \mathbb{G}^2$. Moreover, the game stores the witness d_2 in a table, i.e., $\mathsf{T}_{\mathsf{ddh}}[\tau] := d_2$. Thus, for all τ, $\mathbf{D}^{\tau} \in \mathscr{L}_{\mathsf{ddh}}$. It is clear that we can construct a reduction $\mathcal{B}_1$ for Q-DDH ($Q = \hat{Q}_{\mathsf{ddh}}$), with $T_{\mathcal{A}}(\lambda) \approx T_{\mathcal{B}_1}(\lambda)$ and $\left|\Pr[\mathbf{G}_{1.1}^{\mathsf{bs}} \Rightarrow 1] - \Pr[\mathbf{G}_{1.2}^{\mathsf{bs}} \Rightarrow 1]\right| \leq \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{B}_1', \mathsf{Q\text{-}DDH}}(\hat{Q}_{\mathsf{ddh}}, \lambda)$.

Game $\mathbf{G}_{1.3}^{\mathsf{bs}}$ (Use DDH witness for Σ_1). In this game, we compute Σ_1 transcript $(\mathbf{A}_1^{\diamond}, c_1^{\diamond}, z_1^{\diamond})$ using witness $\mathsf{T}_{\mathsf{ddh}}[\tau]$. Within $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$, the game randomly selects $c_1^{\diamond} \in \mathcal{S}$ and computes $(\mathbf{A}_1^{\diamond}, \mathsf{st}_1) := \mathsf{Init}_1(\mathbb{x}_1, \mathbb{w}_1)$, where $\mathbb{w}_1$ is derived from $\mathsf{T}_{\mathsf{ddh}}[\tau]$ and $\mathbb{x}_1$ equals $(G, \mathbf{D}^{\tau})$. Within $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$, the game generates $z_1^{\diamond} := \mathsf{Resp}_1(\mathsf{st}_1, c_1^{\diamond})$. From the perfect HVZK of Σ_1, the distribution of transcript $(\mathbf{A}_1^{\diamond}, c_1^{\diamond}, z_1^{\diamond})$ of Σ_1 is identical in Game $\mathbf{G}_{1.2}^{\mathsf{bs}}$ and Game $\mathbf{G}_{1.3}^{\mathsf{bs}}$. Thus, we get $\Pr[\mathbf{G}_{1.2}^{\mathsf{bs}} \Rightarrow 1] = \Pr[\mathbf{G}_{1.3}^{\mathsf{bs}} \Rightarrow 1]$.

Game $\mathbf{G}_{1.4}^{\mathsf{bs}}$ (Simulate Σ_0). In this game, we simulate the transcript $(\mathbf{A}_0^{\diamond}, c_0^{\diamond}, \mathbf{z}_0^{\diamond})$ of Σ_0 according to the HVZK of Σ_0. Within $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$, the game generates $\mathbf{A}_0^{\diamond}$ by $c_0^{\diamond} \leftarrow\$\ \mathcal{S}, (\mathbf{A}_0^{\diamond}, \mathbf{z}_0^{\diamond}) := \mathsf{Sim}_0(\mathbb{x}_0^{\diamond}, c_0^{\diamond})$. Within $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_2}$, the game sets $c_1^{\diamond} := c^{\diamond} - c_0^{\diamond}$ and returns $\mathbf{z}_0^{\diamond}$ from $\mathcal{O}_{\mathsf{BS}}^{\mathsf{ISign}_1}$. The computation of $z_1^{\diamond}$ utilizes $\mathbb{w}_1$, as presented in

Game $\mathbf{G}^{\mathsf{bs}}_{1.3}$. Consequently, using Σ_0's perfect HVZK, we obtain that $\Pr[\mathbf{G}^{\mathsf{bs}}_{1.3} \Rightarrow 1] = \Pr[\mathbf{G}^{\mathsf{bs}}_{1.4} \Rightarrow 1]$.

Game $\mathbf{G}^{\mathsf{bs}}_{1.5}$ (Send random $\mathbf{T}^{\diamond}$). In the previous game, the signing oracle computes $\mathbf{T}^{\diamond} := \phi_0([\mathbf{H}], \mathbb{w}_0)$. Now, the game randomly chooses $(\mathbf{T}^{\diamond}_1, \mathbf{T}^{\diamond}_2) \leftarrow \mathcal{R}^2$ and sets $\mathbf{T}^{\diamond}_3 = \mathbf{X}$. At an intuitive level, since $([\mathbf{W}], [\mathbf{G}] \circ \mathbf{s}^{\diamond}, [\mathbf{W}] \circ \mathbf{s}^{\diamond})$ within $\mathbf{T}^{\diamond}$ implies a DDH tuple, replacing $[\mathbf{W}] \circ \mathbf{s}^{\diamond}$ with a random value should be indistinguishable, and this would cause the first part of $\mathbf{T}^{\diamond}$ to be random. The intuition here can be formally expressed by the construction of Game $\mathbf{G}^{\mathsf{bs}}_{8}$. We can construct a reduction $\mathcal{B}_2$ breaking Q-DDH. Thus, we have

$$\left|\Pr[\mathbf{G}^{\mathsf{bs}}_{1.4} \Rightarrow 1] - \Pr[\mathbf{G}^{\mathsf{bs}}_{1.5} \Rightarrow 1]\right| \leq \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{B}'_2, \mathsf{Q\text{-}DDH}}(Q_S, \lambda).$$

At this point, the game simulates the signing oracles via Σ_1's witness and a randomized $\mathbf{T}^{\diamond}$ without the secret key $\mathsf{sk} = \mathbf{sk}$.

Game $\mathbf{G}^{\mathsf{bs}}_{1.6}$ (Abort if $(\mathbb{x}_{\mathsf{ped}}, \mathbb{w}_{\mathsf{ped}}) \notin \mathsf{R}_{\mathsf{ped}}$). If $(\mathbb{x}_{\mathsf{ped}}, \mathbb{w}_{\mathsf{ped}}) \notin \mathsf{R}_{\mathsf{ped}}$, the game aborts. After the game extracts $\mathbb{w}_{\mathsf{ped}} \leftarrow \mathsf{Ext}_{\mathsf{ped}}(\mathcal{Q}_{\mathsf{H}_{\mathsf{ped}}}, \mathbb{x}_{\mathsf{ped}}, \pi_{\mathsf{ped}})$, the game parses $([\bar{\mathbf{H}}], \beta, \varphi) := \mathbb{w}_{\mathsf{ped}}$. If parsing $\mathbb{w}_{\mathsf{ped}}$ fails or $\mathbb{x}_{\mathsf{ped}} \notin \mathscr{L}_{\mathsf{ped}}$ i.e., $[\mathbf{E}] \neq \varphi * [\mathbf{G}] \vee [\hat{\mathbf{H}}] \neq \beta * [\bar{\mathbf{H}}] \vee [\mathbf{H}] - [\hat{\mathbf{H}}] \neq \beta * [\mathbf{E}]$, the game aborts. Because of the abort condition added in Game $\mathbf{G}^{\mathsf{bs}}_{1.1}$, it holds that $(\mathbb{x}_{\mathsf{ped}}, \mathbb{w}_{\mathsf{ped}}) \in \mathsf{R}_{\mathsf{ped}}$. Moreover, by definition of $\mathsf{R}_{\mathsf{ped}}$, if $(\mathbb{x}_{\mathsf{ped}}, \mathbb{w}_{\mathsf{ped}}) \in \mathsf{R}_{\mathsf{ped}}$ it holds that $\mathbb{w}_{\mathsf{ped}} = ([\bar{\mathbf{H}}], \beta, \varphi)$ and $[\mathbf{E}] = \varphi * [\mathbf{G}] \wedge [\hat{\mathbf{H}}] = \beta * [\bar{\mathbf{H}}] \wedge [\mathbf{H}] = \beta * [\bar{\mathbf{H}}] + (\varphi * \beta) * [\mathbf{G}]$. The next step is to bound the probability that $\mathbb{w}_{\mathsf{ped}} = \mathbf{sk}$ with $\mathbf{X} = [\mathbf{G}] \circ \mathbf{sk}$. To achieve this, we construct a reduction $\mathcal{B}'_3$ to the DL assumption. $\mathcal{B}'_3$ simulates Game $\mathbf{G}^{\mathsf{bs}}_{1.5}$ for adversary $\mathcal{A}$. However, $\mathcal{B}'_3$ embeds a DL challenge X_1 into the public key $\mathbf{X}$, i.e.,

$$[\mathbf{G}] = \begin{bmatrix} G & G \\ G_{2,1} & G_{2,2} \end{bmatrix}, \mathbf{X} = \begin{bmatrix} X_1 \\ X_2 \end{bmatrix},$$

where G is generator of $\mathbb{G}$ and $G_{2,1}, G_{2,2}, X_2 \in \mathbb{G}$. If some extracted $\mathbb{w}_{\mathsf{ped}} = \mathbf{sk}$ in $\mathcal{O}^{\mathsf{ISign}_1}_{\mathsf{BS}}$ fulfils $\mathbf{X} = [\mathbf{G}] \circ \mathbf{sk}$, $\mathcal{B}'_3$ outputs $sk_1 + sk_2$ as DL solution. It should be noted that $\mathcal{A}_{\mathsf{DL}}$ is well-defined since the Game $\mathbf{G}^{\mathsf{bs}}_{1.5}$ can be simulated without the preimage $\mathbf{sk}$ of $\mathbf{X}$. Thus,

$$\left|\Pr[\mathbf{G}^{\mathsf{bs}}_{1.5} \Rightarrow 1] - \Pr[\mathbf{G}^{\mathsf{bs}}_{1.6} \Rightarrow 1]\right| \leq \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{A}_{\mathsf{DL}}}(\lambda).$$

Game $\mathbf{G}^{\mathsf{bs}}_{1.7}$ to Game $\mathbf{G}^{\mathsf{bs}}_{1.10}$ (Revert back the changes). In Games $\mathbf{G}^{\mathsf{bs}}_{1.7}$ to $\mathbf{G}^{\mathsf{bs}}_{1.10}$, we undo the modifications introduced in Games $\mathbf{G}^{\mathsf{bs}}_{1.2}$ to $\mathbf{G}^{\mathsf{bs}}_{1.5}$, while we retain the abort condition added in Game $\mathbf{G}^{\mathsf{bs}}_{1.6}$. Roughly, in Game $\mathbf{G}^{\mathsf{bs}}_{1.7}$, we set $\mathbf{T}^{\diamond} := \phi_0([\bar{\mathbf{H}}], \mathbb{w}_0)$ again. This modification is supported by the Q-DDH assumption. In Game $\mathbf{G}^{\mathsf{bs}}_{1.8}$, Σ_0 is computed using the witness $\mathbb{w}_0$ afresh, and in Game $\mathbf{G}^{\mathsf{bs}}_{1.9}$, we employ the HVZK simulator to generate the Σ_1 transcript. In Game $\mathbf{G}^{\mathsf{bs}}_{1.10}$, we produce random elements for $\mathsf{H}_{\mathsf{ddh}}$ afresh, a step again supported by the Q-DDH assumption. Notice that Game $\mathbf{G}^{\mathsf{bs}}_{1.10}$ is the same as Game $\mathbf{G}^{\mathsf{bs}}_{2}$.

By aggregating all the previously established bounds, and using the tight reduction [52] between the DDH assumption and the Q-DDH assumption, i.e., $\mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{F}_{\mathsf{Q\text{-}DDH}}}(Q, \lambda) \leq \mathsf{Adv}^{\mathsf{GGen}}_{\mathcal{F}_{\mathsf{DDH}}}(\lambda) + 1/(p+1)$, we can get the statement in Lemma 2.

E Blindness of Rainblind

Theorem 4. (**Computational Blindness** *of* Rainblind) *Let* DS *be a digital signature scheme, for any adversary* $\mathcal{A}$ *for the game* $\mathsf{Game}^{\mathsf{Blind}}_{\mathsf{TBS}}$ *making at most* Q_{H_*} *queries to* $\mathsf{H}_* \in \{\mathsf{H_c}, \mathsf{H_{ddh}}, \mathsf{H_{ped}}, \mathsf{H_m}, \mathsf{H_{cm}}\}$ *modeled as random oracles, there exists a PPT adversary* $\mathcal{B}$ *with* $T_{\mathcal{B}} \approx T_{\mathcal{A}}$ *against the game* $\mathsf{Game}^{\mathsf{Blind}}_{\mathsf{BS}}$ *making at most* Q_{H_*} *queries to the random oracle, such that* $\mathsf{Adv}^{\mathsf{TBS\text{-}Blind}}_{\mathcal{A},\mathsf{Rainblind}}(\lambda) = \mathsf{Adv}^{\mathsf{BS\text{-}Blind}}_{\mathcal{B},\mathsf{BS_m}}(\lambda)$.

Proof. The core argument is to illustrate that for any adversary $\mathcal{A}$ involved in the threshold blindness game $\mathsf{Game}^{\mathsf{Blind}}_{\mathsf{TBS}}$ (Definition 11) implemented with Rainblind, there is an adversary $\mathcal{B}$ engaging in the single-party blindness game $\mathsf{Game}^{\mathsf{Blind}}_{\mathsf{BS}}$ (Definition 7) implemented with $\mathsf{BS_m}$ such that $\mathsf{Adv}^{\mathsf{TBS\text{-}Blind}}_{\mathcal{A},\mathsf{Rainblind}}(\lambda) = \mathsf{Adv}^{\mathsf{BS\text{-}Blind}}_{\mathcal{B},\mathsf{BS_m}}(\lambda)$. We have the following observations: $\mathsf{Rainblind.USign}_0$ is the same as $\mathsf{BS_m.USign}_0$ except for the issuer set $\mathcal{SS}$ (unrelated to the blinding operations). $\mathsf{Rainblind.USign}_1$ is consistent with $\mathsf{BS_m.USign}_1$ except for aggregation to $\mathbf{T}^{\diamond}_i, \mathbf{A}^{\diamond}_{0,i}, \mathbf{A}^{\diamond}_{1,i}$ and forwarding to $\{\mathsf{cm}_j\}_{j\in\mathcal{SS}}$. $\mathsf{Rainblind.USign}_2$ only forwards to $\{\sigma_j, c^{\diamond}_{1,j}\}_{j\in\mathcal{SS}}$ and has nothing to do with blinding. $\mathsf{Rainblind.USign}_3$ is the same as the $\mathsf{BS_m.USign}_3$, except for the validation of $\mathbf{T}^{\diamond}_i, \mathbf{z}^{\diamond}_{0,i}, z^{\diamond}_{1,i}$ and the aggregation of $\mathbf{z}^{\diamond}_{0,i}, z^{\diamond}_{1,i}$. Rainblind does not introduce any new blinding factors compared to $\mathsf{BS_m}$, thus we get $\mathsf{Adv}^{\mathsf{TBS\text{-}Blind}}_{\mathcal{A},\mathsf{Rainblind}}(\lambda) = \mathsf{Adv}^{\mathsf{BS\text{-}Blind}}_{\mathcal{B},\mathsf{BS_m}}(\lambda)$.

F Analysis of Practical Efficiency

We assume 128-bit security, i.e., $\lambda = 128$, and that group element $\mathbb{G}$ and field element $\mathbb{Z}_p$ are encoded as 256-bit. Following [20], for $\mathsf{NIPS_{ped}}$, we set $b = 8$, meaning the hash output must have its first b bits zero. For $\lambda = 128$ security, we repeat the protocol $r = \lceil \lambda/b \rceil = 16$ times. For $\Sigma_{\mathsf{ped}} = (\mathsf{Init_{ped}}, \mathsf{Resp_{ped}}, \mathsf{Verify_{ped}})$, each transcript consists of $\mathsf{Commitment} = 8\mathbb{G}$, $\mathsf{Challenge} = 2\mathbb{Z}_p$, and $\mathsf{Response} = 2\mathbb{Z}_p$. Hence, the proof size is $|\pi_{\mathsf{ped}}| = r(\mathsf{Commitment} + \mathsf{Challenge} + \mathsf{Response})$, namely $|\pi_{\mathsf{ped}}| = 16(8 \cdot 256 + 2 \cdot 256 + 2 \cdot 256) = 49152$ bits $= 6$ KB. DS is used to authenticate protocol messages. We can instantiate DS with Schnorr or ECDSA, then at the 128-bit security level and under the same 256-bit encoding convention, the signature size is $|\sigma_{\mathsf{DS}}| = 512$ bits $= 64$ Bytes. We provide an efficiency comparison among classical pairing-free TBS schemes. For details, see Table 2. We assume Snowblind, Snowblind+, and Rainblind use the same DS, with $n = 16$ issuers. The signature-size computation is straightforward. For communication overhead, Snowblind requires $n(2\mathbb{G} + 5\mathbb{Z}_p) + 2n^2\mathbb{Z}_p + n^2 \cdot |\sigma_{\mathsf{DS}}|$, Snowblind+ requires $n(2\mathbb{G} + 6\mathbb{Z}_p) + 3n^2\mathbb{Z}_p + n^2 \cdot |\sigma_{\mathsf{DS}}|$, and Rainblind requires $n(12\mathbb{G} + 7\mathbb{Z}_p) + 2n^2\mathbb{Z}_p + n^2 \cdot |\sigma_{\mathsf{DS}}| + n|\pi_{\mathsf{ped}}|$. The comparison indicates that constructing an adaptive TBS scheme without AGM incurs modest overhead.

Table 2. Efficiency comparison of TBS schemes in the pairing-free setting.

Scheme	Signature Size	Communication Cost
Snowblind [8]	96 Bytes	35.5 KB
Snowblind+ [12]	96 Bytes	44 KB
Rainblind (ours)	416 Bytes	137.5 KB